Python - lecture notes

with example programs from archive ***python_examples***

Tomasz R. Werner

Warsaw, March 23, 2026 14:22

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

Contents

Page

1 Intro . . . 1
1.1 History . . . 1
1.2 General remarks on the language . . . 1
1.3 Python shell . . . 2
1.4 The *IDLE* IDE . . . 5
1.5 The Zen of Python . . . 5
1.6 A few remarks about the style . . . 6
1.7 Built-in types . . . 7
1.7.1 Numeric types . . . 7
1.7.2 Sequence types . . . 7
1.7.3 Sets and dictionaries . . . 9
1.7.4 Other types . . . 10
1.8 Objects and references . . . 10
1.9 Functions . . . 14
1.10 Modules . . . 16
1.11 Help . . . 18
1.12 Keywords (reserved words) . . . 20
2 Flow control statements . . . 21
2.1 *pass* statement . . . 21
2.2 *if* statement . . . 22
2.3 *if-else* (conditional) expression . . . 22
2.4 *while* loop . . . 23
2.5 *for* loop . . . 24
2.6 *match* statement . . . 28
3 Basic data types . . . 34
3.1 *NoneType* type . . . 34
3.2 *ellipsis* type . . . 35
3.3 Typ *NotImplementedType* . . . 35
3.4 Numeric types . . . 35
3.4.1 Integers . . . 36
3.4.2 Boolean type . . . 38
3.4.3 Floating point numbers . . . 39
3.4.4 Complex numbers . . . 41
3.5 Strings (introduction) . . . 41
4 Operators . . . 43
4.1 Assignment (not an)operator . . . 43
4.2 Assignment (true) operator . . . 44
4.3 Operators and operator precedence . . . 45
4.4 Parenthesized expressions . . . 46
4.5 Subscription and slicing . . . 47
4.5.1 Indexing . . . 47
4.5.2 Slices . . . 49
4.6 Call operator . . . 52
4.7 await expression . . . 53
4.8 Exponentiation . . . 53
4.9 Unary operators . . . 54
4.10 Arithmetic operators . . . 54

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

4.10.1 Division operators . . . 55
4.10.2 Modulo operator . . . 56
4.11 Comparisons . . . 57
4.11.1 Arithmetic comparisons . . . 57
4.11.2 Identity comparisons . . . 58
4.11.3 Membership test operators . . . 59
4.12 Boolean *not*, *and* and *or* operators . . . 59
4.13 Conditional expression . . . 61
4.14 Bitwise operators . . . 63
5 Functions . . . 67
5.1 Defining functions. Scopes of variables . . . 67
5.2 Passing arguments . . . 74
5.3 Lambdas . . . 79
5.4 Some "special" functions. . . . 83
5.4.1 *filter* . . . 84
5.4.2 *map* . . . 85
5.4.3 *reduce* . . . 86
5.4.4 *accumulate* . . . 87
5.4.5 *sorted* . . . 88
5.5 Function decorators . . . 91
5.6 Function "overloading": (single dispatch) . . . 99
6 Comprehensions and generators . . . 103
6.1 Comprehensions . . . 103
6.1.1 List comprehension . . . 104
6.1.2 Set comprehension . . . 107
6.1.3 Dictionary comprehension . . . 109
6.2 Generator functions . . . 110
6.2.1 Generator comprehension . . . 113
6.3 Communicating with generators . . . 116
6.4 Pipes of generators and coroutines . . . 121
6.5 Some tools from *itertools* . . . 125
7 List of listings . . . 130
Index . . . 131

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

Chapter 1

Intro

1.1 History

Python was initially developed by Guido van Rossum[1] in the late 1980's in the Netherlands. At that time, he worked in a project, called Amoeba, at the CWI (Centrum voor Wiskunde en Informatica) in Amsterdam. The aim of the project was to build a distributed operating system. As a part of it, a general purpose programming language called ABC was developed by a team that van Rossum belonged to. He was not quite satisfied with the language, so in December 1989 he started working on a new language, based on ABC but without some of its problematic features.



Guido van Rossum

The first version of Python (version 0.9.0) was published in February 1991. It already had classes with inheritance, exceptions, functions and fundamental data types (numbers, lists, dictionaries, strings). It was based on the concept of modules (inspired by Modula-3).[2]

In 1994, discussion forum *comp.lang.python* was created, accelerating the growth of the language and its popularity.

Python 2 was released on 16th of October 2000 and its last version is 2.7. It will not be continued and has been replaced by Python 3, first released on 3rd of December 2008. Python 3 contains many new features making it *not* fully compatible with Python 2 (it contains the *2to3* utility which automates the translation of Python 2 code to Python 3).

Guido van Rossum played the rôle of the lead developer of Python (as the "benevolent dictator for life") until 12 July 2018, when he announced his "permanent vacation" from his responsibilities, which were taken over by a five-member Steering Council. Development of the language is now lead by the non-profit organization Python Software Foundation.

The name of the Python language comes from a BBC television comedy-sketch series *Monty Python's Flying Circus*.

1.2 General remarks on the language

Python is an object-oriented language: everything, including integers and floats, is an object. Even classes and modules are implemented as objects!

Python is a **strongly typed** language, which means that every object has a definite type. At the same time, it is **dynamically typed**, meaning that there is no type-checking of the code prior to running it (Java and C/C++ are *statically* typed; types of all objects have to be known to the compiler and cannot be changed).

Python is an interpreted language. Technically, the code *is* compiled into a form of byte-code before it's executed. However, this process of compilation is "invisible" from

[1] For correct pronunciation of his name, see https://www.youtube.com/watch?v=-pz5vhTdkpA

[2] A language, published in 1988, which was never adopted for industrial use, but heavily influenced Java, Python, C# and other modern languages.

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

the user's point of view, who sees the code immediately executing without a compilation phase.

Python is a general-purpose programming language, not intended for use in any particular domain or environment. There are, of course, some areas where it's less suitable than other languages, mainly because it's quite time and memory consuming (this can be remedied by leveraging Python modules written in C).

It is a multi-paradigm programming language supporting object-oriented programming, structured programming, many features of functional programming (inspired by Lisp, Haskell and ML).

It has a garbage collector, as Java, and, also as Java, a huge standard library in the form of packages and modules that can be imported to user's programs. What's more, there is an official collection of third-party Python software, called ***PyPI***.[1] A special application (***pip***) can be used to easily install these packages. Currently (January 2025), ***PyPI*** contains more than 600 000 packages! They can be used for just about anything: automation, data analysis, managing databases, documentation, building graphical user interfaces, image and sound processing, machine learning, multimedia, networking, scientific computing, system administration, test frameworks, web frameworks and so on...

1.3 Python shell

The best way to start programming in Python is to use the Python shell called REPL — **R**ead what the user types in, **E**valuate what was entered, **P**rint the value of the expression entered, **L**oop back and wait for the next input. It can be launched just by entering ***python*** (on some systems ***python3***) and then entering Python instructions at the prompt (which is `>>>`). If the instruction is an expression, i.e., it has a value (different than `None`), then this value will be printed after pressing `ENTER`:

```
$ python
Python 3.8.10 (default, Jun 22 2022, 20:18:18)
[GCC 9.4.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> x = 7
>>> y = 12
>>> x+y
19
>>> [a**2 for a in range(1,11)]
[1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
>>> exit()
```

By the way: as this example demonstrates, assignment (here `x=7` and `y=12`) is *not* an expression in Python[2] — that's why entering `x=7` does not print any result (see sec. 4.1, p. 43 and 4.2, p. 44).

To exit the REPL, use **exit()**, or Ctrl-D (Linux/Mac), or Ctrl-Z-Enter (Windows).

When typing instructions in REPL (but *not* in Python scripts!), you can use a special "variable" called _ (underscore) — it refers to the last value evaluated and printed:

[1] Python Package Index: https://pypi.org/

[2] In C/C++/Java, it has the type and value of the (reference to) the variable on the left-hand side after assignment.

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

```
>>> a = 6
>>> a
6
>>> b = _ + 3  # _ is 6
>>> b
9
>>> _ * 8      # _ is 9
72
```

In order to enter a piece of data from the keyboard, you can use – in both REPL and scripts – the **input** function, optionally passing as argument a prompt to be used:

```
>>> whatever = input("Enter whatever: ")
Enter whatever: Hello
>>> whatever
'Hello'
>>> noprompt = input()
there was no prompt!
>>> noprompt
'there was no prompt!'
```

The function returns whatever was typed as a string; if what you need is a number, you have to convert this string to **int** or **float** (the latter corresponds to **double** in C/C++/Java):

```
1  >>> i = input("Enter an int: ")
2  Enter an int: 123
3  >>> i, type(i)
4  ('123', <class 'str'>)
5  >>> i = int(i)
6  >>> i, type(i)
7  (123, <class 'int'>)
8  >>>
9  >>> f = input("Enter a float: ")
10 Enter a float: 123.45
11 >>> f, type(f)
12 ('123.45', <class 'str'>)
13 >>> f = float(f)
14 >>> f, type(f)
15 (123.45, <class 'float'>)
16 >>> print(i, f)
17 123 123.45
18 >>> print("Numbers are " + str(i) + " and " + str(f))
19 Numbers are 123 and 123.45
```

Some explanations:

- Line 3: two (or more) comma-separated values create a tuple (see sec. ??, p. ??) which is then printed (line 4) as parenthesized sequence of these values.
- Line 3: the **type** function (actually, a constructor of class **type**) returns an object representing the type (class) of the argument — what is printed is its string representation.

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

- Lines 5 and 13: functions **int** and **float** (actually, constructors of the corresponding classes) return numbers represented by strings passed as argument.
- Lines 5 and 13: note that we assign the results of function calls to the same variables (i and f, respectively). This variables (references) now refer to objects of different types: they referred to objects of type **str** (string), and now they refer to **int** and **float**, respectively (compare lines 4 with 7 and 12 with 15).

In lines 16 and 18, we used the **print** function. It accepts any number of comma-separated arguments and prints their values converted to strings using a space as the separator. This can be changed by supplying the sep argument:

```
>>> a = 7
>>> b = 1.5
>>> c = 'Hello'
>>> print(a, b, c)
7 1.5 Hello
>>> print(a, b, c, sep=' - ')
7 - 1.5 - Hello
```

By default, **print** (like **println** in Java) adds new-line character at the end. It also may be changed by supplying the end argument (it can be an empty string, and then nothing will be added):

```
a = 123
b = 123.45
c = 'Hello'
print(a, b, sep=' and ', end=' and then ')
print(c)
```

prints

```
123 and 123.45 and then Hello
```

Note that a and b are numbers, not strings, but they were converted to strings automatically.

As in Java, you *can* concatenate strings using the + operator, but both operands must then be strings. In Java, assuming k is an integer, something like "k = " + k is correct: k will be automatically converted to string and concatenated with "k = ". To do the same in Python, first you have to convert k to string for the concatenation to work:

```
>>> k = 3
>>> print("k = " + str(k))
k = 3
>>> print("k = " + k)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: can only concatenate str (not "int") to str
```

Python uses (and its interpreter enforces) a little strange convention of writing the code: blocks (of loops, **if**s, function and class definitions, ...) are marked by indentation and by indentation *only* (no braces, begin...end and such)! The amount of indentation is in principle arbitrary, but the convention is to use always 4 spaces

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

— do *not* use TABs![1] This also applies to REPL. For example, the following loop has a body consisting of two lines. Note that after beginning of the loop,[2] the REPL expects its body, so the prompt changes from >>> to ... . Both lines of the body have to be indented by the same amount of spaces (4 is recommended); after the last line of a block, just enter a blank line in order to revert to the indentation level used before the loop

```
>>> for n in [2, 6, 8, 10]:
...     m = 2*n + 1
...     print(m)
...
5
13
17
21
```

1.4 The IDLE IDE

You can also use an IDE which usually comes with standard Python installations: it is called ***IDLE*** (Python's Integrated Development and Learning Environment) and can be launched just by typing ***idle*** in the console or
`>>> import idlelib.idle`
in the REPL. It is a simple, lightweight application but for writing small programs and for experimenting may be quite sufficient. It has the following features:

- coded in 100% pure Python, using the standard ***tkinter*** package to create graphical user interface;
- cross-platform: works mostly the same on Windows and Linux/MacOS;
- embedded Python shell window (interactive interpreter) with colorizing of code input, output, and error messages;
- multi-window text editor with multiple undo, Python colorizing, smart indent, call tips, auto completion, and other features;
- search within any window, replace within editor windows, and search through multiple files (***grep***);
- debugger with persistent breakpoints, stepping, and viewing of global and local namespaces;
- configuration, browsers, and other dialogs.

For details, see the manual[3].

1.5 The Zen of Python

Every Pythonista[4] knows the so called Zen of Python — a collection of 20 "guiding principles" for writing programs in Python (published by Tim Peters in 1999) only 19 of which are known. Zen of Python can be found as PEP 20[5] or viewed by typing `import this` in the Python shell (REPL):

[1] All Python-oriented editors will automatically insert 4 spaces when you press TAB at the beginning of a line.
[2] Instructions requiring a body — like loops or **if**s — end with a colon.
[3] https://docs.python.org/3/library/idle.html
[4] Someone who uses (and is a fan of) the Python programming language.
[5] http://peps.python.org/pep-0020

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

```
>>> import this
The Zen of Python, by Tim Peters

Beautiful is better than ugly.
Explicit is better than implicit.
Simple is better than complex.
Complex is better than complicated.
Flat is better than nested.
Sparse is better than dense.
Readability counts.
Special cases aren't special enough to break the rules.
Although practicality beats purity.
Errors should never pass silently.
Unless explicitly silenced.
In the face of ambiguity, refuse the temptation to guess.
There should be one - and preferably only one - obvious way to do it.
Although that way may not be obvious at first unless you're Dutch.
Now is better than never.
Although never is often better than *right* now.
If the implementation is hard to explain, it's a bad idea.
If the implementation is easy to explain, it may be a good idea.
Namespaces are one honking great idea - let's do more of those!
```

By the way, PEPs[1] (Python Enhancement Proposals) are documents describing new features proposed for Python and also various aspects of Python, like its design and style.

1.6 A few remarks about the style

Python is a high-level language and hence the Python programs are just text files. It can be written in many ways and as long at it is syntactically correct the interpreter does not care. However, it is important, for the author and for all who will ever read the program, to use a consistent style which will make the code easy to read and understand. A common style facilitates collaboration on projects and makes it easier to change them later. The famous PEP 8[2] describes in details the recommended (but not enforced) rules of writing clear code in Python (it is continuously updated as the language evolves). Every Python developer should read the whole document; below, we just mention some of these rules:

- Lines shouldn't be longer that 79 characters.
- Continuation lines should be indented by four extra spaces.
- Function and class definitions should be separated by two blank lines.
- Methods in a class should be separated by one blank line.
- Don't put spaces around indices, function calls, or keyword argument assignments.
- Spaces should be used around operators and after commas.
- Comments (from # to the end of line) should be on a line of their own.[3]

[1] https://peps.python.org

[2] http://peps.python.org/pep-0008

[3] There are no multi-line comments in Python. Instead, you can use a multi-line string (enclosed in triple apostrophes or quotation marks) which you don't print or assign to any variable; because

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

- A docstring (which will be discussed later) should be defined for all non-trivial functions, classes and modules.
- It is recommended to use CamelCase (starting with a capital letter) for the names of classes and exceptions.
- Use lower_case_with_underscores for the names of functions, variables, and attributes; the camelCase is also acceptable (but starting with a lowercase letter).
- Names of variables which are meant to be constant are written in ALL_CAPITALS with underscores.
- The first parameter of (non-static) methods, which is the reference to the object the method is invoked on, should be named self.[1]
- The first parameter of class methods should be named cls.
- Imports should be in sections in the following order: standard library modules, third-party modules, your own modules. Imports in each subsection should be in alphabetical order.

1.7 Built-in types

We will describe Python types in a separate section (see sec. 3, p. 34); here we will just briefly enumerate some of them without too many details.

1.7.1 Numeric types

There are only three built-in numeric types: int, float and complex. They can be freely mixed in arithmetic operation — even integers have well defined real *and imaginary* parts! Objects of numeric types are immutable.

Type int represents integer values with practically unlimited values, negative or positive.

Type float describes real numbers (as a matter of fact, an "infinitely tiny" subset of rational numbers, because of finite representation on 64 bits). They are implemented as doubles (*not* floats!) known from C/C++/Java languages.

Type complex describes complex numbers represented as pairs of floats — their real and imaginary parts.

1.7.2 Sequence types

There are built-in types of objects which represent sequences of objects. These are list, tuple and range. As in Java, sequences and collections in Python are really sequences *pointers,* but not directly to objects (as in Java) but to *references* to objects. These references, in turn, are C-structures containing several pointers (more on this later).

Type list corresponds somehow to ArrayList from Java or vector in C++. It's a list of references to objects of any types (also to other lists or tuples). It is denoted by comma-separated list enclosed in square brackets. Lists are mutable — we can add of remove elements or replace a reference to an object with a reference to another object without changing the reference to the list itself and leaving ID number the same:

you don't do anything with it, it will be ignored by the interpreter.

[1] Note that in C++/Java this parameter is *not* mentioned on the parameter list at all, so there is no way to give it a name and therefore its name is fixed once for all as this.

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

```
mylist = []                # empty list
print(type(mylist))
print(id(mylist))
mylist.append(1)
mylist.append([11, 22])
mylist.extend([33, 44])
print(mylist)
print(id(mylist))
del mylist[3]
print(mylist)
print(id(mylist))
```

prints

```
<class 'list'>
140144996134208
[1, [11, 22], 33, 44]
140144996134208
[1, [11, 22], 33]
140144996134208
```

Type **tuple** represents an immutable sequence of references to objects of any type. Immutability means that you can neither add or remove elements of a tuple, nor replace a reference which is its element by another reference. However, if this is the reference to a mutable object (as a list), this object *can* be modified. Tuples are denoted by comma-separated lists enclosed in round parentheses (not always required) — in one-element tuple, its sole element must be followed by a comma:

```
mytuple = (5, 'Hello', [1, 2, 3], (4,))
print(type(mytuple))
print(type(mytuple[3]))
print(mytuple)
mytuple[2].pop(1)
print(mytuple)
mytuple[1] = 'World'
```

prints

```
<class 'tuple'>
<class 'tuple'>
(5, 'Hello', [1, 2, 3], (4,))
(5, 'Hello', [1, 3], (4,))
Traceback (most recent call last):
File "/usr/lib/python3.10/idlelib/run.py",
        line 578, in runcode
  exec(code, self.locals)
File "/home/werner/p.py", line 7, in <module>
  mytuple[1] = 'World'
TypeError: 'tuple' object does not support item assignment
```

Type **range** represents an immutable sequence of integers which is an arithmetic progression. It is mainly used in loops, but is used in numerous other situations.

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

Constructor of **range** takes the value of the first element, the limit such that all elements must be smaller (or bigger for negative difference) than that, and the difference between consecutive elements (step). Constructor with one argument n is equivalent to three arguments: 0, n and 1. Note that ranges themselves are *not* collections: they just yield elements "on demand":

```
myrange1 = range(5, 10, 2)
print(myrange1)
print(list(myrange1))
print(list(range(24, -2, -5)))
print(list(range(5)))
```

prints

```
range(5, 10, 2)
[5, 7, 9]
[24, 19, 14, 9, 4, -1]
[0, 1, 2, 3, 4]
```

1.7.3 Sets and dictionaries

Type **set** represents sets, as in Java or C++. It is a mutable collection of references to objects of any type. The implementation is based on hashing,[1] so elements must be immutable (more precisely, hashable). There is also the **frozenset** type: it represents immutable (and hashable) set, so it can be used as an element of another set. For example:

```
myset = {1, 2, frozenset(['Hello', 'World']), 'Paris'}
print(type(myset))
print(2 in myset)
anotherset = {3, 4}
sum  = myset | anotherset          # sum
prod = myset & set([1, 2, 3, 4])   # product
print(myset-anotherset)            # difference
print(anotherset)
print(sum)
print(prod)
```

prints

```
<class 'set'>
True
{frozenset({'World', 'Hello'}), 1, 2, 'Paris'}
{3, 4}
{1, 2, 3, 'Paris', 4, frozenset({'World', 'Hello'})}
{1, 2}
```

Type **dict** corresponds to **LinkedHashMap** in Java (maps are called dictionaries in Python). It represents a set of key/value pairs, where keys must be unique, immutable and hashable. For example:

[1] As **HashSet** in Java or **unordered_set** in C++.

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

```
mymap = {}      # empty dictionary, not set
mymap['France'] = 'Paris'
mymap[2] = [1, 2, 3]
mymap[2].append(4)
print(2 in mymap)
anothermap = {'Spain' : 'Madrid', 2 : complex(1, 2)}
mergedmap = mymap | anothermap
print(mymap)
print(anothermap)
print(mergedmap)
```

prints

```
True
{'France': 'Paris', 2: [1, 2, 3, 4]}
{'Spain': 'Madrid', 2: (1+2j)}
{'France': 'Paris', 2: (1+2j), 'Spain': 'Madrid'}
```

Implementation of the Python language itself is based on dictionaries.

1.7.4 Other types

There are a few other built-in types that we will encounter later. The most important of them is, of course, **str** representing strings, but we will postpone details for special chapter.
Note, however, that there is no type corresponding to **char** from other languages — a character is just a one-letter string of type **str**.

1.8 Objects and references

Names of variables that we use in Python are in fact names of pointers[1] to the "true" *references* to objects. These references are really C-structures in memory holding pointers to objects representing the type of the current value and to structures representing this value itself, reference counter, and two more pointers needed by the garbage collector.
Structures representing values have also internal identifier, or ID number (an integer, usually just the memory address of the object) — it is guaranteed that **no two objects accessible at the same time may have the same ID.** During the execution of the program, the same reference (and thus name) may change its interpretation and refer to another value, of different type and ID. In other words, Python has strongly typed objects, but untyped variable names.

There is a built-in function, **id**, which, given a reference to an object, returns, as an integer, the ID number of the object referred to by this reference. In the following example, we also use another built-in function, **type**, which returns the object representing the type of the value pointed to by reference passed as the argument:

```
>>> x = 3
>>> y = 7
>>> id(x), id(y)
(9353376, 9353504)          # 1
```

[1] These *pointers* are also called references, as in Java.

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

```
>>> x = y                    # 2
>>> id(x), id(y)
(9353504, 9353504)           # 3
>>> y = 'Python'             # 4
>>> id(x), id(y)
(9353504, 139733635191952) # 5
>>> type(x), type(y)         # 6
(<class 'int'>, <class 'str'>)
```

As one can see (line 1), identities of objects referred to by references `x` and `y` are different: 9353376 and 9353504, respectively. After the assignment[1] `x = y` (line 2), both `x` and `y` refer to exactly the same object (line 3) of type **int** and with value 7. What happened to the object with identity 9353376? It is lost; it's not accessible any more and will be removed from memory by the garbage collector.
In line 4, we assign another value to `y` — now it's a string. As we can see from the output, the ID, type (and value) of `x` and `y` are now different.

Note that even literal values are treated as references to objects, so one can call a method on them; for example, the type (class) **int** has the **bit_length** method which returns number of bits needed to represent a given value, while strings (class **str**) have the **upper** method

```
>>> (1).bit_length()
1
>>> (255).bit_length()
8
>>> (256).bit_length()
9
>>> 'paris'.upper()
'PARIS'
```

(The parentheses around the **int** literals are necessary; otherwise the dot would be interpreted as a decimal dot.)

Programs in Python, like in other languages, can be written in any text editor. To run it, one can simply type

```
python3 python_program.py
```

or just

```
python python_program.py
```

depending on configuration.
A Python program can also be made directly executable by adding, on Linux/Mac, the so called 'shebang' as the first line and setting the mode of the file to 'x' (executable). It was also a good practice to specify explicitly, in the second line, the encoding of the source file. However, in Python 3, it is UTF-8 by default, so many programmers do not do it anymore:

[1] What we assign here are references, not values.

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

Listing 1 AAB-First/First.py

```
1  #!/usr/bin/env python
2  # -*- coding: utf-8 -*-
3
4  # a comment
5  v = 1.25                                    ❶
6  print(id(v))
7  if 1 <= v <= 2:
8      print('v^2 =', v**2, end = '; ')
9      print('v^3 =', v**3, end = '; ')
10     print('v^4 =', v**4)
11     v = "now I'm a string!"                 ❷
12     print(id(v))
13 print('This is v:', v)
```

The program prints

```
20164608
v^2 = 1.5625; v^3 = 1.953125; v^4 = 2.44140625
140529498173656
This is v: now I'm a string!
```

Note that the object pointed to by `v` initially (❶) represents a **float** with identity 20164608, but after modification (❷), the same reference `v` points to another object with different identity (140529498173656) and different type (**string**). The object with its ID equal to 20164608 is now lost.
Note also, that exponentiation is denoted, as in Fortran, by double asterisk (`**`) – never by `^`!
In this example, not only the value, but also the type has been modified. However, objects of all numerical types are in fact *not* modifiable (they are *immutable*), so assigning a new value to a number actually creates a new object:

```
>>> x = 1
>>> id(x)
9353312
>>> x = 2
>>> id(x)
9353344
```

There are many immutable types in Python; beside numerical types, a very important example is the **str** type representing strings:

```
>>> x = 'John'
>>> y = x
>>> id(x), id(y)
(139733656059312, 139733656059312)
>>> x = x + ' and Mary'
>>> x, y
('John and Mary', 'John')
>>> id(x), id(y)
(139733654847152, 139733656059312)
```

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

As we can see, after modification, x represents now a new object with different ID.

On the other hand, objects of many other types *are* modifiable (they are *mutable*). An important example is **list**. A list is, well... a list of *references* to objects (of any type). We can modify a list, for example, by appending a new element. However, the object representing this list will remain the same and its ID will *not* be changed. This is illustrated by the following example:

Listing 2 AAC-Refs/Refs.py

```
1  #!/usr/bin/env python3
2  # -*- coding: utf-8 -*-
3
4  from copy import deepcopy
5
6    # a and be are two names of the same objects
7  a = [1, 2, [3, 5]]
8  b = a
9  print(b is a)        # True
10 a[2][1] = 4
11 print(b, '\n')       # [1, 2, [3, 4]]
12
13   # shallow copy: lists distinct, elements are copied
14   # but these are references referring to the same objects!
15 c = list(a)
16 c.append(5)
17 print(c is a)        # False
18 print(c[2] is a[2]) # True
19 a[2][1] = 6
20 print(a)             # [1, 2, [3, 6]]
21 print(c, '\n')       # [1, 2, [3, 6], 5]
22
23   # deep copy: lists distinct, elements too (recursively)
24 d = deepcopy(a)
25 d.append(5)
26 print(d is a)        # False
27 print(d[2] is a[2]) # False
28 a[2][1] = 7
29 print(a)             # [1, 2, [3, 7]]
30 print(d)             # [1, 2, [3, 6], 5]
```

In line 7, we create a list a: its first two elements are references to (immutable) integers and the third is the reference to another list. After b = a, the two names, b and a, refer to the same list object. We make a change using the name a (line 10): a[2] is the third element of the list (indexing is zero-based) which is the reference to the (mutable) list [3, 5] and, consequently, a[2][1] is its second element (reference to integer 5). Then we print b: as the output shows, the element b[2][1] has been changed, as a and b denote in fact the same object.
This can be seen from the output of the line 9: the **is** operator compares identifiers (addresses) and yields **True** only when we compare exactly the same object (there is also == operator which compares *values* — see sec. 4, p. 43).

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

In line 15, we create a new list `c`, passing to the constructor (more on lists and constructors later, see sec. ??, p. ?? and sec. ??, p. ??). Now `c` and `a` are different list objects (line 17). However, elements of `c` are copies of elements of `a` and these elements are in fact *references*, not values, so they point to the same objects; that's why `print(c[2] is a[2])` prints **True**. This element is itself modifiable (as it's a list), so changing one of its element (line 19) affects both `a` and `c`. Such coupling of lists is often undesirable — in order to create independent copy of a list, one can use **deepcopy** function form the ***copy*** module (line 24).

1.9 Functions

Functions will be described in more detail later (see sec. 5, p. 67). Here, we will just show an example illustrating creating and using a simple function.

As everything else, functions in Python are represented by objects. Definition of a function looks as shown in the example below. After the **def** keyword, we specify the name of a reference that we want to assign the function to, then a list of parameters (without types!) in parentheses and the colon indicating that what follows will be a block of instructions which define the function. The block that follows has to be indented, because, as we know, there is no other way to indicate where a compound instruction begins and where it ends. As in other languages, if a function returns a nontrivial result, the last executed statement must be **return**. All functions in Python return something — if there is no **return**, the special value **None** will be returned.

Notice that in line 3 we define a function (an object representing it) and at the same time we bind the object representing the function to a name, **adder** in our case. In python, this is an *executable* statement, like assignment. The name **adder** is not the name of this function that has to be used to invoke it. Rather, it's the name of a *reference* to this function. If we want, we can copy this reference to another — we do it on line 17

Listing 3 AAY-Funcs/funcs.py

```
1  #!/usr/bin/env python
2
3  def adder(a, b):          # definition + assignment
4      return a + b
5
6  print(type(adder))        # type
7
8  rn = adder(7,4)           # adding numbers
9  print(rn, type(rn))
10
11 rs = adder('Monte ', 'Carlo')  # 'adding' strings
12 print(rs, type(rs))
13
14 print(adder.__str__())  # invoking method on a function!
15
16 print(id(adder))
17 a = adder                # copying reference, now a is adder
18 print (id(a))
```

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

```
19 print(a is adder)        # a and adder are the same thing
20
21 adder = a(1.25, 3.75)    # now adder becomes a float
22 print('Now adder is', adder, type(adder))
23
24 print("But 'a' is still our function!", a(2.5, 3.5))
25
26 print('='*30 + '\nAnd now list of attributes:')
27 print(str(dir(a)) + '\n' + '='*30)
28
29 print('But attribute a.__name__ is still ' + a.__name__)
30 a.__name__ = 'whatever'
31 print('No problem, now it is ' + a.__name__)
```

and from now on references **a** and **adder** refer to exactly the same object and are equivalent. We can even change the type of the reference **adder**, so now it refers to a **float** (line 21), and we still have access to the function through the reference **a** (line 24).

The program prints

```
<class 'function'>
11 <class 'int'>
Monte Carlo <class 'str'>
<function adder at 0x7f078af0bd90>
139670372531600
139670372531600
True
Now adder is 5.0 <class 'float'>
But 'a' is still our function! 6.0
==============================
And now list of attributes:
['__annotations__', '__builtins__', '__call__', '__class__',
'__closure__', '__code__', '__defaults__', '__delattr__',
'__dict__', '__dir__', '__doc__', '__eq__', '__format__',
'__ge__', '__get__', '__getattribute__', '__globals__',
'__gt__', '__hash__', '__init__', '__init_subclass__',
'__kwdefaults__', '__le__', '__lt__', '__module__',
'__name__',  '__ne__', '__new__', '__qualname__', '__reduce__',
'__reduce_ex__', '__repr__', '__setattr__', '__sizeof__',
'__str__', '__subclasshook__']
==============================
But attribute a.__name__ is still adder
No problem, now it is whatever
```

For people raised on C/C++/Java, it may seem strange that nowhere in the definition of the function have the types of parameters and of the return value been specified. In Python this is not only not required, but actually there is no way to do it![1] This means that we can pass to **adder** any two values of arbitrary types, as long

[1] There are methods to indicate these types, but only for human readers or third-party applications; the interpreter will just ignore this information anyway.

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

as the + operator used in its definition works![1]
Being objects, functions have attributes: actually, quite many of them (even *methods* – line 14). We can see their list by invoking the **dir** function, as on line 27. Among these attributes, there is also `__name__`, which is still `adder` (line 29). But this is just a string, and if it bothers us, we can replace it with whatever we wish (what we do on line 30).

1.10 Modules

As in Java or C/C++, in order to use the standard library, we have to somehow "import" its relevant parts to our program.
In Java, we often use the **import** keyword to do it. As the matter of fact, **import** in Java is not necessary at all, it just makes the code easier to type and read. Something like:

```
import java.util.List;
```

essentially just tells the compiler *"whenever you encounter the name* `List`*, replace it by its full, qualified name* `java.util.List`*"*. If you are fine with typing `java.util.List` instead of just `List`, you don't need any imports.
In C/C++ we have **#include** preprocessor directive. This is different:

```
#include <map>
```

physically includes the contents of file ***map*** and sends it together with our code to the compiler.[2]
Python is an interpreted language, so there is no separate compilation phase; everything happens at runtime. In Python, we use **import** keyword, but it's more like **#include** from C/C++ than **import** from Java.
For example,

```
import math
```

imports (and *executes*) module (file) ***math***. Modules correspond to normal Python source files with extension ***.py***. They are looked for in directories from the list of paths, which we can see like this[3]

```
>>> import sys
>>> sys.path
['', '/usr/lib/python38.zip', '/usr/lib/python3.8',
 '/usr/lib/python3.8/lib-dynload',
 '/home/werner/.local/lib/python3.8/site-packages',
 '/usr/local/lib/python3.8/dist-packages',
 '/usr/lib/python3/dist-packages']
```

(empty string at the beginning means *current directory*.) When found, the file is read in by the Python interpreter *and executed.* Most often, modules contain only definitions of classes, functions, constants, etc., but if there are, e.g., **print** statements there, we will see their output.

[1] This is an example of the so called *duck typing*.
[2] Compilers use clever tricks to make the process more efficient and to avoid recompiling the same code over and over again, but these are technicalities.
[3] We can add more directories to this list, see sec. ??, p. ??.

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

The names of all entities defined in the module will be put into the namespace whose name is the same as the name of the module, so to access them, we will have to qualify them with the name of the namespace. For example, module ***math*** contains definition of **sqrt** function. To access it, we type **math.sqrt**

```
>>> import math
>>> math.sqrt(234.567)
15.315580302424065
```

Sometimes it's more convenient (or is expected by tradition in a given community of the users) to give a module another name (alias) under which we will refer to it

```
>>> import math as m
>>> m.sqrt(234.567)
15.315580302424065
```

We can import only selected definitions from a module, and then their names don't have to be qualified:

```
>>> from math import sqrt, factorial
>>> sqrt(999.99)
31.6226184874055
>>> factorial(45)
119622220865480194561963161495657715064383733760000000000
```

If we want to save on typing long names, or to avoid name clashes, we can rename entities imported from a module:

```
>>> from math import sqrt as root, factorial as fac
>>> root(87)
9.327379053088816
>>> fac(12)
479001600
```

Finally, we can import all names from a module, although it's not recommended, because then it's sometimes not so easy to tell what comes from which module and may lead to name clashes:

```
>>> from math import *
>>> from time import *
>>> gcd(3219, 1508)
29
>>> factorial(23)
25852016738884976640000
>>> log(999, pi)
6.033523597161682
>>> 4*atan(1.0)
3.141592653589793
>>> strftime("%a, %d %b %Y %H:%M:%S +0000", gmtime())
'Mon, 19 Sep 2022 17:21:38 +0000'
```

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

1.11 Help

In REPL, we have access to Python help system. We can get help for modules, classes, functions, etc.

```
>>> help()

Welcome to Python 3.10's help utility!

If this is your first time using Python, you should definitely check out
the tutorial on the internet at https://docs.python.org/3.10/tutorial/.

Enter the name of any module, keyword, or topic to get help on writing
Python programs and using Python modules.  To quit this help utility and
return to the interpreter, just type "quit".

To get a list of available modules, keywords, symbols, or topics, type
"modules", "keywords", "symbols", or "topics".  Each module also comes
with a one-line summary of what it does; to list the modules whose name
or summary contain a given string such as "spam", type "modules spam".
```

Let's try `help(math)`:

```
    Help on built-in module math:

    NAME
        math

    DESCRIPTION
        This module provides access to the mathematical functions
        defined by the C standard.

    FUNCTIONS
        acos(x, /)
            Return the arc cosine (measured in radians) of x.

        acosh(x, /)
            Return the inverse hyperbolic cosine of x.

        asin(x, /)
            Return the arc sine (measured in radians) of x.
        ...
```

and **sin** function from this module `help(math.sin)`

```
    Help on built-in function sin in math:

    math.sin = sin(x, /)
        Return the sine of x (measured in radians).
```

(type 'q' to leave the help system and return to the REPL.)

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

To explore properties of modules, classes, functions, etc., we can also use the built-in function **dir**[1]. Called without arguments, it returns the list of names in the current local scope. With an object as the argument, returns a list of attributes for that object. If the object is a module object, the list contains the names of the module's attributes. If the object is a type or class object, the list contains the names of its attributes, and recursively of the attributes of its base classes. Otherwise, the list contains the object's attributes' names, the names of its class's attributes, and recursively of the attributes of its class' base classes.
For example:

```
>>> dir()                # attributes in local scope
['__annotations__', '__builtins__', '__doc__', '__file__',
'__loader__', '__name__', '__package__', '__spec__']
>>>
>>> import math
>>> dir()                # now 'math' added to the list
['__annotations__', '__builtins__', '__doc__', '__file__',
'__loader__', '__name__', '__package__', '__spec__', 'math']
>>> __name__             # name of this module (by default)
>>>
'__main__'
>>>
>>> dir(math)            # attributes of module object 'math'
['__doc__', '__loader__', '__name__', '__package__',
'__spec__', 'acos', 'acosh', 'asin', 'asinh', 'atan',
'atan2', 'atanh', 'ceil', 'comb', 'copysign', 'cos',
'cosh', 'degrees', 'dist', 'e', 'erf', 'erfc', 'exp',
'expm1', 'fabs', 'factorial', 'floor', 'fmod', 'frexp',
'fsum', 'gamma', 'gcd', 'hypot', 'inf', 'isclose',
'isfinite', 'isinf', 'isnan', 'isqrt', 'ldexp', 'lgamma',
'log', 'log10', 'log1p', 'log2', 'modf', 'nan', 'perm',
'pi', 'pow', 'prod', 'radians', 'remainder', 'sin', 'sinh',
'sqrt', 'tan', 'tanh', 'tau', 'trunc']
>>>
>>> type(math.gamma)     # what's gamma?
<class 'builtin_function_or_method'>
>>> dir(math.gamma)      # attributes of gamma function
['__call__', '__class__', '__delattr__', '__dir__',
'__doc__', '__eq__', '__format__', '__ge__',
'__getattribute__', '__gt__', '__hash__', '__init__',
'__init_subclass__', '__le__', '__lt__', '__module__',
'__name__', '__ne__', '__new__', '__qualname__',
'__reduce__', '__reduce_ex__', '__repr__', '__self__',
'__setattr__', '__sizeof__', '__str__', '__subclasshook__',
'__text_signature__']
>>> math.gamma.__name__
'gamma'
>>> math.gamma.__doc__   # docstring for gamma function
'Gamma function at x.'
>>> type(math.pi)        # what's pi?
```

[1] https://docs.python.org/3/library/functions.html?highlight=dir#dir

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

```
<class 'float'>
>>> math.pi              # value of pi
3.141592653589793
```

Names of many of these attributes begin and end with double underscore. These are "special" names, often of the so called "magic methods", which we will cover later (see sec. ??, p. ??).

1.12 Keywords (reserved words)

As in other languages, some words (names) are reserved for the language itself (like for, while) and cannot be used as names of variables, functions, classes, etc.
These are

Table 1: Python reserved words

False	await	else	import	pass
None	break	except	in	raise
True	class	finally	is	return
and	continue	for	lambda	try
as	def	from	nonlocal	while
assert	del	global	not	with
async	elif	if	or	yield

In addition to keywords, that we cannot use for our own purposes, there are also so-called "soft" keywords, which have special meaning only in certain contexts; currently, there are four such soft keywords: _ (underscore), case, match, and type.

Of course, Python is still evolving and the list of "soft" and "hard" keywords may change. However, we always can easily check it. Current information on keywords can be obtained interactively leveraging the *keyword* module:

```
Python 3.13.0 (main, Oct  8 2024, 08:51:28) [GCC 11.4.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> import keyword
>>> keyword.iskeyword('for')
True
>>> keyword.iskeyword('case')
False
>>> keyword.issoftkeyword('case')
True
>>> keyword.kwlist
['False', 'None', 'True', 'and', 'as', 'assert', 'async',
 'await', 'break', 'class', 'continue', 'def', 'del', 'elif',
 'else', 'except', 'finally', 'for', 'from', 'global', 'if',
 'import', 'in', 'is', 'lambda', 'nonlocal', 'not', 'or',
 'pass', 'raise', 'return', 'try', 'while', 'with', 'yield']
>>> keyword.softkwlist
['_', 'case', 'match', 'type']
```

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

Chapter 2

Flow control statements

2.1 *pass statement*

The **pass** statement plays the rôle of an empty statement (like semicolon in C/-C++/Java). The statement does nothing at all, but is useful when syntax requires a statement, but we don't want or need anything.
For example, we can define class which describe some types of exceptions. What we need is just a type of this exception; we don't need to define any methods or fields. We can then leave the definition of such classes empty, as in the example below

Listing 4 AAM-Pass/Pass.py

```
#!/usr/bin/env python

from math import log10

class ZeroArg(Exception):
    pass

class NegArg(Exception):
    pass

class ZeroDenom(Exception):
    pass

def f(num, denom):
    if denom == 0: raise ZeroDenom
    elif num == 0: raise ZeroArg
    elif num < 0:  raise NegArg
    return log10(num)/denom

for n, d in ( (-10, 2), (1000, 2), (100, 0), (0, 2) ):
    try:
        res = f(n, d)
        print("({:4d}, {}) -> {}".format(n, d, res))
    except Exception as e:
        print("({:4d}, {}) -> exception {}"
              .format(n, d, e.__class__.__name__))
```

which prints

```
( -10, 2) -> exception NegArg
(1000, 2) -> 1.5
( 100, 0) -> exception ZeroDenom
(   0, 2) -> exception ZeroArg
```

2.2 *if statement*

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

The if statements is similar to analogous constructs known from other languages. The conditions (logical expressions) after if, elif and else do *not* require to be enclosed in parentheses. Remember also to add a colon after each of them:

Listing 5 AAG-Iffs/Iffs.py

```
#!/usr/bin/env python3

number = int(input("Enter an int "))
if number < 0:
    print("negative numbers do not count")
elif number == 0:
    print("Nothing")
elif number == 1:
    print("One")
elif number == 2:
    print("Two")
else:
    print("Many")
```

Of course, the elif and else clauses are optional, and an else clause, if it has been used, must appear as the last one.

2.3 if-else (conditional) expression

The if-else conditional is not a flow-control statement, but is somehow related to if statement, so we will mention it here (for a more detailed description, see sec. 4.13, p. 61).

It corresponds to conditional expression denoted in C/C++/Java with ternary operator ?:, but the syntax is somewhat different. It's an *expression*, what means it yields a value which can be assigned to a variable or used as a part of another expression. The syntax is

```
expr_if_true if condition else expr_if_false
```

what yields the value of expr_if_true if the condition evaluates to True and expr_if_false otherwise. As is the case with conditional expressions (?:) in other languages, it is short-circuited, i.e., depending of the logical value of condition only expr_if_true or only expr_if_false is evaluated

```
>>> a, b = 5, 0
>>> c = a//b if a > 6 else 3*a
>>> c
15
>>> d = a//b if a < 6 else 3*a
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ZeroDivisionError: integer division or modulo by zero
```

(in the first case, the condition was false and the expression a//b was not evaluated at all.)

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

Conditional expressions can be nested, although this should be avoided, as it leads to rather vague expressions

```
>>> for t in (20, 50, 90):
...     print("green" if t < 40 else "yellow" if t < 80 else "red")
...
green
yellow
red
```

2.4 while loop

The **while** looks as in other languages. The only difference is that it can optionally contain also an **else** clause which will be executed only when the loop has been exhausted without **break**: if the flow of control exits a loop because of **break**, the **else** clause will *not* be executed.[1] As in other languages, **break** inside a loop means 'go out of this loop' and **continue** means 'go to the next iteration, without completing this one'.

These constructs are illustrated by the program below. Note that the last line will never be executed, because you have to use **break** to exit the main loop:

Listing 6 AAJ-While/While.py

```
#!/usr/bin/env python

while True:
    number = int(input("Enter an int "))
    if   number == 0:
        break
    if number <= 1:
        print("Your number must be >= 2.")
        continue
    if number in (2, 3, 5, 7):
        print(str(number) + ' is prime.')
    elif number % 2 == 0:
        print(str(number) + ' is not prime.')
    else:
        i = 3
        while i*i <= number:
            if number % i == 0:
                print(str(number) + ' is not prime.')
                break
            i += 2
        else:
            print(str(number) + ' is prime.')
else:
    print('This will never be printed')
```

[1] The idea has been borrowed from Dijkstra's Guarded Command Language (GCL).

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

2.5 for loop

The form of the for loop resembles the so called "for-each" (or "ranged for") loop from other languages. The syntax is

```
for target_list in expression_list:
    body_of_the_loop
[else:
    block_of_code]
```

where the else clause is optional and is executed only when the loop has been exhausted without encountering any break or return statement (as is also the case for the while loop). The expression_list is evaluated first (and once only) — it should yield an **iterable** object.

Iterables are objects which provide the __iter__ function which in turn returns an **iterator**.[1] Instead of calling __iter__ on an object, one can use build-in function iter with an iterable object as the argument. In some situations, if the object provides the __getitem__ method, this function can "produce" an iterator without calling __iter__; for an example, see **??** in sec. **??**, p. **??**.

Iterator in turn is an object which provides the __next__ function returning the next element of a sequence (again, one can use the built-in function next with an iterator as the argument). When the sequence is exhausted, __next__ should raise (throw) a StopIteration exception. An iterator should also possess the __iter__ method which just returns itself (this makes iterators themselves iterables).

The for loop takes care about all this automatically. Having the iterable (provided by expression_list), iter is invoked to get the iterator. The body of the loop is then executed for each item provided by the next function. Each item is assigned to the target_list. The target_list doesn't have to be a single value if items returned by the iterator are, e.g., key-value pairs of a dictionary or tuples, as in the example below. At some point next raises the StopIteration exception, which is handled by the for loop, so we don't see it.

In the example below, we use a for loop and then we repeat the same action "by hand", calling iter, and then next until we get an exception:

Listing 7 AAK-For/For.py

```
1  #!/usr/bin/env python
2
3  persons = [("Ann", 23), ("Bea", 34), ("Sue", 29)]
4  for name, age in persons:
5      print('{} is {} years old'.format(name, age))
6
7  print('\n' + '*'*5 + " now 'by hand' " + '*'*5 + '\n')
8
9  it = persons.__iter__() # or it = iter(persons)
10
11 a = it.__next__()
12 print('{} is {} years old'.format(a[0], a[1]))
```

[1] Like Java objects implementing the Iterable interface, which declares abstract method iterator returning an object of a class implementing Iterator.

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

```
13
14 a = next(it) # equivalently like this
15 print('{} is {} years old'.format(a[0], a[1]))
16
17 a = next(it)
18 print('{} is {} years old'.format(a[0], a[1]))
19
20 a = it.__next__()
21 print('{} is {} years old'.format(a[0], a[1]))
```

The program prints

```
Ann is 23 years old
Bea is 34 years old
Sue is 29 years old

***** now 'by hand' *****

Ann is 23 years old
Bea is 34 years old
Sue is 29 years old
Traceback (most recent call last):
File "/home/werner/python/py_progs/sources/AAK-For/For.py",
        line 21, in <module>
    a = it.__next__()
StopIteration
```

And another example, this time with a dictionary

Listing 8 AAN-DictLoop/DictLoop.py

```
1 #!/usr/bin/env python
2
3 fourSeasons = {'Spring' : 'E major', 'Summer' : 'G minor',
4                'Autumn' : 'F major', 'Winter' : 'F minor'}
5
6 for key, value in fourSeasons.items():
7     print(key + ' is in the key of ' + value)
```

which prints

```
Spring is in the key of E major
Summer is in the key of G minor
Autumn is in the key of F major
Winter is in the key of F minor
```

Note that there is no variable denoting the index of the element provided by the iterator (as in traditional **for** loops in C/C++/Java). Very often such index would be useful, though, so there is a built-in function **enumerate** which takes an iterable and returns an *enumerate* object which can be iterated over and provides 2-tuples, the

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

first element of which is the index and the second is an item that would be returned by a "normal" iterator; for example

Listing 9 AAL-ForEnum/Enum.py

```
#!/usr/bin/env python

lst = [1, 7, 4, 9, 2, 6, 3]
indsOdd = []
for i, e in enumerate(lst):
    if e % 2 != 0:
        indsOdd += [i]
print('Odd elements at positions',indsOdd)
```

prints the indices of all odd elements of the list lst

```
Odd elements at positions [0, 1, 3, 6]
```

Indices returned by an enumerate object start from zero, but this can easily be changed by providing additional argument to **enumerate**:

Listing 10 AAO-EnumStart/EnumStart.py

```
#!/usr/bin/env python

seasons = [ "Spring", "Summer", "Autumn", "Winter" ]
for ind, s in enumerate(seasons, 1):
    print('Season', ind, '->', s)
```

which prints

```
Season 1 -> Spring
Season 2 -> Summer
Season 3 -> Autumn
Season 4 -> Winter
```

Indices can also be obtained by using a built-in function **range**, which returns a *range object* — an iterable representing an arithmetic progression of integer numbers. Such sequence may be created by the `range(start, stop, step)` expression: the first number of the sequence will be start (default is 0), the difference between consecutive numbers will be step (default is 1) and the last element will be the last which is still smaller than stop (still greater than stop for negative steps). Special version of this function, `range(n)`, with only one argument, is equivalent to **range(0, n, 1)**. Here, we show the "anatomy" of using **range** and then just the **for** loop which takes care about everything...

```
>>> r = range(1, 6, 2)
>>> type(r)
<class 'range'>
>>> len(r)
3
```

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

```
>>> hasattr(r,'__iter__')
True
>>> it = iter(r)
>>> hasattr(it,'__next__')
True
>>> it.__next__()
1
>>> next(it)
3
>>> next(it)
5
>>> next(it)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
StopIteration
>>>
>>> for i in r:    # everything taken care of by 'for'
...     print(i)
1
3
5
```

(built-in function **hasattr** checks if an object has a given attribute).

As we already mentioned, in the construct

```
for target_list in expression_list:
    body_of_the_loop
[else:
    block_of_code]
```

the expression_list is evaluated once only, at the very beginning, and should provide an iterable. Therefore, this iterable cannot be modified during the execution of the loop, as this would lead to situation when the current iterable is not as it was when expression_list was evaluated. Doing so may cause unexpected and sometimes hard to detect errors; for example in

```
>>> nums = [2, 1, 3, 4, 5, 6]
>>> for n in nums:
...     if n%2 != 0:
...         nums.remove(n)
...
>>> nums
[2, 3, 4, 6]
```

we see that number 3 remained in the list, even though it should have been removed! If we really want to modify an iterable, we should iterate over a copy while modifying the original:

```
>>> nums = [2, 1, 3, 4, 5, 6]
>>> for n in nums[:]:
...     if n%2 != 0:
...         nums.remove(n)
```

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

```
...
>>> nums
[2, 4, 6]
```

(here we used the fact that the slice nums[:] creates a (shallow) *copy* of nums; see sec. 4.5.2, p. 49).

2.6 match statement

match statement corresponds to switch in Java or C++, but is richer in features, resembling similar constructs in Haskell, Scala or Rust.

In its simplest form, it is easy to interpret and understand if we remember switch from C/C++/Java:

Listing 11 AAQ-MatchBasic/MatchBasic.py

```
#!/usr/bin/env python

k = 3
match k:
    case 0:
        print('zero')
    case 1 | 2 | 3 | 4:          ❶
        print('less than 5')
    case 5:
        print('five')
    case _:                      ❷
        raise ValueError()
```

Note that patterns in case-branches can be "ORed" (❶) using the vertical bar |. Instead of a default clause, as in other languages, one can use "catch all" clause ❷, denoted by _ (underscore) symbol. In this context, the underscore becomes a "soft" keyword (see sec. 1.12, p. 20). Any other valid identifier can also play the rôle of a "catch all" pattern and would be bound to the value of the selector (called *subject* in Python):

```
>>> k = 7
>>> match (k):
...     case 3:
...         print('three')
...     case a:
...         print(f'{a=}')
...
a=7
```

while the underscore here means "anything" but is *not* bounded to any value:

```
>>> k = 7
>>> match (k):
...     case 3:
...         print('three')
```

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

```
...     case _:
...         print(f'{_=}')
...
Traceback (most recent call last):
  File "<python-input-6>", line 5, in <module>
    print(f'{_=}')
          ^
NameError: name '_' is not defined
```

Note also that there is no "fall through" here: only the first matching case branch is executed (or none, if there is no matching case pattern).

We can match not only values, but also structured data; moreover, it is possible to assign values found in matched structures to variables that can be then used in a corresponding case clause.
In the example below, consecutive case clauses correspond to data of different lengths (these could be tuples or lists). Note the notation used in line ❶: the *d symbols allows us to use d as a list containing all the remaining (except the first which is bound to name c) elements of the matched sequence, even if number of these remaining elements is 0 (however the type of item must be such, that these element could have potentially occurred).

Listing 12 AAT-MatchLen/MatchLen.py

```
1  #!/usr/bin/env python
2
3  tuples = [ (1, 2, 3), ['a', 'b'], (4, 5, 6, 7, 8), 101 ]
4
5  for item in tuples:
6      match item:
7          case [a1, a2]:
8              print('Two elements', a1, a2)
9          case [b1, b2, b3]:
10             print('Three elements', b1, b2, b3)
11         case [c, *d]:                                    ❶
12             print('Many elements', c, end=' ')
13             for x in d:
14                 print(x, end=' ')
15             print()
16         case e:
17             print('One element', e)
```

The program prints

```
Three elements 1 2 3
Two elements a b
Many elements 4 5 6 7 8
One element 101
```

One might ask why in the last iteration the value 101 was not "caught" on line ❶. After all, it is a single value followed by zero other values. This is not the case, because

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

the expression `[c, *d]` means that it must be a type that allows the existence of subsequent elements. For example

```
>>> val = 1
>>> match val:
...     case [c, *d]:
...         print('case 1', c, d, type(d))
...     case a:
...         print('case 2', a)
...
case 2 1
```

but

```
>>> val = (1,)
>>> match val:
...     case [c, *d]:
...         print('case 1', c, d, type(d))
...     case a:
...         print('case 2', a)
...
case 1 1 [] <class 'list'>
```

The list notation is used here for sequences of different lengths, but instead of, for example, `case [a1, a2]:`, we could have used `case (a1, a2):` or even just `case a1, a2:` with no brackets at all.

We can also match structure and values at the same time. In the example below, the first element of cmd is matched to a literal string, while the remaining are bound to variables then used in the corresponding case clause

Listing 13 AAV-MatchVS/MatchValStr.py

```
#!/usr/bin/env python

commands = (
             ('store', 'MyTable', 'John', 'Doe', 123),
             ('send',  'Hello John', 'john@somewhere'),
             ('print', 'Hello, World!'),
             'Error no 42'
           )

def printMessage(message):
    print('Printing message:', message)

def sendMessage(message, address):
    print("Sending '" + message + "' to", address)

def storeInDB(database, *args):
    print('Storing in DB:', end=' ')
    for a in args:
        print(a, end=' ')
```

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

```
20      print()
21
22  def printError(error):
23      print("ERROR: couldn't process '" + error + "'")
24
25
26  for cmd in commands:
27      match cmd:
28          case "print", message:
29              printMessage(message)
30          case ("send", message, address):
31              sendMessage(message, address)
32          case ["store", database, *data]:
33              storeInDB(database, *data)
34          case someError:
35              printError(someError)
```

The program prints

```
Storing in DB: John Doe 123
Sending 'Hello John' to john@somewhere
Printing message: Hello, World!
ERROR: couldn't process 'Error no 42'
```

In the next example, one **case** clause "catches" all two-element sequences binding both elements to variables and then processes them (adding to appropriate collections); anything that is not a two-element sequence is handled differently:

Listing 14 AAX-MatchSN/MatchSeqNam.py

```
1   #!/usr/bin/env python
2
3   fruit   = {}        # empty dictionary
4   veggies = dict()    # also empty dictionary
5   others  = dict()
6   noprice = set()
7
8   frunames = ["Apple", "Pear"]
9   vegnames = ["Carrot","Beetroot" ]
10
11  shopping = [
12                ("Carrot", 4), ("Pear", 7),    "Hammer",
13                ("Tulip", 6),  ("Carrot", 5), ("Pear", 3),
14                "Chocolate",   ("Apple", 8),  ("Carrot", 4),
15                ("Apple", 2),  "Laptop",      ("Beetroot", 1)
16              ]
17
18  for item in shopping:
19      match item:
20          case [fruveg , price]:
```

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

```
            aux = (fruit if fruveg in frunames
                         else veggies if fruveg in vegnames
                                      else others)
            if fruveg not in aux: aux[fruveg] = 0
            aux[fruveg] += price
        case justName:
            noprice.add(justName)

print('Fruit:    ', fruit)
print('Veggies: ', veggies)
print('Others:  ', others)
print('No price:', noprice)
```

The program prints

```
Fruit:    {'Pear': 10, 'Apple': 10}
Veggies:  {'Carrot': 13, 'Beetroot': 1}
Others:   {'Tulip': 6}
No price: {'Laptop', 'Hammer', 'Chocolate'}
```

Patterns may also "catch" values of the selector (*subject*) by its *type,* not necessarily the value, or even both, as we can also bind the its value to a name using `as <name>`. Moreover, it is also possible to limit matched values by adding the so called **guard** after the pattern: it has the form of an `if` statement which sets some restrictions on these values.

This is illustrated by the following example

Listing 15 ABF-MatchT/MatchTypeAs.py

```
#!/usr/bin/env python

subjects = [ {}, 8, "Warsaw", [1, 2], -2.0, 0, -1, "Dog"]

for sub in subjects:
    match sub:
        case int() | float() as a if a < 0:     ❶
            print("Negative number:", a)
        case int() | float() as a if a == 0:
            print("Number 0")
        case int() | float() as a:              ❷
            print("Positive number:", a)
        case str() as w if len(w) > 5:          ❸
            print("Long word:", w)
        case str() as w:                        ❹
            print("Short word:", w)
        case w:                                 ❺
            print(f'Value {w} of type {type(w).__name__}')
```

which prints

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

```
Value {} of type dict
Positive number: 8
Long word: Warsaw
Value [1, 2] of type list
Negative number: -2.0
Number 0
Negative number: -1
Short word: Dog
```

Note the following

- In line ❶, the pattern will match any value of type **int** *or* **float** (notice the parentheses at the names of types) as long as it is negative.
- In line ❷, we don't have to specify that the value should be positive; negative or zero values of type **int** or **float** would have been caught earlier.
- Similarly, in line ❹, only short strings will be caught, as all long strings would have been caught in line ❸.
- In line ❺, we don't use "catch all" symbol `_`, because we want to have access to the value that has been caught in the body of this branch.

More details can be found in PEP 636.[1]

[1] http://peps.python.org/pep-0636

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

Chapter 3

Basic data types

Basic data types which are built-in into the language itself are

- boolean
- numbers (integers, floating-point and complex). There are also useful numeric types in the library: fractions.Fraction and decimal.Decimal;
- sequences (lists, tuples, strings, types bytes, bytearray, memoryview);
- sets;
- dictionaries (maps).

Note that there are no classic arrays here but there are lists, sets and maps (called dictionaries in Python) which in other languages belong to the standard libraries rather than to the language itself.

As everything in Python is an object, there are other built-in types describing

- modules,
- classes and class instances (objects),
- functions,
- methods,
- code objects,
- type objects,
- the None object,
- the Ellipsis object,
- the NotImplemented object,
- internal objects.

Many builtin types (int, float, str) have converting constructors, e.g., `int('123')` yields integer 123, while `str(123)` gives the string `'123'`.

3.1 NoneType type

There is a special type denoting 'nothing' — only one singleton object of this type is created; its name is None. It has no attributes and evaluates to False in Boolean context. None is returned by functions that don't explicitly return any value (i.e., correspond to void functions in C/C++/Java).

```
>>> type(None)
<class 'NoneType'>
>>> a = print()

>>> b = print()

>>> type(a), type(b)
(<class 'NoneType'>, <class 'NoneType'>)
>>> id(a), id(b)
(94793807807456, 94793807807456)
>>> if a:
...     print('true')
```

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

```
... else:
...     print('false')
...
false
```

None is also often used as the default value of optional arguments, so the function can easily detect whether the caller has actually passed a value for that argument or not.

3.2 ellipsis type

A special object of type ellipsis, which may be written as Ellipsis or ... (three dots), is used almost uniquely in numerical packages for slicing in multidimensional arrays (matrices). In Boolean context, it evaluates to True.

```
>>> type(Ellipsis)
<class 'ellipsis'>
>>> type(...)
<class 'ellipsis'>
>>> id(Ellipsis), id(...)
(9422880, 9422880)
>>> if Ellipsis: print('T')
... else:        print('F')
...
T
```

3.3 Typ NotImplementedType

A special type with one singleton object named NotImplemented. It is used as the return value in special methods which should exist but haven't been implemented yet.
It is not recommended (and considered obsolete) to use this object in a logical context.

3.4 Numeric types

Numeric types describe numbers of integer and floating-point types. We will include here also the Boolean type bool, because it inherits from int and *can* be used (unfortunately) in numerical contexts.

Generally, Python is rather reluctant to implicitly convert from one type to another. However, as in most other languages, you can mix numeric values of different types when operating on them. When you add an integer to a float, for example, the result will be float (as "wider" than int)

```
>>> a = 2; b = 2.5
>>> a+b, type(a+b)
(4.5, <class 'float'>)
```

Similarly, complex type is wider than float, so in operations involving complex numbers, integers and floats are promoted to complex:

```
>>> a = 1; b = 1.5; c = (1 + 1j)
>>> d = a + b + c;
```

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

```
>>> d, type(d)
((3.5+1j), <class 'complex'>)
```

In order to facilitate mixed-type (**int**, **float**, **complex**) arithmetic, floats and integers do have their real and imaginary attributes real and imag defined (their values are, of course, equal to 0):

```
>>> a = 7
>>> a, type(a), a.real, a.imag
(7, <class 'int'>, 7, 0)
>>> b = 3.75
>>> b, type(b), b.real, b.imag
(3.75, <class 'float'>, 3.75, 0.0)
```

Constructors of numeric types may be used to explicit conversions (from other numeric type or strings).
Casting **float**→**int** rounds towards zero:

```
>>> a = 3.9; b = -3.9
>>> int(a), int(b)
(3, -3)
```

and you can also use strings:

```
>>> int('123'), float('-1.2e+2'), complex('2-3j')
(123, -120.0, (2-3j))
```

Note that for complex numbers, the string must not contain spaces:

```
>>> complex('1 + 1j')
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: complex() arg is a malformed string
```

3.4.1 Integers

There is only one integer type – **int**. However, the size of the representable integers is not limited: Python will transparently select an appropriate internal representations for any, arbitrary large value; e.g., 33^{33} will be calculated correctly, giving

$$129110040087761027839616029934664535539337183380513$$

(operating on such numbers may be inefficient, though).
Integer literals can be prefixed with a digraph denoting the base: `0b` or `0B` for binary, `0o` or `0O` for octal, and `0x` or `0X` for hexadecimal. For example,

$$47 \equiv 0b101111 \equiv 0o56 \equiv 0x2F.$$

As in Java, underscores can be used in integer literals to separate groups of digits

```
>>> 1_234_567
1234567
```

The **int** class has constructors which can be used as conversion function: in particular, there are such conversions from strings (but also from **bytes** or **bytearray**). The second, optional, argument specifies the radix (of course, the default is 10):

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

```
>>> int('1D', 16)
29
>>> int('1D', 16), int('0x1D', 16)
(29, 29)
>>> int('57', 8), int('0o57', 8)
(47, 47)
>>> int('11001',2), int('0b11001', 2)
(25, 25)
>>> int('123', 10), int('123')
(123, 123)
>>> int('2Z', 36)
107                 # 2*36 + 35
```

There are also built-in functions **bin**, **oct** and **hex** for **int** → **str** conversions, where the resulting string represents the given number with radix 2, 8 or 16 (for decimal representation, just use the constructor of **str**):

```
>>> a = bin(0x8E)
>>> a, type(a)
('0b10001110', <class 'str'>)
>>>
>>> b = oct(0xF)
>>> b, type(b)
('0o17', <class 'str'>)
>>>
>>> c = hex(255)
>>> c, type(c)
('0xff', <class 'str'>)
>>>
>>> d = str(123)
>>> d, type(d)
('123', <class 'str'>)
```

Implementation of the **int** type is rather unique in Python. All integers, small and arbitrary big, are represented by objects of the same type – **int**; no **long**s, **short**s, signed and unsigned types, etc.[1]

Objects representing small integers, at least in the range $[-5, 256]$, are created once only and cached, so all references to their values point to exactly the same objects:

```
>>> a = -2; b = 0b1011 - 0xD; c = -4000//2000
>>> a, b, c
(-2, -2, -2)
>>> id(a), id(b), id(c)
(9801152, 9801152, 9801152)
```

Integers are internally represented by C-structures **PyLongObject** containing, among other fields,[2]

- pointer to array ob_digit of 'digits', which are just **uint32_t** numbers;

[1] But see sec. ??, p. ??.
[2] On 64-bit platforms.

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

- size of this array ob_size.

For numbers smaller than 2^{30}, the array has only one element which is interpreted directly as an unsigned four byte integer. For bigger numbers, elements of the array ('digits') are interpreted as coefficients at increasing powers of 2^{30} and themselves have values in the range $[0, 2^{30} - 1]$. For example,

$$398540651359084385821447815 = 1234567 + 23456789 * 2^{30} + 345678912 * 2^{30*2}$$

so for this number, ob_digit is $[1234567, 23456789, 345678912]$ and ob_size is 3.

Negative numbers are represented in the same way and their reversed sign is signaled only by negative value of ob_size. If ob_size is 0, the number is 0.

Due to their complicated implementation, objects of type **int** occupy different amount of memory, depending on their value. Minimum size of these objects is 28 bytes (3.5 times the size of a C/C++/Java **long**.) This happens when the array ob_digit of size 1 is sufficient (i.e., for numbers smaller than 2^{30}). However, for bigger numbers, the size of this array must grow and objects become larger:

```
>>> a = 1
>>> a, type(a), a.__sizeof__()
(1, <class 'int'>, 28)
>>>
>>> a = 2**30-1
>>> a, type(a), a.__sizeof__()
(1073741823, <class 'int'>, 28)  # still one element
>>>
>>> a = a + 1
>>> a, type(a), a.__sizeof__()
(1073741824, <class 'int'>, 32)  # two elements needed
>>>
>>> a = (2**30-1)*2**30 + (2**30 - 1)
>>> a, type(a), a.__sizeof__()
(1152921504606846975, <class 'int'>, 32) # still two elements
>>>
>>> a = a + 1
>>> a, type(a), a.__sizeof__()
(1152921504606846976, <class 'int'>, 36) # now three needed
```

There are, of course, algorithms for arithmetic operations on numbers represented in this way.

3.4.2 Boolean type

Python supports special type for Boolean values – **bool** (which is a subclass of **int**). It has only two values: **True** and **False**. However, as in many other languages (but not Java!), values of other types may be used in contexts requiring a Boolean value; the general rule is that the following objects

- numerical values equal to zero,
- empty sequences and collections (lists, sets, dictionaries, strings, tuples),
- special object **None**,
- constant **False**

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

are all "falsy" — they are interpreted as **False** in Boolean contexts. All other objects are "truthy". The only object that shouldn't be used in the logical context is **NotImplemented** (which is, however, evaluated to **True**).

If we really want something "truly" Boolean, we can always use conversion to **bool** just by passing an object to the **bool**'s converting constructor:

```
>>> a = bool("Hello")
>>> a, type(a)
(True, <class 'bool'>)
>>> bool({})
False
>>> bool(0)
False
>>> bool(78)
True
>>> bool(NotImplemented)
<stdin>:1: DeprecationWarning: NotImplemented should
           not be used in a boolean context
True
>>> bool(Ellipsis)
True
>>> bool([[]]) # non-empty list: contains one element!
True
```

Unfortunately, objects of type **bool** are represented by integers (0 and 1), so formally they *are* of a numeric type (although they shouldn't be...). For example,

```
>>> True/3
0.3333333333333333
>>> (7-False)/(True+1)
3.5
```

This fact that **bool** inherits from **int** has sometimes unexpected consequences; for example

```
>>> True == 2-1
True
>>> True == 3-1
False
```

and it's impossible to have both **True** and 1 in a set or as keys in a dictionary.

Logical alternative and conjunction are denoted by the **or** and **and** keywords, respectively. As in most other languages, they are both short-circuit operators – the second operand is evaluated if, and only if, the result cannot be established after evaluation of the first one, i.e., only when the first operand is interpreted as **False** for alternative or as **True** for conjunction.

Logical negation is denoted by the **not** keyword.

Note that **||**, **&&** and **!**, known from other languages, will *not* work in Python.

3.4.3 Floating point numbers

There is only one floating point type – **float**. It corresponds to what is called **double** in, for example, C/C++/Java. These numbers occupy 8 bytes (64-bit) with 1 sign

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

bit, 11 bits for exponent part and 52 bits for the mantissa; this gives 53 binary-digits precision, because in binary system the most significant digit of any number must be 1, so there is no need to store it physically in memory (for details, see, e.g., description here[1]).

Remember that floating point numbers are almost always *approximations* only; as numbers are internally expressed in binary system, even such "innocent" values like 0.1 do not have exact representation. Only rational numbers with a power of 2 in the denominator may be represented exactly:

```
>>> v0100 = 0
>>> v0125 = 0
>>> for i in range(1_000_000):
...     v0100 += 0.100
...     v0125 += 0.125
>>> v0100 - 100_000
1.3328826753422618e-06
>>> v0125 - 125000
0.0
```

In the second case, we were adding 0.125, which is $1/2^3$ and so *is* represented exactly and sums up to the correct value 125 000. However, there is no such number like 0.1, so adding it milion times does not give us the correct sum 100 000: we can see this by printing 0.1 as a **Decimal**:

```
>>> import decimal
>>> decimal.Decimal(0.1)
Decimal('0.1000000000000000055511151231257827021181583404541015625')
```

The above result is *exact* — it is, in decimal system, exactly the number represented by *approximation* of 0.1 on 53 binary digits of a **double**; this value in binary system has no finite representation, much like 1/3 in decimal system.

Sometimes, in particular in numerical computations, one has to deal with infinities (Inf) or not-a-number (NaN) values; these are also defined in Python:

```
>>> a = float('nan')
>>> b = float('nan')
>>> c = a + b
>>> a == b, a, b, c
(False, nan, nan, nan)
```

As we can see, NaN compared with anything, even with itself, gives **False**, and any arithmetic operation involving a NaN yields also a NaN.

Let us look at infinities:

```
>>> x = float('inf')
>>> y = float('-inf')
>>> x, y, x + 100, x - y, x + y
(inf, -inf, inf, inf, nan)
```

Adding/subtracting anything finite to/from ±Inf gives ±Inf; also infinities of the same sign add up to infinity. However, subtracting infinities (i.e., adding infinities of opposite signs) is not mathematically defined, so it yields a NaN.

[1] https://en.wikipedia.org/wiki/IEEE_floating_point

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

3.4.4 Complex numbers

Literals of the **complex** type are written with the letter 'j' (or 'J', but *not* 'i', as in mathematics) appended to the imaginary part.
For example `0.5*(1+1j)**2` gives `1j`. Both real and imaginary parts are represented by 64-bit **float**s and can be accessed as read-only attributes **real** and **imag**:

```
>>> z = 3+4j
>>> z.real
3.0
>>> z.imag
4.0
>>> z.imag = 8
Traceback (most recent call last):
File "<stdin>", line 1, in <module>
  AttributeError: readonly attribute
```

The **abs** function (which is built-in and doesn't require any imports) works also for complex values; moreover, the ***cmath*** module contains many other mathematical functions that operate on and return complex numbers (like **phase**, **exp**, **log**, **sqrt**, trigonometric and hyperbolic functions, etc.). One can mix **float**s and **int**s with **complex** numbers in all mathematical operations: if there is at least one complex operand, the others will be promoted to **complex** and the result will also be of that type (perhaps with imaginary part equal to 0):

```
>>> z = 1 + 1j
>>> import cmath
>>> 2 + cmath.log(z)/1.5
(2.2310490601866486+0.5235987755982988j)
>>> from cmath import exp as cexp
>>> z - cexp(cmath.log(z))
2.220446049250313e-16j       # essentially 0
```

3.5 Strings (introduction)

Literal strings (strings are of type **str** in Python) may be written in several ways. They can be enclosed by single apostrophes or double quotes. If the apostrophes are used, quotation mark inside behaves as a normal character and doesn't need to be escaped, while the apostrophe itself has to be escaped (and the other way around for strings enclosed by double quotes). Two adjacent literal strings are concatenated and represent one string, as in C/C++

```
>>> "That's OK, " 'double quote doesn\'t have to be "escaped" here'
'That\'s OK, double quote doesn\'t have to be "escaped" here'
```

One can insert into a string any character knowing its Unicode code. To enter a character with 16-bit Unicode code point, notation `\uxxxx` is used; for 32 bit values, substitute capital 'U' for 'u' (`\Uxxxxxxxx`). Each 'x' here stands for one hexadecimal digit.

```
>>> '\u00a9 \u017b\u00f3\u0142w'
'© Żółw'
```

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

As in most other languages, certain special characters can be entered by escaping one character with the backslash:

- `\a` for bell character (BEL);
- `\b` for backspace (BS);
- `\f` for formfeed (FF);
- `\n` for new line a.k.a. line feed (LF);
- `\r` for carriage return (CR);
- `\t` for horizontal tab (HT);
- `\v` for vertical tab (VT);
- `\'` for single quote (apostrophe);
- `\"` for double quote.

It is also possible to delimit a string with triple quotes (single or double). Inside such strings, escape characters work as usual, but apostrophes and double quotes do not need to be escaped; also, new-line characters are preserved

```
>>> """That's OK, quotes don't
... have to be "escaped" here"""
'That\'s OK, quotes don\'t\nhave to be "escaped" here'
```

By prefixing a string with the letter 'r' (or 'R'), we get something known as a **raw string**. In such strings, the backslash is taken literally, not as the escape character, so string `r"\t\v\b"` contains 6 characters, three of them backslashes and three letters (without the 'r', it would be a three-character string containing a tabulator, a vertical tabulator and a backspace).
Raw strings are especially helpful when defining regular expressions, which usually contain lots of backslashes.

Important: As in Java, strings (objects of type `str`) are *immutable*. It is not possible to modify a string: functions which seemingly do it are in fact returning a new string based on an existing one:

```
>>> s = "Alice"
>>> id(s)
139733635023176
>>> s = s.lower()
>>> s, id(s)
('alice', 139733635017168)
```

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

Chapter 4

Operators

As we know from other languages, like C++/Java, operators are really functions in disguise, referred to not by "normal" name but rather symbols (*, &, +, etc) placed in front of or between operands (arguments). We know also that there are unary (!, ~, etc.), binary (+, %, etc.) and even one ternary operator ?:.

In this chapter, we will present an overview of operators in Python, their features and precedence. But first a few words about assignment, which is *not* an operator in Python.

4.1 Assignment (not an)operator

Operators, unary or binary, when applied to their operands, always, by definition, yield values. Note, however, that in Python assignment is not an expression, it's a statement

```
>>> a = (b = 2)              # OK in C/C++/Java
File "<stdin>", line 1
    a = (b = 2)
        ^
SyntaxError: invalid syntax
```

It follows that assignment =, but also augmented forms like +=, *=, etc., are *not* operators, as they are in C/C++/Java, where assignment is a "normal", binary, right-associative operator.

What can be surprising, though, `a = b = 2` *is* valid! Chained assignments are treated in a special way: first the rightmost expression (it has to be an expression!) is evaluated and then its value is, one by one, *from left to right* assigned to "targets" in front of it:

```
>>> b = [3, 3]
>>> a = b[a] = b[a-1] = len(b)-1
>>> b
[1, 1]
```

As we can see, when the value of `len(b)-1`, which is 1, is assigned to `b[a]`, variable `a` already exists and has value 1. Thus, conceptually,

$$\text{target}_1 = \text{target}_2 = \text{target}_3 = \text{expr}$$

is equivalent to evaluating expr and then assigning

$$\begin{aligned} \text{target}_1 &= \text{expr} \\ \text{target}_2 &= \text{expr} \\ \text{target}_3 &= \text{expr} \end{aligned}$$

in this order.

However,

```
>>> x = y = z+= 2
SyntaxError: invalid syntax
```

doesn't work, because the rightmost part, `z += 2`, is *not* an expression — it has no value.

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

4.2 Assignment (true) operator

In Python 3.8 the "true" assignment (assignment expression) was added to the language. That means that such assignment does have the type and value of the left-hand side after the assignment, as it does in C/C++/Java.
It is denoted by := symbol[1] and has the form `name := expr`, where `name` is an identifier and `expr` is any expression (except an unparenthesized tuple).

Such assignments often simplify the code and make it more readable. To avoid errors, the implementation of this operator is such that there is no context in which both forms of assignment would be syntactically correct. For example

```
>>> x = 5
>>> x
5
>>> (x = 6)
  File "<python-input-34>", line 1
    (x = 6)
     ^^^^^
SyntaxError: invalid syntax.
             Maybe you meant '==' or ':=' instead of '='?
>>> x := 7
  File "<python-input-35>", line 1
    x := 7
      ^^
SyntaxError: invalid syntax
>>> (x := 7)
7
>>> x
7
```

In most cases, the assignment expression therefore requires it to be enclosed in parentheses.

The := operator can be used only to variables with simple names, not dotted or indexed:

```
>>> lst = [1, 2, 3]
>>> lst[1] = 4
>>> (lst[2] := 5)
  File "<python-input-62>", line 1
    (lst[2] := 5)
     ^^^^^^
SyntaxError: cannot use assignment expressions with subscript
```

Let us consider an example: suppose we read data from the user until they enter "quit" (and we add the data read to a list). We would have to write something like this:

```
numbers = []
inp = input("Enter number (or 'quit'): ")
while inp != "quit":
```

[1] Called 'walrus'.

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

```
    numbers.append(int(inp))
    inp = input("Enter number (or 'quit'): ")
```

This works, but is ugly — the line with **input** is repeated twice. Now we can do better using the *walrus* operator (:=) which works more like assignments in C/C++/Java and yields a value:

```
numbers = []
while (inp := input("Enter number (or 'quit'): ")) != "quit":
    numbers.append(int(inp))
```

Assignment expressions are commonly used to give a name to an intermediate result, so it can be used without the need to recalculate it, even in the same statement:

```
s = 'rotator'
if (r := s[::-1]) == s:
    print('Palindrom')
else:
    print('Not palindrom -> ' + r)
print('r is ' + r)
```

This program prints

```
Palindrom
r is rotator
```

what shows another important feature of Python: although `r` is created inside **if** statement, it is still visible after the **if** (see discussion of scopes, sec. 5.1, p. 67).

Assignment expressions can also be used in comprehensions (sec. 6.1, p. 103): for example

```
def fib(n):
    if n < 2:
        return n
    else:
        return fib(n-1) + fib(n-2)

print([(n, r) for n in range(24,31) if (r := fib(n)) % 2 != 0])
```

prints odd Fibonacci numbers F_n for $n \in [24, 30]$

```
[(25, 75025), (26, 121393), (28, 317811), (29, 514229)]
```

and we didn't have to recalculte the value of `r` in the expression `(n, r)` as it was already calculated in the **if** part.

We will encounter other uses of the walrus operator in the course of the lecture.

4.3 Operators and operator precedence

The precedence of Python operators is presented in the table below:

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

Table 2: Operator precedence

Operator(s)	Description
(expressions...), [expressions...], {key: value...}, {expressions...}	Parenthesized expressions: literal lists, sets, dictionaries
x[i], x[i:j], x[i:j:ks], x(arguments...), x.attribute	Subscription, slicing, call operator, attribute reference
await x	Await expression
**	Exponentiation
+x, −x, ~x	Unary +/-, bitwise NOT
*, @, /, //, %	Multiplication, matrix multiplication, division, floor division, modulo
+, −	Addition, subtraction
<<, >>	Bitwise shifts
&	Bitwise AND
^	Bitwise XOR
|	Bitwise OR
in, not in, is, is not, <, <=, >, >=, !=, ==	Comparisons, including membership tests and identity tests
not	Boolean NOT
and	Boolean AND
or	Boolean OR
if-else	Conditional expression
lambda	Lambda expression
:=	Assignment expression

4.4 Parenthesized expressions

Parenthesized expressions are used to create lists (`[ expressions... ]`) (sec. ??, p. ??), sets (`{ expressions... }`) (sec. ??, p. ??), tuples (`( expressions... )`) (sec. ??, p. ??), dictionaries (`{ key:value... }`) (sec. ??, p. ??):

```
>>> a = [1, 2, 'Hello']
>>> a, type(a)
([1, 2, 'Hello'], <class 'list'>)
>>> b = {1, 2, 'Hello'}
>>> b, type(b)
({1, 2, 'Hello'}, <class 'set'>)
>>> c = {1 : 'un', 3 : 'trois', 'un' : 1, 'trois' : 3}
>>> c, type(c)
({1: 'un', 3: 'trois', 'un': 1, 'trois': 3}, <class 'dict'>)
```

Instead of specifying a sequences of elements, one can use comprehensions for lists (sec. 6.1.1, p. 104), sets (sec. 6.1.2, p. 107), dictionaries (sec. 6.1.3, p. 109) and generators (sec. 6.2.1, p. 113):

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

```
>>> a = [x*x for x in range(1, 11) if x%2 != 0]
>>> a, type(a)
([1, 9, 25, 49, 81], <class 'list'>)
>>>
>>> b = {x for x in dir(a) if x[:1] == 'c'}
>>> b, type(b)
({'count', 'copy', 'clear'}, <class 'set'>)
>>>
>>> c = {x : len(x) for x in b}
>>> c, type(c)
({'count': 5, 'copy': 4, 'clear': 5}, <class 'dict'>)
>>>
>>> d = (x for x in range(1,101) if x**2 % 10 == 6)
>>> d, type(d)
(<generator object <genexpr> at 0x7f52a900>, <class 'generator'>)
>>> for x in d:
    print(x, end=' ')

4 6 14 16 24 26 34 36 44 46 54 56 64 66 74 76 84 86 94 96
```

4.5 Subscription and slicing

4.5.1 Indexing

Indexing in Python behaves as in C/C++/Java — index is given in square brackets and indexing starts from 0. Indices can be used for objects of types implementing the **__getitem__** method (although it's not enough for object to be a full-fledged sequence). In particular, all sequences do support indexing (**list**s, **tuple**s, **range**s, **str**ings).

```
>>> ls = [1, 2, '3', 'four', ('Hello', 'World')]
>>> ls[0]
1
>>> ls[3]
'four'
>>> ls[4]
('Hello', 'World')
>>> ls[4][1]
'World'
```

This is not surprising, but Python has more to offer. Sometimes, it's more convenient for us to specify an index counting from the end; for example "I want the third last element of this sequence". Instead of calculating which index it corresponds to, we can use *negative* indices.

Index -1 corresponds to the *last* element, and so is equivalent to positive index $-1+$leng, where leng is the number of elements in our sequence. Index -2 corresponds to the second last element, and so on. Generally, negative index $-n$ corresponds to positive index $-n + \text{leng}$. For example, the string `'PYTHON'` has length 6 and its elements (characters) can be indexed as follows:

```
 0   1   2   3   4   5
 P   Y   T   H   O   N
-6  -5  -4  -3  -2  -1
```

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

Hence, legal indices are

$$-\text{leng}, -\text{leng}+1, \ldots, -1, 0, 1, \ldots, \text{leng}-1$$

what can be seen from the following snippet:

```
>>> s = 'PYTHON'
>>> s, len(s)
('PYTHON', 6)
>>> s[0]
'P'
>>> s[5]
'N'
>>> s[-1]
'N'
>>> s[-6]
'P'
>>> s[6]
Traceback (most recent call last):
File "<stdin>", line 1, in <module>
IndexError: string index out of range
>>>
>>> s[-7]
Traceback (most recent call last):
File "<stdin>", line 1, in <module>
IndexError: string index out of range
```

index

A very common task is to find the index of an element of a sequence (list, tuple, string,...). There is a useful method **index** which allows to do it (it finds the first occurrence of an element in a sequence):

```
>>> a = (1, 4, 5, 8)     # tuple
>>> b = [1, 4, 5, 8]     # list
>>> c = '1458'           # string
>>> a.index(5), b.index(5), c.index('5')
(2, 2, 2)
```

One can add an additional argument which indicates from which index to start the search

```
>>> a = [5, 6, 7, 5, 6, 7]
>>> a.index(5, 2)
>>> 3
```

The result is 3 because searching for 5 starts here from the element with index 2 (and value 7). Optionally, one can add one more argument indicating the last index defining a half-open range of elements which will be searched (as usual, the element with this last index is *excluded* from this range):

```
>>> a = [5, 6, 7, 8, 6, 4, 6]
>>> a.index(6, 2, 5)
4
```

Here, the value 6 was searched in a range of indices $[2, 6)$, i.e., $2, 3, 4, 5$ (without 6). If there is no element with a given value, **ValueError** exception is thrown:

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

```
>>> a = [1, 2, 4, 5, 3]
>>> a.index(3)
4
>>> a.index(3, 0, 4)
Traceback (most recent call last):
File "<stdin>", line 1, in <module>
ValueError: 3 is not in list
```

Exception has been raised here, because the value 3 is not in the range of indices $[0, 4)$, which does not include index 4. In order to avoid exceptions, we can first check if the element we are looking for occurs in a given sequence at all (**in** and **not in** operators):

```
>>> a = [1, 2, 3, 4]
>>> b = 'Hello'
>>> 2 in a
True
>>> 5 in a
False
>>> 5 not in a
True
>>> 'el' in b
True
```

Analogous method, **find**, works for strings and objects of type **bytes** (see sec. ??, p. ??); instead of raising an exception, it returns -1 if a value searched for has not been found. find

```
>>> s = 'abcdefg'
>>> s.find('d')
3
>>> s.find('h')
-1
>>> s.find('b', 2)     # start from index 2
-1
```

4.5.2 Slices

Slices are *copies* of subsequences of sequences (taking a slice does not change the original sequence).

Suppose seq is a sequence and it's length (number of elements) is leng. Then after

```
sl = seq[beg:end:step]
```

sl will be a sequence of *copies of* elements (don't forget these are *references*!) of seq corresponding to indices beg, beg + step, beg + 2 ∗ step, *ldots* up to, but **not** including, end.

For example, for `[2:7:1]` these will be elements with indices $2, 3, 4, 5, 6$, and for `[2:7:2]` — $2, 4, 6$.

If step is not specified, it defaults to 1, so `[2:5]` is equivalent to `[2:5:1]`. One can omit beg (but not a colon that follows it) and/or end; then, for positive step:

- beg defaults to 0, so `[:5]` is equivalent to `[0:5:1]`,
- end defaults to the length of the sequence.

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

For example:

```
>>> ls = ['a', 'b', 'c', 'd', 'e', 'f']
>>> ls[:3]                          # = ls[0:3:1]
['a', 'b', 'c']
>>> ls[1::2]                        # = ls[1:6:2]
['b', 'd', 'f']
>>> ls[:]                           # = ls[0:6:1]
['a', 'b', 'c', 'd', 'e', 'f']
>>> ls[2:6:2]
['c', 'e']
>>> ls[5:1:-1]
['f', 'e', 'd', 'c']
```

In the last example, step is negative, but both beg and end are explicitly specified. But negative steps become tricky when beg and/or end are omitted. Then

- beg defaults to -1,
- end defaults to `-leng-1`.

For example, taking a string as our sequence (note that its length is 6, so the default value of end is -7):

```
>>> s = 'abcdef'
>>> s[:-2:-1]     # -1
'f'
>>> s[-3:-6:-1]   # -3, -4, -5
'dcb'
>>> s[:-4:-1]     # -1, -2, -3
'fed'
>>> s[-2::-1]     # -2, -3, -4, -5, -6
'edcba'
>>> s[:-5:-2]     # -1, -3
'fd'
>>> s[::-2]       # -1, -3, -5
'fdb'
>>> s[::-1]       # -1, -2, -3, -4, -5, -6
'fedcba'
```

In comments, after the # symbol, indices that are included in the resulting slice are shown. They are negative, but we remember that they are equivalent to positive indices after adding the length of the sequence (6 in our case). Remember also that the index end is never included.

The last example is interesting: as we can see, the slice `[::-1]` yields a copy of the entire sequence, but in reverse order![1]

In order to see better what is going on, we can use objects of type **slice**. As a matter of fact, when we use something like `seq[beg:end:step]`, under the hood object of this type will be created and used to find "real" (non-negative) indices. We can do it "by hand" (arguments **None** of the constructor mean "take the default"):

[1] Hence, `if ls==ls[::-1]:...` checks if ls is a palindrome.

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

```
>>> ls = [1, 2, 3, 4, 5]
>>> sl = slice(None, 3, 2)
>>> ls[sl]
[1, 3]
```

As we noted, `seq[::-1]` returns the copy of seq in reverse order. Let's see how the indices are calculated. Objects of type **slice** have the **indices** function which takes the length of a sequence for which to calculate the indices:

indices

```
>>> sl = slice(None, None, -1) # corresponds to [::-1]
>>> sl.indices(6)
(5, -1, -1)
>>> 'abcdef'[sl]
'fedcba'
```

The result, $(5, -1, -1)$, tells us that for sequences of the length 6, indices from 5 (last element) down to -1, but *not* including -1, will be used; in other words they will be 5, 4, 3, 2, 1, 0.

Similarly:

```
>>> a = [1, 2, 3, 4, 5, 6, 7, 8, 9]
>>> sl = slice(-2, 1, -2)
>>> sl.indices(9)
(7, 1, -2)           # 7 5 3
>>> a[sl]
[8, 6, 4]
>>> a[-2:1:-2]
[8, 6, 4]
```

Expressions like `[beg:end:step]` give us a copy of the (sub)sequence (in the reverse order for negative step). These constructs are often used, but there is a caveat we must be aware of. Consider:

```
>>> ls = [1, ['a', 'b'], 'c']
>>> copy = ls[::]
>>> reve = ls[::-1]
>>> reve[1][1] = 'd'
>>> copy
[1, ['a', 'd'], 'c']
>>> reve
['c', ['a', 'd'], 1]
>>> ls
[1, ['a', 'd'], 'c']
```

Lists copy and reve contain *copies* of the second element of the original list ls (with index 1). However, this element is the *reference* to a list, and lists *are* mutable. We modify this list, (`reve[1][1] = 'd'`) using reve, but ls and copy still contain the reference to this, now modified, list. That's why all lists seem to be modified.

We can avoid this problem using a *deep* copy of a list, which copies references to immutable objects, for mutable elements creates (recursively, if needed) new objects. In the ***copy*** package, we have functions for both methods of copying: the program:

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

```
import copy
a = [1, ['a', 'b'], 2]
b = copy.copy(a)
c = copy.deepcopy(a)
print(id(a[0]), id(a[1]), id(a[2]))
print(id(b[0]), id(b[1]), id(b[2]))
print(id(c[0]), id(c[1]), id(c[2]))
```

printed

```
139948167725296 139948166858368 139948167725328
139948167725296 139948166858368 139948167725328
139948167725296 139948166871680 139948167725328
```

As we can see, references to numbers were just copied, as numbers are immutable. But for the second element of the original list, which is the reference to a list, the
copy **copy** function also copied the reference, but **deepcopy** created new list, independent
deepcopy of the list pointed to by the same reference a[1] and b[1].

Slices can also be used to modify mutable sequences, like lists. We can remove subsequences defined by slices or replace them with something else:

```
>>> a = [1, 4, '5', [6, '7'], 2.25]
>>> a[2:4] = 'hello'                    #1
>>> a
[1, 4, 'h', 'e', 'l', 'l', 'o', 2.25]
>>> a[1:3] = [8, 9, 'X']                #2
>>> a
[1, 8, 9, 'X', 'e', 'l', 'l', 'o', 2.25]
>>> a[2:-2] = ''                        #3
>>> a
[1, 8, 'o', 2.25]
>>> a[1:3] = ['hello']                  #4
>>> a
[1, 'hello', 2.25]
```

Note, that if the expression on the right is itself a sequence, it will be flattened and inserted into the sequence element by element (#1 and #2). If what we want is to insert a sequence as a single element, we can use one-element list containing this sequence (#4). Replacing a subsequence with an empty sequence just removes it (#3).

4.6 Call operator

Call operator is denoted by round parentheses and is used mostly to call (invoke) a function. But, as in C++, we can define classes, objects of which behave as if they were functions: we just provide the **operator()** method in C++, while in Python
__call__ we define a special[1] **__call__** method — see sec. ??, p. ??. Also the object representing a class itself is callable — "calling" a class gives an object of this class (which itself may be callable, if the class defines the **__call__** method).
callable There is a built-in function **callable** which returns **True** if, and only if an object is callable:

[1] These methods are called "magic", or "dunder" methods.

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

```
>>> class Hello:
...     def __call__(self):
...         return 'Hello'
...
>>> h = Hello()
>>> print(h(), callable(Hello), callable(h))
Hello True True
```

4.7 await expression

The await expression is used in concurrent programming using the *asyncio* package.

4.8 Exponentiation

Exponentiation (raising to a power, like x^y in mathematics) is denoted by double asterisk (**), as in Fortran, so what is x^y in math, would be `x**y` in Python. Its precedence is higher than that of unary operator on its left but lower than unary operator (−, in particular) on its right: `a**-2` is legal and interpreted as `a**(-2)`, while `-2**n` means `-(2**n)`.

The ** operator can be used where pow function is used in C/C++/Java:

```
>>> 2**8
256
>>> 2**-8
0.00390625
>>> 2**(-8)
0.00390625
>>> 8**(-1/3)
0.5
>>> 8**(1/3)
2.0
>>> 8**1/3    # this is just 8/3
2.6666666666666665
>>> 26**10    # number of ten-letter strings
141167095653376
>>> 33**33
129110040087761027839616029934664535539337183380513
```

Remember, though, that ^ *is not* exponentiation, as it is in some other languages: this symbol, as in C/C++/Java, is reserved for bitwise XOR operator!

The pow function exists also in Python (and is one of the builtin functions, so using it doesn't require any imports). The syntax is the same as in C/C++/Java, but there is also a three-argument version: all three arguments must be then ints and the result is the remainder modulo third argument of the first argument raised to the power specified by the second: `pow(x, y, n)` is therefore $x^y \mod n$. For example

pow

```
print(pow(2, 3), pow(2, 3, 5))
```

prints `8 3`

To define ** operator for our classes, we have to implement the magic method `__pow__` (see sec. ??, p. ??). For example:

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

```
class bizzarePow:
    def __init__(self, s):
        self.s = s
    def __pow__(self, p):
        return (self.s*p + '\n')*p*len(self.s)

print(bizzarePow('ab')**3)
print(bizzarePow('X')**5)
```

prints

```
ababab
ababab
ababab
ababab
ababab
ababab

XXXXX
XXXXX
XXXXX
XXXXX
XXXXX
```

4.9 Unary operators

There are three unary operators in Python: Two of them

- numeric negation (sign reversal): − and
- unary plus, +, which applied to any numeric argument returns it unchanged

do not require any explanations, and the third, bitwise negation, ~, will be covered in the section on bitwise operators (see sec. 4.14, p. 63).

4.10 Arithmetic operators

Binary arithmetic operators are generally known to us from C/C++/Java, although there are two specific for Python, and there are some differences in interpretation of some of them.

Five of them are of equal precedence - these are

- * — multiplication (no problems with that one);
- @ — matrix multiplication: no standard Python types support it, but it's used in third-party packages, notably for scientific computing (like *NumPy*[1]);
- / — float division (but see below);
- // — floor division (see below);
- % — remainder (modulo) operator (see below).

The two remaining arithmetic operators have lower precedence than the five mentioned above, but are rather trivial

[1] https://numpy.org

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

- + — addition;
- − — subtraction;

and no surprises here.

Let's talk only about non-trivial operators, whose interpretation is not always so obvious.

4.10.1 Division operators

The / operator corresponds to division. If at least one of the operands is **float** (which is **double** in C/C++/Java), the result is also **float** and no surprises here.

However, in C/C++/Java, this operator applied to integers yields integer, truncating the result towards zero. In Python, even if both operands are **int**s, the result is of type **float**!

So, in Java

```
jshell> 9/3      // JAVA
$1 ==> 3

jshell> 9/4
$2 ==> 2

jshell> 1/3
$3 ==> 0

jshell> -4/3
$4 ==> -1
```

but in Python

```
>>> 9/3          #  PYTHON
3.0
>>> 9/4
2.25
>>> 1/3
0.3333333333333333
>>> -4/3
-1.3333333333333333
```

For integer division in Python, we have a special operator //. Therefore, if we divide two integers using this operator, the result will be truncated but *not*, as in C/C++/Java[1], towards zero, but towards $-\infty$, downwards (and hence it's called *floor division*). In Java, we have

```
jshell> 5/4      // JAVA
$1 ==> 1

jshell> -5/4
$2 ==> -1        // -1.25 truncated towards zero

jshell> Math.floor(-5./4)
$3 ==> -2.0
```

[1] In Java, the **Math.floorDiv** function behaves as the // operator in Python.

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

while in Python

```
>>> import math #  PYTHON
>>> 5//4
1
>>> -5//4       #  -1.25 truncated towards minus infinity
-2
>>> math.floor(-5/4)
-2
```

so // is like float division / followed by taking the floor of the result.

4.10.2 Modulo operator

As in C/C++/Java, modulo, or remainder, operator % does not correspond to the modulo operator in mathematics for negative operands. But the behavior of this operator in Python is different from that in C/C++/Java.

In all these languages, the following formula must be true for all integer values of a and b (with $b \neq 0$, of course)

$$(a/b) * b + a\%b = a$$

where integer division is understood by /.

Using a small program in C++

```
#include <iostream>
#include <cstdio>
void pr(int a, int b) {
    static int fst = 0;
    if (++fst == 1)
        std::printf("C++\n  a   b a/b a%%b (a/b)*b+a%%b\n");
    std::printf("%3d%4d%3d%4d%9d\n",a,b,a/b,a%b,(a/b)*b+a%b);
}
int main() {
    pr( 17,  10);
    pr( 17, -10);
    pr(-17,  10);
    pr(-17, -10);
}
```

which prints

```
C++
  a   b a/b a%b (a/b)*b+a%b
 17  10  1   7        17
 17 -10 -1   7        17
-17  10 -1  -7       -17
-17 -10  1  -7       -17
```

we can verify that indeed $(a/b)*b+a\%b = a$. The rule for negative operands is simple here: apply modulo operator to absolute values of the operands and then take the result with the sign of a (ignoring the sign of b).

However, for this formula to hold, the definition of the modulo operator in Python must be different than that in C/C++/Java, because the definition of integer division is different!

A similar program in Python

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

```
def pr(a, b, fst=[]):
    if not fst:
        print("PYTHON\n  a   b a//b a%b (a//b)*b+a%b")
        fst.append(1)
    print(f"{a:3}{b:4}{a//b:4}{a%b:4}{(a//b)*b+a%b:9}");

pr( 17,  10);
pr( 17, -10);
pr(-17,  10);
pr(-17, -10);
```

prints

```
PYTHON
  a   b a//b a%b (a//b)*b+a%b
 17  10   1   7        17
 17 -10  -2  -3        17
-17  10  -2   3       -17
-17 -10   1  -7       -17
```

and the easiest way to grok the result with negative arguments is to remember that $a\%b = a - (a/b) * b$ — this works for both Python and C/C++/Java with / assumed to be the integer division appropriate for the language at hand (// in Python).

As we can see, the best we can do is to avoid using integer divisions and modulo operators if we suspect that operands might be negative.

4.11 Comparisons

4.11.1 Arithmetic comparisons

There are six arithmetic comparison operators comparing *values* of objects, which are well known from many other languages: <, < =, ==, !=, > = and >. There is no point in explaining them, but one of their features is specific to Python — there are separate rules for chains of these operators and these rules cannot be expressed as associativity (right or left). Such chains would be illegal in Java. In C/C++, they would compile but give nonsensical results; for example

```
int a = 1, c = 3;
cout << boolalpha << (c < a < a) << endl;
```

prints **true**.

Such chains of (in)equalities *are* legal and make sense in Python, though. For example,

```
a < b < c
```

is neither

```
(a < b) < c
```

(as it would be in C/C++), nor

```
a < (b < c)
```

but

```
a < b and b < c
```

(but b will be evaluated once only). Chains of (in)equalities is equivalent to ANDed chains of separate binary operations evaluated form left to right:

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

```
>>> 5 < 7 > 2
True
>>> 5 < 7 and 7 > 2
True
>>>
>>> 5 < 7 > 7
False
>>> 5 < 7 and 7 > 7
False
>>>
>>> a = 5; b = 7
>>> a < b == b - a + 5
True
>>> a < b and b == b - a + 5
True
>>>
>>> a == b - 2 < b >= a
True
>>> a == b - 2 and b - 2  < b and b >= a
True
```

4.11.2 Identity comparisons

There are two identity comparison operators: **is** and **is not**. They are (rather seldom) used to check if two objects are actually the same object, or two distinct objects, perhaps representing the same value. Remember, that == is supposed to compare values, not IDs (addresses). It is defined for many Python builtin types. But, if it's not defined — e.g., for user defined types — the default implementation, defined in class **object**, will be used, which is equivalent to **is** — comparing addresses (this is analogous to the **equals** method in Java).

For type **str** (string), the == operator is, of course, well defined (as is **String::equals** in Java):

```
>>> a = 'Hello, '; b = 'World'
>>> s1 = a + b
>>> s2 = 'Hello, World'
>>> s1 is s2
False
>>> s1 == s2
True
>>> s1 is not s2
True
```

We remember, that small integers are created once only and all references to the same small value refer to exactly the same object (see sec. 3.4.1, p. 36): we can check it using the **is** operator

```
>>> a = 17; b = 17
>>> a == b
True
>>> a is b
True
```

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

```
>>> id(a), id(b)
(9801760, 9801760)
>>>
>>> # but:
>>>
>>> a = 1200; b = -1000; c = a - b - 1000
>>> a == c
True
>>> a is c
False
>>> id(a), id(c)
(140120910778288, 140120910340336)
```

4.11.3 Membership test operators

Membership tests belong to the same precedence group as comparisons. Operators **in** and its negation **not in** check if an object is a member of a collection: list, tuple, dictionary, set, but also string, which is a sequence of individual characters. For dictionaries, a key is searched for, values are ignored. The usage of these operators is straightforward:

```
ls = [(1, 2), (3, 4, 5), (6, 7, 8, 9)]
di = {'one': 1, 'two':2, 'three':3}
se = {1, 2, 3, 4}
st = 'abcde'
print((1, 2) in ls)    # True
print((2, 1) in ls)    # False
print(4 in ls[1])      # True
print(4 in ls[2])      # False
print('one' in di)     # True
print(2 in di)         # False
print(4 in se)         # True
print(5 in se)         # False
print('cd' in st)      # True
print('f' in st)       # False
```

4.12 Boolean not, and and or operators

Boolean operators: **not**, **and** and **or** correspond to operators known from C/C++/Java: **!**, **&&** and **||**, respectively.

As in these languages, **and** and **or** operators are short-circuited — the second operand is not evaluated if the result is known after evaluation of the first.

As we remember (see sec. 3.4.2, p. 38) values of any type may be used in a context requiring a Boolean: numerical values equal to zero, empty sequences and collections, **None** and **False** are all "falsy", everything else is "truthy".

It follows, that operands of Boolean operators can be of any type, not necessarily **bool** — they will be treated as **True** or **False** anyway. But what about the type of a return value?

For the **not** operator, the return value is **True** or **False**:

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

```
>>> not {}
True
>>> not {7}
False
>>> not 3
False
>>> not 0
True
>>> not [1, 2, 3]
False
>>> not []
True
>>> not 'a'
False
>>> not ''
True
>>> not False
True
>>> not True
False
```

However, the **and** and **or** operators do not return **True** or **False**, but simply one of their operand (which, however, always can be interpreted as **True** or **False** in the logical context) according to the following rules:

- Conjunction: `expr1 and expr2` returns
  - `expr1` (and `expr2` is not even evaluated) if `expr1` is "falsy";
  - `expr2` otherwise.
- Alternative: `expr1 or expr2` returns
  - `expr1` (and `expr2` is not even evaluated) if `expr1` is "truthy";
  - `expr2` otherwise.

For example:

```
>>> a = 0 and [1, 2]
>>> a, type(a), bool(a)
(0, <class 'int'>, False)
>>>
>>> a = -1 and [1, 2]
>>> a, type(a), bool(a)
([1, 2], <class 'list'>, True)
>>>
>>> a = -1 and {}
>>> a, type(a), bool(a)
({}, <class 'dict'>, False)
>>>
>>> a = [] or {}
>>> a, type(a), bool(a)
({}, <class 'dict'>, False)
>>>
>>> a = [] or {1, 2}
```

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

```
>>> a, type(a), bool(a)
({1, 2}, <class 'set'>, True)
```

As this example shows, the **bool** function — strictly speaking, a constructor of the **bool** class — can be used to convert any value to a "true" Boolean, that is **True** or **False**. These, in turn, are actually just **int**s 1 or 0. This fact allows us to easily simulate the *logical* XOR operator. If a and b are any logical expressions, then `bool(a)^bool(b)` (or `bool(a)+bool(b)==1`) can be used as substitutes for the logical XOR.

4.13 Conditional expression

Conditional expression corresponds to ternary operator ?: in C/C++/Java. The syntax is a little different, though:

```
expr1 if condition else expr2
```

and the value of the whole expression is the value of

- expr1 if `condition` evaluates to **True** (and then expr2 is not evaluated at all);
- expr2 if `condition` evaluates to **False** (and then expr1 is not evaluated).

Note that it is acceptable if the value of expr1 and/or expr2 is **None** — for example, if it is a function call and the function doesn't return anything (i.e., returns **None**):

```
a = 6
b = print('even') if a % 2 == 0 else print('odd')
print(b)
```

prints

```
even
None
```

Another example:

```
n = p = 7
while p > 1:
    print(n, end=' ')
    p, n = n, n//2 if n%2 == 0 else 3*n + 1
```

prints the Collatz (hailstone) sequence starting from 7

```
7 22 11 34 17 52 26 13 40 20 10 5 16 8 4 2 1
```

The following program checks which of the Fibonacci numbers F_n for $n \in [3, 15]$ are prime; here, we use not only conditional expression, but also, in order to avoid recalculating Fibonacci number, assignment expression (see the last line):

Listing 16 ABB-Cond/Cond.py

```
1 #!/usr/bin/env python
2
```

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

```
def is_prime(n):
    if n <= 1: raise ValueError
    if n <= 3: return True
    if n % 2 == 0 or n % 3 == 0: return False
    k = 5
    while k**2 <= n:
        if n % k == 0 or n % (k+2) == 0: return False
        k += 6
    return True

def fib(n):
    f = 0; s = 1
    if n < 2: return n
    for i in range(n-1): f, s = s, f+s
    return s

for n in range(3, 16):
    print(f"{v} yes" if is_prime(v := fib(n)) else f"{v} no")
```

The program prints

```
2 yes
3 yes
5 yes
8 no
13 yes
21 no
34 no
55 no
89 yes
144 no
233 yes
377 no
610 no
```

Conditional expressions may be nested

```
expr1 if condition1 else expr2 if condition2 else expr3
```

Using such a vague form is discouraged, as it's hard to read and understand. Conditional expressions are right-associative (as for `?:` in C/C++/Java), so the above is equivalent to

```
expr1 if condition1 else (expr2 if condition2 else expr3)
```

which is easier to comprehend; the value of the whole expression is therefore

- `expr1` if `condition1` is true — the remaining conditions and expressions are not evaluated then;
- `expr2` if `condition1` is false, but `condition2` is true;
- `expr3` if both `condition1` and `condition2` are false.

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

For example:

```
a = 1; b = 7
x1 = 'expr1' if a < b else 'expr2' if a > b else 'expr3'
x2 = 'expr1' if a > b else 'expr2' if a < 2 else 'expr3'
x3 = 'expr1' if a > b else 'expr2' if b < a else 'expr3'
print(x1)
print(x2)
print(x3)
```

prints

```
expr1
expr2
expr3
```

Conditional expressions can be used in list comprehensions; for example here we replace all numbers from range $[-1, 8)$ that are smaller than 2 by 2 and those greater than 5 by 5:

```
ls = [5 if n > 5 else 2 if n < 2 else n for n in range(-1, 8)]
print(ls)  # prints [2, 2, 2, 2, 3, 4, 5, 5, 5]
```

4.14 Bitwise operators

There are six bitwise operators in Python; in the order of descending precedence, these are:

- bitwise NOT (~),
- left (<<) and right (>>) shifts,
- bitwise AND (&),
- bitwise XOR (^),
- bitwise OR (|).

In Python, bitwise operators are sometimes tricky. The problem with them originates from the fact that integer values are represented in a complicated way (see sec. 3.4.1, p. 36). This representation doesn't have anything to do with integers in C/C++/Java. In these languages, integer values are represented always by a fixed-length sequence of bits, with the most significant bit, the so called sign bit, treated specially (for *signed* types). Because the length (number of bits) is known, it is also known which bit is the most significant one. However, representation of integers in Python is completely different and by construction allows to represent arbitrary large numbers, whose "normal" two's complement binary representation would require arbitrarily long sequences of bits. As the first (from the left) non-zero bit is always 1, it would be not known if contribution from this bit is to be taken with plus sign (positive number) or with minus sign (negative numbers).

Shift operators work as expected, but with some quirks. Shifting to the left is *always* equivalent to multiplication by a power of 2:

```
m << n   ≡   m * pow(2, n)
```

For positive m, we can easily check that this indeed corresponds to shifting bits of "normal" binary representation of m by n bits to the left, with zeros entering from the right (below we use **bin** function to display binary representation of a *positive* number):

bin

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

```
>>> a = 153
>>> bin(a)
'0b10011001'
>>> b = a << 3
>>> bin(b)
'0b10011001000'
>>> bin(a*2**3)
'0b10011001000'
```

Note that in a fixed-length representation, as in C/C++/Java, leftmost bits may be dropped if they are shifted beyond the left "edge" of the binary representation of the number: then shifting will *not* be equivalent to multiplication by a power of 2, and the result may even become negative (for signed types): the following snippet in C++

```
int a = 100'000'000;
cout << (a << 10) << endl;
```

prints

```
-679215104
```

and similarly, in Java

```
jshell> int a = 100_000_000
a ==> 100000000

jshell> System.out.println(a << 10)
-679215104
```

In Python, however, it's not a problem, as the length of the binary representation of a number is not limited:

```
>>> a = 1_000_000
>>> b = a << 10
>>> c = a * 2**10
>>> bin(a)
'0b11110100001001000000'
>>> bin(b)
'0b11110100001001000000000000000000'
>>> bin(c)
'0b11110100001001000000000000000000'
>>> b, c
(1024000000, 1024000000)
>>> b == c
True
```

With negative numbers the situation is a little worse. The equivalence

```
m << n    ≡    m * pow(2, n)
```

still holds. However, it's not so simple to see the bit pattern, because for negative numbers, the **bin** function shows the bit representation of the absolute value with minus sign added at the front, not the two's complement bit representation of the number itself:

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

```
>>> a = 913
>>> b = -a
>>> bin(a)
'0b1110010001'
>>> bin(b)
'-0b1110010001'
```

However, we can use the **to_bytes** method to see individual bytes:[1] to_bytes

```
>>> a = 913
>>> b = -a
>>> c = b << 2
>>> d = b * 2**2
>>> c == d
True
>>> c, d
(-3652, -3652)
>>> e = d.to_bytes(4, 'big', signed=True) # 4 bytes as int in Java
>>> e
b'\xff\xff\xf1\xbc'
>>> int.from_bytes(e, 'big', signed=True)
-3652
```

\xff\xff\xf1\xbc is[2] 11111111111111111111000110111100 in binary system and indeed, we can check in Java that this *is* -3652:

```
jshell> a = -3652
a ==> -3652

jshell> Integer.toBinaryString(a)
$1 ==> "11111111111111111111000110111100"
```

Bitwise AND (&), bitwise OR (|) and bitwise XOR (^) work as expected (again, it's better not to touch negative numbers):

```
>>> m = 0b11001010
>>> n = 0b11010
>>> bin(m & n)
'0b1010'
>>> bin(m | n)
'0b11011010'
>>> bin(m ^ n)
'0b11010000'
```

Bitwise NOT (bit-flip, ~) is not so trivial. As integers are unlimited, we can imagine that their representation for a positive number contains an infinite sequence of zeros to the left of the most significant bit which is set. Flipping the bits would

[1] Calling this method, we specify that the result is to be on 4 bytes (as **int** in C/C++/Java) and we want to see bytes ordered from the most to the least significant. [Instead of little or big, we can use sys.byteorder which gives the byte order on the current platform; usually little.]

[2] Notation \xhh represents one byte with the numerical value 0xhh, where hh stands for exactly two hexadecimal digits.

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

then create an infinite sequence of ones. And this is what happens: of course, all these ones are not physically present, but conceptually they are there.

Let us show what's going on when we bitwise negate a number. Consider, for example, 321_{10} (=101000001_2). To represent it, 9 bits are needed

```
>>> bin(321)
'0b101000001'
>>> (321).bit_length()
9
```

Python adds one leading zero (as it is positive), takes the sequence of 10 bits and negates them:

```
0101000001          # 321 plus leading 0
1010111110          # flipped
```

and this is interpreted as two's complement representation of a 10-bit integer; the first bit is treated as the sign bit, so its contribution will be negative and the number is $-2^9 + 2^7 + 2^5 + 2^4 + 2^3 + 2^2 + 2^1$ or $-512 + 128 + 32 + 16 + 8 + 4 + 2 = -322$.

```
>>> a = 321
>>> nega = ~a
>>> bin(a)
'0b101000001'
>>> bin(nega)
'-0b101000010'
>>> a, nega
(321, -322)
>>> negabytes = nega.to_bytes(2, 'big', signed=True)
>>> negabytes
b'\xfe\xbe'
>>> int.from_bytes(negabytes, 'big', signed=True)
-322
```

\xfe\xbe is 1111111010111110: there is "infinite" sequence of ones at the beginning, so the number is negative and only the last one is significant (giving, as we showed above, -322).

As we remember, in two's complement notation, reversing the sign is equivalent to flipping all bits followed by addition of 1. We can check, that this feature is preserved:

```
>>> ~321+1
-321
>>> ~-9987654321+1
9987654321
>>> ~123123123123123123+1
-123123123123123123
>>> ~99999999999999999999999999999999999999999999999+1
-99999999999999999999999999999999999999999999999
```

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

Chapter 5

Functions

5.1 Defining functions. Scopes of variables

In Python, functions are represented, as everything else, by objects. A simple definition of a function may look like this

```
def mul(x, y):
    return x*y

def printData(x):
    print('Data is {}'.format(x))   # No return
```

As we can see, the return type is *not* declared. Actually, all Python functions return something — even if a function ends with just **return,** (without specifying any value to be returned) or does not contain any **return** statements at all — it then returns **None**.

Also, parameters types remain unspecified: their type will be determined dynamically by the type of arguments passed to the function every time it is being invoked. This means, that the same function may be called with arguments of completely different and unrelated types and, if the syntax of the body of the functions turns out to be correct for these types, all such invocations will be correct — this feature is called *duck typing.*

```
>>> def mul(x, y):
...     return x*y
...
>>> def printData(x):
...     print('Data:', x, type(x))
...
>>> printData(1.5)
Data: 1.5 <class 'float'>
>>>
>>> printData((1, 2, 3))
Data: (1, 2, 3) <class 'tuple'>
>>>
>>> printData([1, 'Eve'])
Data: [1, 'Eve'] <class 'list'>
>>>
>>> printData(r := mul(3, 3))
Data: 9 <class 'int'>
>>>
>>> printData(r := mul((1+2j), (2-4j)))
Data: (10+0j) <class 'complex'>
>>>
>>> printData(r := mul([1, 2], 3))
Data: [1, 2, 1, 2, 1, 2] <class 'list'>
```

Contrary to most other languages, definition of a function is an executable statement. For example, the definition of the **printData** function above is syntactically similar to

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

just defining a numeric variable x = 7. The name printData is, as usual, the name of a *reference* which points to the object representing our function. We can create other references to the same object:

```
>>> pp = printData
>>> pp(1+2j)
Data: (1+2j) <class 'complex'>
>>> printData = 17
>>> printData
17
```

However, the original name of a function is kept as one of the attributes of the object representing it, but this can be easily changed:

```
>>> dir(pp)
['__annotations__', '__builtins__', '__call__', '__class__',
 '__closure__', '__code__', '__defaults__', '__delattr__',
 '__dict__', '__dir__', '__doc__', '__eq__', '__format__',
 '__ge__', '__get__', '__getattribute__', '__globals__',
 '__gt__', '__hash__', '__init__', '__init_subclass__',
 '__kwdefaults__', '__le__', '__lt__', '__module__',
 '__name__', '__ne__', '__new__', '__qualname__', '__reduce__',
 '__reduce_ex__', '__repr__', '__setattr__', '__sizeof__',
 '__str__', '__subclasshook__']
>>> pp.__name__
'printData'
>>> pp.__name__ = 'pp'
>>> pp.__name__
'pp'
```

Functions, being objects, can be created inside other functions, passed as arguments to and/or returned from other functions. Functions returning or taking as their argument(s) other functions are called **higher order functions**.

In the example below, the **compos** function takes two functions, f and g, and returns also a function, namely their composition[1]

Listing 17 AAP-FunCompos/Compos.py

```
1  #!/usr/bin/env python
2
3  def compos(f, g):
4      def _h(x):
5          return f(g(x))
6      # just for fun...
7      _h.__name__ = g.__name__ + '_and_then_' + f.__name__
8      return _h
9
10 def squa(x):
11     return x*x
12 def add2(x):
```

[1]The composition of functions f i g is a function $f \circ g$ such that $(f \circ g)(x) \equiv f(g(x))$.

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

```
13      return x+2
14
15  sqad = compos(squa, add2)
16  adsq = compos(add2, squa)
17
18  print(type(squa), type(add2))
19
20  print('name of sqad:', sqad.__name__)
21  print('name of adsq:', adsq.__name__)
22
23  print(sqad(2))    # 16
24  print(adsq(2))    #  6
```

The program prints

```
<class 'function'> <class 'function'>
name of sqad: add2_and_then_squa
name of adsq: squa_and_then_add2
16
6
```

Let us look at another example:

Listing 18 AAR-FunCount/Counter.py

```
1  #!/usr/bin/env python3
2
3  def getCounter():
4      n = 0
5      def count():
6          nonlocal n
7          n += 1
8          return n
9      return count
10
11 f = getCounter()
12 for i in range(3):
13     print(f(), end = ' ')    # 1 2 3
14
15 print('\n' + str(type(f)))  # <class 'function'>
16 print(f.__name__)           # count
```

What is interesting here is the **nonlocal** keyword (line 6). This leads us to the notion of a **scope**: a part of a program in which a set of name bindings is visible and directly (just by a single, non-qualified name) accessible. At run-time, names are searched for in the following scopes:

1. Innermost scope with **L**ocal names.
2. The scope of **E**nclosing functions, if any (applicable for nested functions).
3. The current module's **G**lobal scope.

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

4. The scope of the **B**uilt-in entities.

When a name is used, its meaning is searched for in these scopes in the order as above — this is known as the **LEGB** rule. There are some principles governing the search:

- in a local scope (inside a function), the name appearing on the left-hand side of an assignment is considered to be the name of a local variable (which is created, if it doesn't exist);
- otherwise, a name refers to a local variable, if it already exists; if not, the surrounding non-global scope (if there is any) is examined, then the global scope of the module, and finally the built-in namespace;
- if a global variable is to be assigned a new value inside a local scope (in a function, for example), it has to be declared as **global** before it is used.
- Suppose we have an inner scope inside another local scope (like in definition of a function inside another, surrounding function). Then, in order to assign a new value to a variable from the surrounding scope (but *not* the global one) inside the inner scope, it has to be declared in the inner scope as **nonlocal** before being used.

For example, the code below will not work: `n` inside the function is assigned a value, so it is considered to be a new local variable. However, the value which is to be assigned depends itself on the value of `n`:

```
>>> n = 1
>>> def fun():
...     n = n + 7
...
>>> fun()
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "<stdin>", line 2, in fun
UnboundLocalError: local variable 'n' referenced before assignment
```

but this will work

```
>>> n = 1
>>> def fun():
...     global n
...     n = n + 7
...
>>> fun()
>>> print(n)
8
```

Similarly, this is incorrect

```
>>> def fun():
...     n = 9
...     def inn():
...         n += n
...         return n - 8
...     return inn
...
```

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

```
>>> f = fun()
>>> print(f())
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "<stdin>", line 4, in inn
UnboundLocalError: local variable 'n' referenced before assignment
```

but the code below is OK: as we don't attempt to assign a new value to n, it will be searched for, and found, in the surrounding scope)

```
>>> def fun():
...     n = 9
...     def inn():
...         return n - 8
...     return inn
...
>>> f = fun()  # f is now inn
>>> print(f())
1
```

The code below is correct as well, as is this version, where we declared n as **nonlocal**

```
>>> def fun():
...     n = 9
...     def inn():
...         nonlocal n
...         n += n
...         return n - 8
...     return inn
>>> f = fun()
>>> print(f())
10
```

And another example. The variable n is created when the **halves** function is called (once only). Inside the nested function **addhalf**, it's declared as **nonlocal**, so it is the same variable defined on line 7. Note that it is used (and redefined) inside the **addhalf** function which is returned from **halves**. One could think that after the return, the variable n doesn't exist any more, as it was local in the **halves** function. However, it will be copied do the so called **closure** of **addhalf**, and can be used and modified by this function:

Listing 19 AAS-Scopes/Scopes.py

```
1  #!/usr/bin/env python
2
3  n = 3.0                              # global
4  print('Global n:', id(n), n, end = '\n\n')
5
6  def halves():
7      n = -0.5                         # local
8      print('Local  n:', id(n), '\n')
9      def addhalf():
10         nonlocal n                   # n from line 7
```

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

```
11          print('Inner1 n:', id(n), n) # same as local
12          n += 0.5
13          print('Inner2 n:', id(n), n) # changed now
14          return n
15      return addhalf
16
17  f = halves()
18  for i in range(2):
19      print(f(), end = '\n\n')
20
21  print('Global n:', id(n), n)
22  print('From closure:',f.__closure__[0].cell_contents)
```

The output of the program is

```
Global n: 139640753183728 3.0

Local  n: 139640753183472

Inner1 n: 139640753183472 -0.5
Inner2 n: 139640753183632 0.0
0.0

Inner1 n: 139640753183632 0.0
Inner2 n: 139640753183600 0.5
0.5

Global n: 139640753183728 3.0
From closure: 0.5
```

[See also discussion in section on lambdas, sec. 5.3, p. 79]

Let us notice, that the interpreter will add local variables from the encompassing scope to the closure of an inner function only if they are actually used in this function. For example, in the following program

Listing 20 AEB-Closures/Closures.py

```
1   #!/usr/bin/env python
2
3   def outer():
4       a = 10                    ❶
5       b = 20
6
7       def innA(arg):            ❷
8           return arg + a
9
10      def innB(arg):
11          return arg + b
12
13      def innC(arg):
```

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

```
14          return arg + a + b
15
16      return innA, innB, innC     ❸
17
18  fA, fB, fC = outer()
19
20  closA = fA.__closure__
21  closB = fB.__closure__
22  closC = fC.__closure__
23                                                  ❹
24  print(type(closA).__name__, 'len =', len(closA)) # tuple len = 1
25  print(type(closB).__name__, 'len =', len(closB)) # tuple len = 1
26  print(type(closC).__name__, 'len =', len(closC)) # tuple len = 2
27
28  print(type(closA[0]).__name__,                  ❺
29              'valA =', closA[0].cell_contents)   # cell valA = 10
30  print(type(closB[0]).__name__,
31              'valB =', closB[0].cell_contents)   # cell valB = 20
32  print(type(closC[0]).__name__,
33              'valC0 =', closC[0].cell_contents)  # cell valC0 = 10
34  print(type(closC[1]).__name__,
35              'valC1 =', closC[1].cell_contents)  # cell valC1 = 20
```

we define a function **outer** with two local variables, a and b (❶). Three inner functions are then defined (❷): one uses only a, one only b, and the third one both a and b. They are returned as a tuple (❸) and assigned to variables fA, fB and fC. We then define variables closA, closB and closC representing their closure attribute. As we can see from the output (❹), they are tuples of length 1 in the first two cases and of length 2 in the third one. Printing their contents (❺) convinces us that the first only "remembers" the value of a, the second only that of b, and the third both of them.

Contrary to languages like C/C++/Java, blocks of code (for example under **if** of inside a loop) do *not* create new scopes. Variables defined in such blocks are just dynamically added to the dictionary of local names and will stay there even when the flow of control leaves the block. Consider the following example

```
x = int(input('Enter x (negative or positive) '))

if x >= 0:
    a = 'a'
else:
    b = 'b'

print('"a" exists?', 'a' in locals())
print('"b" exists?', 'b' in locals())
```

If, running this program, we enter a positive number, variable a will exist after the if statement, but b will not

```
Enter x (negative or positive) 4
"a" exists? True
"b" exists? False
```

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

while with a negative number as input

```
Enter x (negative or positive) -3
"a" exists? False
"b" exists? True
```

b will exist, but a will not.

5.2 Passing arguments

Let's look at the following example

```
>>> def fun(a, b, c):
...     return f'{a=} {b=} {c=}'
...
>>> fun(1, 'Hello', 2.5)
"a=1 b='Hello' c=2.5"
>>>
>>> fun(c='World', a=17, b=(3, '3'))
"a=17 b=(3, '3') c='World'"
>>>
>>> fun(21, c=8.28, b='Ottawa')
"a=21 b='Ottawa' c=8.28"
>>>
>>> fun(a=21, 'Ottawa', c=8.28)
  File "<stdin>", line 1
    fun(a=21, 'Ottawa', c=8.28)
                              ^
SyntaxError: positional argument follows keyword argument
```

Parameters of **fun** are *positional* — there are exactly three of them and they don't have any default values. Therefore, when invoking the function, we have to pass exactly three arguments. However, as we can see from the second invocation, their order can be changed, if *names* of parameters are specified (these are called **keyword arguments**).

In the third invocation, the name of the first argument hasn't been specified, so it must correspond to the first parameter. Generally, *unnamed* arguments must be given first and in the correct order; the remaining arguments may be passed to the function in any order as keyword arguments. However, if an argument was passed by keyword, all the remaining arguments must also be passed by keyword; otherwise we'll get an exception, as in the last invocation above.

It may happen that a function expects several arguments, but in our program we have them collected into a sequence (a list or a tuple). We cannot pass just a list (or a tuple) because that would be *one* argument (of type **list** or **tuple**). However, when passing a collection, we can "unpack" it into a sequence of its single elements separately using the *-operator:

```
>>> def fun(a, b, c):
...     return f'{a=} {b=} {c=}'
...
>>> lst = ['a', 1, 'Hello']
>>> fun(*lst)
```

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

```
"a='a' b=1 c='Hello'"
>>> tpl = 6, 'World', 1.5
>>> fun(*tpl)
"a=6 b='World' c=1.5"
```

In a similar way, we can unpack a dictionary into a sequence of its entries with names of parameters as keywords and pass it as a sequence of keywords arguments; we can unpack a dictionary with **-operator:

```
>>> def fun(a, b, c):
...     return f'{a=} {b=} {c=}'
...
>>> dct = dict(b = 'Hello', c = 1.25)
>>> fun('First', **dct)
"a='First' b='Hello' c=1.25"
```

Here, the first argument was not a keyword one, so it has to correspond to the first positional parameter; the remaining arguments will be keyword arguments after unpacking the dictionary.

Another example:

```
def fun(a, b, c, d, e, f):
    print(a, b, c, d, e, f)

fun(1, *(2, 3), f=6, *(4, 5))
fun(*(1, 2), e=5, *(3, 4), f=6)
fun(1, **{'b': 2, 'c': 3}, d=4, **{'e': 5, 'f': 6})
fun(c=3, *(1, 2), **{'d': 4}, e=5, **{'f': 6})
fun(d=4, *(1, 2), **{'c': 3}, **{'f': 6}, e=5)
```

prints

```
1 2 3 4 5 6
1 2 3 4 5 6
1 2 3 4 5 6
1 2 3 4 5 6
1 2 3 4 5 6
```

Some or all arguments of a function can have default values (as in C++, but not Java or C). If there are parameters with default values, they have to be listed *after* parameters without them. Functions with default arguments can be called with fewer arguments — the interpreter will use the default values for arguments' values that are missing. Let's look at the example:

Listing 21 AAU-DefArgs/DefArgs.py

```
1 #!/usr/bin/env python3
2
3 def f(a, b='defaultB', c=None):
4     if c == None:
5         print("c hasn't been specified!", end = ' ')
```

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

```
6       print(f'{a=} {b=} {c=}')
7   f(1, 2, 3)  # a=1 b=2 c=3
8   f(1, 2)     # c hasn't been specified! a=1 b=2 c=None
9   f(1)        # c hasn't been specified! a=1 b='defaultB' c=None
10  f(c='c', a='a', b='b')  # a='a' b='b' c='c'
11
12  i = 7
13  def fundef(a, b=i): # default value of b will always be 7!
14      return a + b;
15
16  i += 10
17    # this will print 'i = 17 fundef(1) = 8'
18  print('i =', i, 'fundef(1) =', fundef(1))
19
20    # important example
21  def funlist(val, lst = []): # default list created once!
22      lst.append(val)
23      return lst;
24  funlist(1)
25  funlist(2)
26  funlist(3)
27  print(funlist(4))               # [1, 2, 3, 4]
28  print(funlist('c', ['a', 'b'])) # ['a', 'b', 'c']
29  print(funlist('d'))             # [1, 2, 3, 4, 'd'] !
```

The example illustrates a very important problem that we have to keep in mind when using default values for arguments of functions. The object representing a default value is created once only: when the interpreter evaluates the definition of a function. In all subsequent invocation, reference to exactly the same object containing the default value will be used. For example, in line 13, we define a function with default value. This default value will be 7 "forever". Even if we change variable i on line 16, and invoke the function again, the old value of the default value, i.e., 7, will be used. This is clear when the default object is immutable (a number, a strings, ...). However, as we can see from the example with empty list as the default value (line 21), it can lead to surprises if this object *is* mutable (like, e.g., a **list**). Then in all invocations, when the corresponding argument is omitted, the same object is used and its value will be as it was left after the previous invocation with this argument omitted. Consequently, in all subsequent invocations, the list will *not* be empty!

Python supports variadic functions, i.e., functions with arbitrary number of arguments. A sequence of arguments (perhaps empty) is denoted, on the parameter list, by *args (the name args is conventional, what matters is the asterisk). When invoking the function, all arguments (after the obligatory, positional arguments, but before keyword arguments, if any) will be collected into a tuple and passed to the function — therefore, inside the function, args is the reference to a, perhaps empty, tuple:

```
#!/usr/bin/env python

def f(a, *args):
    print("Positional:", a, end=', ')
```

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

```
        if len(args) == 0:
            print("No var args")
            return
        print("Var args:", end=' ')
        for i, v in enumerate(args):
            print(f'{i+1} -> {v};', end=' ')
        print()

    f(1)
    f(2, 'A', 1.5, 'B')
    f(3, ('X', 'Y'))
    f(4, *('X', 'Y'))
```

The program prints

```
Positional: 1, No var args
Positional: 2, Var args: 1 -> A; 2 -> 1.5; 3 -> B;
Positional: 3, Var args: 1 -> ('X', 'Y');
Positional: 4, Var args: 1 -> X; 2 -> Y;
```

Note the last two invocations: in the third we pass tuple `('X', 'Y')` as a single argument, while in the fourth, we unpack the tuple into two individual arguments, which will be gathered back to a tuple args on the function's side.

At the very end of the parameter list of a function, one can write **kwargs (again, the name kwargs is conventional, what matters is the double asterisk). When calling the function, we pass any number of keyword arguments (in the form `k1=v1, k2=v2,...`). They will be gathered into a dictionary and passed to the function — therefore, inside the function, kwargs is the reference to a, perhaps empty, **dict**:

Listing 22 AAW-ArgsKwargs/ArgsKwargs.py

```
#!/usr/bin/env python

def f(a, *args, **kwargs):
    print('Positional:', a, end=', ')
    if len(args) == 0:
        print('No var args;', end=' ')
    else:
        print('Var args:', end=' ')
        for i, v in enumerate(args):
            print(f'{i+1} -> {v};', end=' ')
    if len(kwargs) == 0:
        print('No kwargs;')
    else:
        print('KWargs:', end=' ')
        for k, v in kwargs.items():
            print(f'{k} : {v}; ', end = ' ')
        print()

f(1)
f(2, 'x', 'y')
f(3, fName='Sue', lName='Lee')
```

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

```
22 f(4, 'x', 'y', w=24, h=30)
23
24 lst = ['a', 'b']
25 dic = dict(sue="S", joe="J")
26 f(5, *lst, **dic)
```

In the last invocation, we first collected our arguments into a list and a dictionary, and then we unpacked them when passing to the function: they are packed back by the interpreter to the args tuple and kwargs dictionary on the function's side.

The program prints:

```
Positional: 1, No var args; No kwargs;
Positional: 2, Var args: 1 -> x; 2 -> y; No kwargs;
Positional: 3, No var args; KWargs: fName : Sue;  lName : Lee;
Positional: 4, Var args: 1 -> x; 2 -> y; KWargs: w : 24;  h : 30;
Positional: 5, Var args: 1 -> a; 2 -> b; KWargs: sue : S;  joe : J;
```

It is also possible to force some parameters to be "positional-only" or "keyword-only". There is a "special" parameter: just single /. It means that arguments preceding this position on the parameter list *cannot* be given names:

```
>>> def fun(a, b, c, /, d):
...     print(a, b, c, d)
...
>>> fun(1, 2, 3, 4)
1 2 3 4
>>> fun(1, 2, 3, d=4)
1 2 3 4
>>> fun(1, 2, c=3, d=4)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: fun() got some positional-only arguments
  passed as keyword arguments: 'c'
```

We can also force some arguments to be passed as keyword-only. One way of doing it is as follows

```
>>> def fun(*args, a):
...     print(args, a)
...
>>> fun(1, 2, 3, a=4)
(1, 2, 3) 4
>>> fun(a=4)
() 4
>>> fun(1, 2, 3, 4)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: fun() missing 1 required keyword-only argument: 'a'
```

Here we have to specify the name of a — otherwise it would be "swallowed" by the args tuple.

There is also a "special" parameter denoted by *. All arguments after the position marked with this "parameter" *must* be passed by keyword

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

```
>>> def fun(a, b=2, *, c, d):
...     print(a, b, c, d)
...
>>> fun(1, c=3, d=4)
1 2 3 4
>>> fun(1, 2, c=3, d=4)
1 2 3 4
>>> fun(1, 2, 3, d=4)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: fun() takes from 1 to 2 positional arguments but
3 positional arguments (and 1 keyword-only argument) were given
```

An example is provided by the built-in function **sorted** which is declared as sorted

```
sorted(iterable, /, *, key=None, reverse=False)
```

which means that the first argument (an iterable) must be passed without any name, while using optional arguments (key and/or reverse) requires specifying their names

```
>>> lst = ['Jane', 'Sophia', 'Liz']
>>> sorted(lst, reverse=True)
['Sophia', 'Liz', 'Jane']
>>> sorted(lst, key=lambda e: e[-1])
['Sophia', 'Jane', 'Liz']
>>> sorted(lst, reverse=True, key=lambda e: e[-1])
['Liz', 'Jane', 'Sophia']
```

5.3 Lambdas

Another way of creating functions is by using a **lambda** expression. Despite the protests of Guido van Rossum, they were added to Python under pressure of users (and fans of the Lisp and Haskell programming languages).
The syntax of a lambda function is very simple:

```
lambda args_list: expr
```

where args_list is a list of comma separated list of formal parameters and expr is an expression (i.e., something that has a value) in which the parameters can be used. The value of expr will be returned by the function represented by this lambda; note, however, that there is no explicit **return** statement here.

As expr has to be *one expression*, one cannot, for example, use **if-else** statement; conditional expressions, however, can be used:

```
pal = lambda ls: 'palindrom' if ls == ls[::-1] else 'not palindrom'

print(pal('rotor')) # prints 'palindrom'
print(pal('rover')) # prints 'not palindrom'
```

The example above illustrates also the fact, that references to lambdas may be given names, which then can be used exactly as names references to functions

```
q = lambda x: 0.25*x**2 - x - 3
print(q(2), q(3), q(-4))          # prints -4.0 -3.75 5.0
```

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

Lambda expressions are of type **function** and they can be used as (often anonymous) functions created "on the fly", like lambdas in Java or C++. Note, however, that they are not so useful as they are in Java or, even more so, in C++, because the body of a lambda-function is limited to a single expression the value of which is returned.

Any lambda is basically equivalent to a function, as **sq** and **lam** below

```
>>> def sq(x):
...     return x*x
...
>>> lam = lambda x: x*x
>>> type(sq), type(lam)
(<class 'function'>, <class 'function'>)
>>>
>>> dir(sq)
['__annotations__', '__builtins__', '__call__', '__class__',
 '__closure__', '__code__', '__defaults__', '__delattr__',
 '__dict__', '__dir__', '__doc__', '__eq__', '__format__',
 '__ge__', '__get__', '__getattribute__', '__globals__',
 '__gt__', '__hash__', '__init__', '__init_subclass__',
 '__kwdefaults__', '__le__', '__lt__', '__module__',
 '__name__', '__ne__', '__new__', '__qualname__',
 '__reduce__', '__reduce_ex__', '__repr__', '__setattr__',
 '__sizeof__', '__str__', '__subclasshook__']
>>>
>>> dir(lam)
['__annotations__', '__builtins__', '__call__', '__class__',
 '__closure__', '__code__', '__defaults__', '__delattr__',
 '__dict__', '__dir__', '__doc__', '__eq__', '__format__',
 '__ge__', '__get__', '__getattribute__', '__globals__',
 '__gt__', '__hash__', '__init__', '__init_subclass__',
 '__kwdefaults__', '__le__', '__lt__', '__module__',
 '__name__', '__ne__', '__new__', '__qualname__',
 '__reduce__', '__reduce_ex__', '__repr__', '__setattr__',
 '__sizeof__', '__str__', '__subclasshook__']
>>>
>>> sq.__name__, lam.__name__
('sq', '<lambda>')
```

As we see, these two objects are essentially the same, except for their `__name__` attribute. However, lambdas can be used in contexts where one couldn't use a definition of a function — for example directly on the argument list of a function invocation, as in the example below:

```
>>> def apply_n_times(f, n, x):
...     for i in range(n):
...         x = f(x)
...     return x
...
>>> apply_n_times(lambda x: x*x, 3, 2)
256
```

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

Of course, number of parameters in lambdas is not limited to one; in the example below lambda has two parameters and returns a two-element tuple with the sum of squares of the arguments as the first element and the square of their sum as the second one:

```
>>> myfun = lambda x, y: (x**2+y**2, (x+y)**2)
>>> myfun(4, -1)
(17, 9)
```

Lambdas, being just objects, can be passed or returned to/from functions. In the example below, the **quad** function accepts three coefficients of a quadratic function $ax^2 + bx + c$ which is then returned as a lambda:

```
def quad(a, b, c):
    print('id of c is:', id(c))
    return lambda x: a*x*x + b*x + c

f, g, h = quad(1, 1, 1), quad(2, -1, 3), quad(-1, 4, -2)

print('\n', f(2), g(2), h(2))
print()
print('h.__closure__ is:', h.__closure__, '\n')
print('id of h.__closure__[2].cell_contents is:',
        id(h.__closure__[2].cell_contents))
print('h.__closure__[2].cell_contents is:',
    h.__closure__[2].cell_contents)
```

The program prints

```
id of c is: 140437638578416
id of c is: 140437638578480
id of c is: 140437638578320

 7 9 2

h.__closure__ is:
(<cell at 0x7fba2e1fbcd0: int object at 0x7fba2f8f80b0>,
 <cell at 0x7fba2e1fbca0: int object at 0x7fba2f8f8150>,
 <cell at 0x7fba2e1fbc70: int object at 0x7fba2f8f8090>)

id of h.__closure__[2].cell_contents is: 140437638578320
h.__closure__[2].cell_contents is: -2
```

The **quad** prints also the ID of variable `c` which is *local* inside this function, so it should cease to exist when the function exits.

However, we then investigate the object representing the **h** lambda. Its attribute `__closure__` is a tuple containing three "cells": they correspond to the three local variables, `a`, `b` and `c`, as they were when the **quad** was executed. Printing the `cell_content` of the third (with index 2) cell, we can see that its ID and value are exactly the ID and value of `c` when it was used as local variable during the execution of the function. Therefore, local variables needed in the definition of a function (lambda

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

in particular) are "remembered" in the objects representing the function, even when they seemingly ceased to exist after returning.

A closure is created when a function (lambda) is defined inside a function and "remembers" references to current values of local variables. When it's not in a function, no closure is created.

Let's look at the following snippet of code. Note that we create a lambda which uses a, b and c although they don't exist yet! The lambda contains only their *names:* their values will always correspond to *current* values when the lambda is invoked, so their modifications between calls will be visible, even when they refer to completely new objects of different type:

```
1    quad = lambda x: a*x**2 + b*x + c # a, b, c do not exist!
2    a = 1; b = 1; c = 1               # ints
3    print(quad(-2))                   # prints 3
4    a = 2.5; b = 2.5; c = -0.75       # now floats
5    print(quad(-2))                   # prints 4.25
6    print(quad.__closure__)           # prints None
```

Here, in line 4, we redefine a, b and c, so they refer to different objects of different type (now they are **float**s) and when the lambda **quad** is executed, *new* values are used. No closures are created.

However, if the same code is used in a function, the closure *will* be created

```
1    def d():
2        quad = lambda x: a*x**2 + b*x + c # a, b, c do not exist!
3        a = 1; b = 1; c = 1               # ints
4        print(quad(-2))                   # prints 3
5        a = 2.5; b = 2.5; c = -0.75       # now floats
6        print(quad(-2))                   # prints 4.25
7        print(quad.__closure__)           # now closure exists!
8    d()
```

as seen from the output of this program:

```
3
4.25
(<cell at 0x7f5ef349b820: float object at 0x7f5ef3deddb0>,
 <cell at 0x7f5ef349b850: float object at 0x7f5ef3deddb0>,
 <cell at 0x7f5ef349b880: float object at 0x7f5ef3ded3f0>)
```

Lambdas, being functions, can be passed to functions and returned from functions — these functions may be lambdas themselves. Let us consider the following example:

```
1    cmp = lambda f, g: lambda x: f(g(x)) # composition
2
3    fun1 = cmp(lambda x: x-1,  lambda x: x**2)
4    fun2 = cmp(lambda x: x**2, lambda x: x-1)
5
6    print(fun1(3), fun2(3))              # prints 8 4
```

In line 1, we create a lambda which has two parameters, f and g. They both are supposed to be one-argument functions. The lambda's value is also a lambda — in

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

our case corresponding to the composition $f \circ g$ (recall that the function composition is defined as $(f \circ g)(x) = f(g(x))$). The lambda created is given the name **cmp**.
In lines 3 and 4, we invoke **cmp** passing two functions, as required, and these functions are also defined as lambdas: one corresponds to function $x \mapsto x - 1$ and the second to $x \mapsto x^2$ (we pass them to **fun1** and to fun2 in different order). Now **fun1** represents the function $x \mapsto x^2 - 1$ and fun2 $x \mapsto (x - 1)^2$, so the result printed is '8 4'.

Lambdas are often used when calling a function which expects a function as one of its arguments. For example, the **sort** method of lists needs to know according to what the elements are to be sorted: this information is passed as a function, which applied to elements of the list yields values that are to be used for comparing these elements. For example,

sort

```
>>> ls = [('Joe', 182, 79), ('Sue', 173, 59), ('Liz', 175, 62)]
>>> ls.sort(key=lambda person: person[2])
>>> ls
[('Sue', 173, 59), ('Liz', 175, 62), ('Joe', 182, 79)]
```

sorts list of 3-tuples, representing names, heights and weights of persons, according to the third element (index 2: weight).
Similarly, in the example below, we find the most frequently occurring item on the list using again the key argument of the **max** function:

max

```
>>> lst = [3, 5, 2, 3, 5, 3, 1, 3, 6]
>>> mx = max(set(lst), key=lst.count)
>>> mx
3
```

Although the body of a lambda must be a single expression, it can, in particular, contain invocation of itself — lambdas can be recursive, although for this to be possible, they have to have a name. Recursive lambdas are not very common because they tend to be hard to understand, but they are possible. For example, the factorial function can be defined as:

```
>>> fac = lambda n: 1 if n <= 1 else n*fac(n-1)
>>> fac(5)
120
>>> fac(45)
119622220865480194561963161495657715064383733760000000000
```

On the right-hand side of the assignment on the first line, the name fac is undefined. However, it *will* be known after the assignment, when the lambda is *used.*

5.4 Some "special" functions.

In particular, lambdas are used with **filter**, **map**, **reduce**, **accumulate**[1] and also **sort**/**sorted** functions.
They are used as functions, although, strictly speaking, some of them, as **map** and **filter**, are actually classes (but classes *are* callable, so this isn't so strange):

map
filter

[1] These are basic functions in functional languages, like Haskell.

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

```
>>> lst = [1,2,3,4,5,6]
>>>
>>> f = filter(lambda n: n%2 != 0, lst)
>>> type(f)
<class 'filter'>
>>> f
<filter object at 0x7fd978400e20>
>>> list(f)
[1, 3, 5]
>>>
>>> m = map(lambda n: n**n, lst)
>>> type(m)
<class 'map'>
>>> m
<map object at 0x7fd978403f10>
>>> list(m)
[1, 4, 27, 256, 3125, 46656]
```

5.4.1 *filter*

filter

The **filter** function takes an iterable (something that can be iterated over) and a function (a predicate) which for any element of the iterable yields **True** (or anything "truthy") or **False** (or anything "falsy"). It returns an iterable yielding those elements form the input iterable, for which the predicate gives **True**. If the predicate is **None**, it defaults to identity function, which will then leave only "truthy" elements while removing the "falsy" ones.

For example

Listing 23 AAZ-Filter/Filter.py

```
#!/usr/bin/env python

def is_prime(n):
    if n <= 1: raise ValueError
    if n <= 3: return True
    if n % 2 == 0 or n % 3 == 0: return False
    k = 5
    while k**2 <= n:
        if n % k == 0 or n % (k+2) == 0: return False
        k += 6
    return True

lst = [1, 5, 1, 0, '', {}, 2]

print(type(filter))           # <class 'type'>
a = filter(None, lst)
print(a)                      # <filter object at 0x7f569fabbc40>

for e in filter(None, lst):
    print(e, end=' ')         # only 'truthy': 1 5 1 2
```

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

```
21 print()
22
23 lst = ["Kyle", "Sue", "Mary", "Ada"]
24 for e in filter(lambda s: len(s) > 3, lst):
25     print(e, end=' ')          # Kyle Mary
26 print()
27
28 tup = (3, 9, 12, 11, 27, 83, 787)
29 for e in filter(lambda n: is_prime(n), tup):
30     print(e, end=' ')          # 3 11 83 787
31 print()
32
33 dic = dict(Alice = 15, Jane = 21, Kyle = 12, Mary = 28)
34 d = dict(filter(lambda item: item[1] >= 18, dic.items()))
35 print(d)                      # {'Jane': 21, 'Mary': 28}
```

Note that in the last example, we passed the resulting iterable directly to the constructor of a dictionary.

There is also (in the *itertools* module) the function **filterfalse** function which behaves like **filter**, but selects elements *not* meeting the predicate: filterfalse

```
>>> from itertools import filterfalse
>>> nums = [2, 1, 3, 4, 5, 6]
>>> for n in filterfalse(lambda x: x%2 != 0, nums): # filterfalse
...     print(n, end=' ')
... print()
...
2 4 6
>>> for n in filter(lambda x: x%2 != 0, nums):      # filter
...     print(n, end=' ')
... print()
...
1 3 5
```

5.4.2 *map*

The **map** function takes a function and one or more input iterable objects and creates an iterable of values obtained by applying the given function to values of input iterable(s). The function must be of the same arity (number of parameters) as is the number of iterables provided. If there are more than one input iterables, the resulting iterable will generate as many tuples of values as there are elements in the shortest of the input iterables. map

For example

```
lst = [1, 4, 7, 2, 6]
m = map(lambda x: (x-1)**2, lst)

print(map)          # <class 'map'>
print(m)            # <map object at 0x7f4c7f403a60>
print(list(m))      # [0, 9, 36, 1, 25]
```

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

Another example, where we apply formatting (see sec. ??, p. ??) to elements of a list

```
lsf = [1.2345, 1.2223e2]
print(list(map(lambda x: f'{x:.1f}', lsf))) # ['1.2', '122.2']
```

or, with more input iterables

```
lsti = [ 1,    3,    2,      2,   2]
lsts = ['a', 'bc', 'def', 'ghij']

m = map(lambda x, y: x*y, lsti, lsts)
print(map)          # <class 'map'>
print(m)            # <map object at 0x7ffaf3583a90>
print(list(m))      # ['a', 'bcbcbc', 'defdef', 'ghijghij']
```

(we get four elements, as the shorter of the two lists has 4 elements).

5.4.3 *reduce*

reduce

The **reduce** function (from the ***functools*** module) takes a *binary* function, an iterable and, optionally, an initializer (seed).

If the initializer is given, it becomes the initial value of the so called **accumulator**. Then the function is applied iteratively on the accumulator and consecutive elements of the iterable assigning the result back to the accumulator after each application, thus reducing the iterable to a single value:

```
iterable = [1, 2, 3, 4]
init = 0
fun = lambda x, y: x + y

acc = init
for e in iterable:
    acc = fun(acc, e)
print(acc)                                      # 10

  # now with reduce
from functools import reduce
print(reduce(lambda x, y: x+y, iterable, 0))    # 10
```

With two arguments, without initializer, acc will be initialized with the value of the first element and the first application of the function will be on two first elements:

```
iterable = [1, 2, 3, 4]
init = 0
fun = lambda x, y: x + y

it = iter(iterable)
acc = next(it)                                  # first element
for e in it:                                    # it, not iterable
    acc = fun(acc, e)
print(acc)                                      # 10

  # now with reduce
from functools import reduce
print(reduce(lambda x, y: x+y, iterable))       # 10
```

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

In the two examples above, the results are the same, because in the first of them, we took the the neutral element of addition as the seed; with another value, the results will be, of course, different:

```
from functools import reduce

iterable = [1, 2, 3, 4]

print(reduce(lambda x, y: x+y, iterable))    # 10
print(reduce(lambda x, y: x+y, iterable, 5)) # 15
```

Other examples:

```
from functools import reduce

lsi = [3, 6, -1, 3, 7, 2]

mn = reduce(lambda x, y: min(x, y), lsi) # min
mx = reduce(lambda x, y: max(x, y), lsi) # max
nb = reduce(lambda x, y: x+1, lsi, 0)    # number of elems

print(mn, mx, nb)           # -1 7 6


lss = ['Kate', 'Joanna', 'Sue', 'Mary']

  # longest word
ln = reduce(lambda x, s: x if len(x) > len(s) else s, lss)
  # initials
ii = reduce(lambda i, s: i + s[0], lss, '')
  # length of the longest word
lo = reduce(lambda cnt, s: cnt if cnt > len(s) else len(s), lss, 0)

print(ln, ii, lo)           # Joanna KJSM 6
```

5.4.4 *accumulate*

The **accumulate** function (from the ***itertools*** module) is similar to **reduce**, but instead of returning a single value, it returns an iterable yielding all "intermediate" results. The first argument of **accumulate** is an iterable, and the second is a binary function. By default it corresponds to just addition (**operator.add**), so these two invocations

accumulate

```
from itertools import accumulate
lsi = [1, 2, 3]

print(list(accumulate(lsi)))                    # [1, 3, 6]
print(list(accumulate(lsi, lambda x, y: x + y))) # [1, 3, 6]
```

are equivalent.

In the example below, the numbers are payments on a saving account; every month the interest 5% of the accumulated capital is added first and then the amount of the

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

current payment: the returned iterable yields the account balance after each payment (we used **map** to format the numbers):

```
from itertools import accumulate

lsi = [150, 200, 300, 350]

a = accumulate(lsi, lambda summ, e: 1.05*summ + e)
print(type(a))
print(list(map(lambda e: round(e,1), a)))
```

The program prints

```
<class 'itertools.accumulate'>
[150, 357.5, 675.4, 1059.1]
```

One can specify also an initial value by passing the argument initial (this is keyword-only parameter) — then the number of resulting elements will be by one greater than of those in the input iterable:

```
from itertools import accumulate

lsi = [150, 200, 300, 350]

a = accumulate(lsi, lambda summ, e: 1.05*summ + e, initial=100)
print(list(map(lambda e: round(e,1), a)))
```

Now the program prints

```
[100, 255.0, 467.8, 791.1, 1180.7]
```

5.4.5 *sorted*

sorted The **sorted** function takes an iterable and returns its (shallow) copy which is sorted (by default, in ascending order). Note that **sorted** always returns a list, even if if an iterable passed as the first argument is a tuple or a set.

sort There is also a *method* **sort**, of class **list** which when invoked on a list, sorts it. Therefore, the list is rearranged, not copied, and nothing is returned (i.e., **None** is returned).

Important: Algorithm used for sorting is not fixed once for all[1] and can be changed in the future, but it *is* guaranteed to be stable, i.e., elements that are equal, retain their relative order after sorting.

By default, **sorted** uses the $<$ operator to compare elements, so it will work for numbers (but not **complex**) and strings, as well as other types for which this operator is defined (for example, for which **__lt__** method is defined):

```
lsiorig = [7, 2, 8, 1, 5]
lsorted = sorted(lsiorig)
print('original', lsiorig) # [7, 2, 8, 1, 5]
print('  sorted', lsorted) # [1, 2, 5, 7, 8]
```

[1] For many years it was Timsort – a hybrid algorithm based on merge sort and insertion sort (invented by Tim Peters in 2002). This algorithm is also used in Java and in Android system. In Python 3.11, it was replaced by the new algorithm – Powersort (I. Munro i S. Wild).

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

It follows, that we cannot "mix" types, because the < will not work for them

```
>>> sorted([2, 'b', 1, 'a'])
  Traceback (most recent call last):
    File "<python-input-9>", line 1, in <module>
      sorted([2, 'b', 1, 'a'])
      ~~~~~~^^^^^^^^^^^^^^^^^^
TypeError: '<' not supported between instances of 'str' and 'int'
```

Strings are also comparable (in lexicographical order), so they can be sorted as well

```
>>> st = {'Jane', 'Zoe', 'Mary', 'Alice'}  # set
>>> sorted(st)
['Alice', 'Jane', 'Mary', 'Zoe']
```

But strings are themselves iterable, so we can sort characters of a single string

```
>>> sorted('Mississippi')
['M', 'i', 'i', 'i', 'i', 'p', 'p', 's', 's', 's', 's']
```

(remember that all uppercase letters are "smaller" than any lowercase letter).

Besides a sequence to be sorted, there are two additional parameters of **sorted**, both are optional and keyword-only.
One of them is reverse, with obvious meaning and default value **False**:

```
lsiorig = [7, 2, 8, 1, 5]
lsorted = sorted(lsiorig, reverse=True)
print('original', lsiorig) # [7, 2, 8, 1, 5]
print('  sorted', lsorted) # [8, 7, 5, 2, 1]
```

The third (also keyword-only) parameter of **sorted** is called key. The corresponding argument should be a unary function[1] which will be applied to elements and then the results will be compared instead of original values. Note however that these results will be used only for comparing, elements of the input sequence themselves will not be modified, only rearranged.
In the example below , we sort tuples in two ways: by their first element (name) and then by the second (age):

```
ltup = [('Jane', 29), ('Alice', 31), ('Betty', 7)]
print(sorted(ltup, key=lambda t: t[0]))
print(sorted(ltup, key=lambda t: t[1]))
```

prints

```
[('Alice', 31), ('Betty', 7), ('Jane', 29)]
[('Betty', 7), ('Jane', 29), ('Alice', 31)]
```

In the following example, we first sort without specifying any key function, so the < will be used for strings (recall that all capital letters are "smaller" than all lowercase letters). Then we sort, but this time passing the **lower** method as the key function: elements of the list will not be modified, but comparisons will now be case-insensitive. Finally, we print the original list just to show that it remained unmodified — **sorted** returns the sorted *copy:*

lower

[1] Sometimes, in particular in C++, called *projector*.

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

```
lstring = ['Jane', 'Hugh', 'bertha', 'henry', 'alice']
lsorted = sorted(lstring)
ltolowe = sorted(lstring, key = lambda s: s.lower())
print('   original', lstring)
print(' sort as is', lsorted)
print('case insens', ltolowe)
print('   original', lstring)
```

prints

```
   original ['Jane', 'Hugh', 'bertha', 'henry', 'alice']
 sort as is ['Hugh', 'Jane', 'alice', 'bertha', 'henry']
case insens ['alice', 'bertha', 'henry', 'Hugh', 'Jane']
   original ['Jane', 'Hugh', 'bertha', 'henry', 'alice']
```

Very often criteria for comparison are more complex; for example we want to compare using one, "primary", criterion, but then for elements which come out to be equal, use other, "secondary", criteria in addition. To solve this problem, we can leverage the fact, that tuples *are* comparable (using $<$ operator) if only their consecutive elements are. Therefore, the **key** function can return tuples, which will be then compared lexicographically.

For example here, we sort strings by their lengths. However, those with equal lengths will be additionally sorted alphabetically and case-insensitively:

```
ls = ['jane', 'Bertha', 'henry', 'alice', 'Hugh']
print(sorted(ls, key=lambda s: (len(s), s.lower())))
```

prints

```
['Hugh', 'jane', 'alice', 'henry', 'Bertha']
```

Alternatively, thanks to the guaranteed stability of sorting algorithm, we can first sort by the *secondary* criteria and then sort the result using the *primary* ones.

For example, in the snippet below, we sort strings by their lengths. However, those with equal lengths will be additionally sorted alphabetically and case-insensitively:

```
ls = ['jane', 'Bertha', 'henry', 'alice', 'Hugh']
    # secondary: alphabetically, case-insensitively
ls = sorted(ls, key=lambda s: s.lower())
    # and now primary: length
ls = sorted(ls, key=lambda s: len(s))
print(ls)
```

prints as before

```
['Hugh', 'jane', 'alice', 'henry', 'Bertha']
```

In many languages, sorting is based on binary comparing functions which, given two objects, return anything negative if the first is to be considered "smaller", anything positive if it's the second one, and 0 if they are deemed equal. For those used to this kind of comparisons, there is a special function, **cmp_to_key** from the ***functools*** module, which takes a binary comparing function, as described above, and returns a callable that given *one* argument yields a value which can be used as the sort key:

cmp_to_key

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

```
from functools import cmp_to_key

def my_cmp(s1, s2):
      # by length
    if (d := len(s1) - len(s2)) != 0: return d
      # alphabetically, case-insensitively
    if   s1.lower() < s2.lower(): return -1
    elif s2.lower() < s1.lower(): return +1
    else                        : return  0

ls = ['jane', 'Bertha', 'henry', 'alice', 'Hugh']

ls = sorted(ls, key=cmp_to_key(my_cmp))
print(ls)
```

prints again

```
['Hugh', 'jane', 'alice', 'henry', 'Bertha']
```

5.5 Function decorators

A decorator is a function (more generally – a callable) that takes another callable[1] as its argument and returns also a callable.

For example:

```
def decorator(func):
    return func

def adder(a, b):
    """Return sum of arguments."""
    return a + b

add = decorator(adder)
print(add(5, 7))        # 12
print(add.__name__)     # adder
```

In this trivial example, **add** behaves just as **adder**, although its `__name__` property is `adder`, as the original name of the function wrapped by the **decorator**. However, we can make it less trivial by returning a function which is like the original, but with some features added or modified, and this is so useful that it gained a special syntax which makes it easier and more readable: instead of invoking the decorator function explicitly, we *annotate* our function with the name of the decorator:

```
def decorator(func):
    return func

@decorator
def adder(a, b):
    """Return sum of arguments."""
    return a + b
```

[1] Classes are callable and objects of classes can be made callable too. Therefore, they also can be used to decorate functions. In the examples below, we will use just functions, though.

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

```
print(adder(5, 7))  # 12
```

One can say that

```
@decorator
def adder(a, b):
    """Return sum of arguments."""
    return a + b
```

is equivalent to

```
def adder(a, b):
    """Return sum of arguments."""
    return a + b
adder = decorator(adder)
```

Normally, of course, the returned function is not just identical to the input function but has some features added or modified. These could be, among others, registering a function somewhere, modifying its input or output, logging or counting its invocations, caching the results of its invocations, etc.

Consider

```
def counting(func):                          # decorator
    counter = 0
    def modified_func(*args, **kwargs):      # 'universal'
        nonlocal counter
        counter += 1
        return func(*args, **kwargs)
    return modified_func

@counting
def adder(a, b):
    """Return sum of arguments."""
    return a + b

print(adder(-1, 2), adder(5, 7))             # 1 12
print(adder.__closure__[0].cell_contents)    # 2
print(adder.__name__)                        # modified_func
print(adder.__doc__)                         # None
```

Now the decorator **counting** returns the original function but with a counter of invocations added to its closure (see p. 81). However, there are some problems: its `__name__` property is now `modified_func` instead of `adder` and the docstring has been lost. Moreover, adding the counter to the closure, even though works, makes accessing it a bit convoluted. We will solve this particular problem by making the counter an *attribute* of the returned function. As to the docstring and the name, we can fix it "by hand":

```
def counting(func):                  # decorator
    def modified_func(*args, **kwargs):
        modified_func.counter += 1
        return func(*args, **kwargs)
```

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

```
        modified_func.counter = 0
        modified_func.__name__ = func.__name__
        modified_func.__doc__  = func.__doc__
        return modified_func

    @counting
    def adder(a, b):
        """Return sum of arguments."""
        return a + b

    print(adder(-1, 2), adder(5, 7)) # 1 12
    print(adder.counter)             # 2
    print(adder.__name__)            # adder
    print(adder.__doc__)             # Return sum of arguments.
```

However, this would be cumbersome and we wouldn't be sure if all function's metadata (like __name__, __doc__ or __module__) has been correctly copied. The solution is to use special function **wraps** from the ***functools*** module (which is a decorator itself!) which takes care about it:

wraps

```
    import functools

    def counting(func):              # decorator
        @functools.wraps(func)       # wrapping
        def modified_func(*args, **kwargs):
            modified_func.counter += 1
            return func(*args, **kwargs)
        modified_func.counter = 0
        return modified_func

    @counting
    def adder(a, b):
        """Return sum of arguments."""
        return a + b

    print(adder(-1, 2), adder(5, 7)) # 1 12
    print(adder.counter)             # 2
    print(adder.__name__)            # adder
    print(adder.__doc__)             # Return sum of arguments.
```

Moreover, the decorator **wraps** adds a new attribute, __wrapped__, to the decorated function — this is the reference to the original, undecorated function. Therefore, we can decorate a function and then return to its original version, as in the example below:

Listing 24 ACP-UnDeco/undeco.py

```
1 #!/usr/bin/env python
2 import functools
3
4 def deco(func):                          ❶
5     @functools.wraps(func)
```

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

```
6       def _f(*args, **kwargs):
7           print(f'Decorated {func.__name__} '
8                 f'called with args={args}')      ❷
9           return func(*args, **kwargs)
10      return _f
11
12  @deco                                          ❸
13  def adder(a, b):
14      return a + b
15
16  print('Calling decorated adder')
17  print(adder(3, 5))
18  print('Has __wrapper__?', hasattr(adder, '__wrapped__'))
19  print('Name =', adder.__name__)
20
21  adder = adder.__wrapped__                      ❹
22
23  print('*'*40, '\nCalling original adder')
24  print(adder(3, 5))
25  print('Has __wrapper__?', hasattr(adder, '__wrapped__'))
26  print('Name =', adder.__name__)
```

We create a decorator (❶) which returns a function returning what the original function would, but before that prints a message (❷). It is then used to decorate a simple function **adder** (❸). Then we print some information, in particular we check that the decorated function indeed has the `__wrapped__` attribute. In line ❹, we copy the `__wrapped__` attribute back to **adder**: the printout

```
Calling decorated adder
Decorated adder called with args=(3, 5)
8
Has __wrapper__? True
Name = adder
****************************************
Calling original adder
8
Has __wrapper__? False
Name = adder
```

shows that now **adder** refers to the original version.

Adding data to attributes of a function makes it possible to implement the so called *memoization,* , i.e., ability of remembering information from previous invocations of a function.[1]

Let us consider the Fibonacci numbers; as we know, calculating them recursively is extremely ineffective, as the number of invocations grows as 2^N for the N-th Fibonacci number. But it becomes trivial and practically immediate, if we can remember all previous results for smaller numbers.

[1] As a matter of fact, such decorator is already implemented in the standard library as `lru_cache` (**l**east **r**ecently **u**sed cache) from the ***functools*** module — see documentation.

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

In the example below, the decorator **memo** adds a dictionary cache as attribute of the decorated function. The cache remembers results with tuples of arguments as keys, so it can immediately return the answer if the result with given arguments has already been calculated before. We can access this cache directly as cache attribute of the decorated function:

Listing 25 ABS-Deco/Deco.py

```
#!/usr/bin/env python

import functools

def memo(func):
    @functools.wraps(func)
    def memo(*args):
        if args not in memo.cache:
            memo.cache[args] = func(*args)
        return memo.cache[args]
    memo.cache = {}  # cache will be an attribute
    return memo

@memo
def fibo(n):
    if n < 2: return n
    else    : return fibo(n - 1) + fibo(n - 2)

for i in range(50, 201, 50):
    print(f'fibonacci[{i:3}] = {fibo(i)}')

print('Cache:\n', fibo.cache)
```

The program prints

```
fibonacci[ 50] = 12586269025
fibonacci[100] = 354224848179261915075
fibonacci[150] = 9969216677189303386214405760200
fibonacci[200] = 280571172992510140037611932413038677189525
Cache:
{(1,): 1, (0,): 0, (2,): 1, (3,): 2, (4,): 3, (5,): 5, (6,): 8,
 ...
 ...
 ...
 (197,): 66233869353085486281758142155705206899077,
 (198,): 107168651819712326877926895128666735145224,
 (199,): 173402521172797813159685037284371942044301,
 (200,): 280571172992510140037611932413038677189525}
```

Note that the **memo** decorator can be applied to very wide spectrum of functions, not necessarily to binary and/or recursive.[1]

[1] Don't even try to run the above program without caching!

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

Decorators can be nested: the outer decorator is then applied to the result of the inner decorator, as in the example below:

Listing 26 ABT-DecoNest/DecoNest.py

```
#!/usr/bin/env python

import functools

def italic(func):
    @functools.wraps(func)
    def _f(s):
        return '<i>' + func(s) + '</i>'
    return _f

def strong(func):
    @functools.wraps(func)
    def _f(s):
        return '<strong>' + func(s) + '</strong>'
    return _f

@italic
def ital(s):
    return s

@strong
def stro(s):
    return s

@italic
@strong
def itst(s):
    return s

@strong
@italic
def stit(s):
    return s

print(ital(" just ital "))
print(stro(" just strong "))
print(itst(" ital and strong "))
print(stit(" strong and ital "))
```

which prints

```
<i> just ital </i>
<strong> just strong </strong>
<i><strong> ital and strong </strong></i>
<strong><i> strong and ital </i></strong>
```

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

Very often it would be convenient to pass additional information to a decorator, so its functionality depends somehow on this information. This is possible by constructing a "decorator factory" — a function which depends on some arguments and returns an actual decorator with which we decorate a function. W can describe it by saying that

```
@decofactory(argA, argB, ...)
def func(arg1, arg2, ...):
    # ...
```

is equivalent to

```
func = decofactory(argA, argB, ...)(func)
```

In the example below, **rangecheck** is *not* a decorator: it *returns* a decorator. The **rangecheck** function accepts two arguments, mn and mx, which define the range [mn, mx] to which all arguments of a function are supposed to belong (if they don't, an exception is raised — see chap. ??, p. ??). For given mn and mx, the **rangecheck** function creates and returns the **checker** function: this is the decorator proper, which, being a decorator, takes a function and returns a function (here called **wrap**) that will "replace" a decorated function (**color** on line 18, and **fRGBColor** on line 22). What is important, references mn and mx can be used inside **wrap** and hence the resulting decorated function will depend on them:

Listing 27 ABU-DecoParam/DecoParam.py

```
1  #!/usr/bin/env python
2
3  import functools
4
5  def rangecheck(mn, mx):
6      def checker(func):                # decorator to be returned
7          @functools.wraps(func)
8          def wrap(*args):              # wraper of func
9              for i in args:
10                 if i < mn or i > mx:
11                     raise ValueError(
12                         f'Argument {i} in {args} is out of range')
13             return func(*args)        # if args ok, just call func
14         return wrap
15     return checker
16
17 @rangecheck(0, 255)
18 def color(r, g, b):
19     return f'Color {r=}, {g=}, {b=}'
20
21 @rangecheck(0,1)
22 def fRGBColor(r, g, b):
23     return f'fColor {r=:.1f}, {g=:.1f}, {b=:.1f}'
24
25 try:
26     print(color(  2, 255,  0))
```

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

```
27      print(color(  0, 128, 34))
28      print(color(103,  12, 88))
29      print(color(  0, 256,  0))
30  except ValueError as e:
31      print(e)
32
33  print()
34
35  try:
36      print(fRGBColor(0.0, 0.9, 0.0))
37      print(fRGBColor(0.5, 0.8, 0.0))
38      print(fRGBColor(0.2, 0.1, 0.8))
39      print(fRGBColor(1.1, 0.9, 0.3))
40  except ValueError as e:
41      print(e)
```

The program prints

```
Color r=2, g=255, b=0
Color r=0, g=128, b=34
Color r=103, g=12, b=88
Argument 256 in (0, 256, 0) is out of range

fColor r=0.0, g=0.9, b=0.0
fColor r=0.5, g=0.8, b=0.0
fColor r=0.2, g=0.1, b=0.8
Argument 1.1 in (1.1, 0.9, 0.3) is out of range
```

Decorators are executed when the module is loaded (imported), so they are "ready to use" inside the module. The example below illustrates this feature:

Listing 28 ABY-DecoTime/DecoTime.py

```
1  #!/usr/bin/env python
2
3  import functools
4
5  def decorator(func):
6      print('Decorating', func.__name__)
7      @functools.wraps(func)
8      def wrap(arg):
9          val = func(arg)
10         print('Decorated', func.__name__, ':', arg, '->', val)
11         return val
12     return wrap
13
14 @decorator
15 def funA(x):
16     return x
17
```

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

```
@decorator
def funB(x):
    return x*x

print('\n*** Starting the program\n')

print('Calling funA with arg =', 4)
v = funA(4)

print('Calling funB with arg =', 5)
v = funB(5)
```

As we can see from the printout

```
Decorating funA
Decorating funB

*** Starting the program

Calling funA with arg = 4
Decorated funA : 4 -> 4
Calling funB with arg = 5
Decorated funB : 5 -> 25
```

the lines

```
Decorating funA
Decorating funB
```

appeared *before* entering the program proper.

5.6 Function "overloading": (single dispatch)

Generally, functions (and methods of classes as well) cannot be overloaded in Python. In one module (or class), we cannot define two or more functions (methods) with the same name, as names serve as keys in dictionaries representing objects (including functions, classes and modules).

However, it's possible to create a function with its several versions kept as values of a dictionary with types of the first parameter as keys. The dictionary becomes an attribute of the object representing the function. Then, depending on the type of the first argument, the correct version from the dictionary is selected — this concept is called **single dispatch**; *single,* because the choice depends on the type of one argument only.

We don't have to implement single dispatch ourselves, because the standard module ***functools*** already contains a decorator **singledispatch** will do it for us. singledispatch

Let's look at an example:

Listing 29 ACN-SingDis/singleDispatch.py

```
#!/usr/bin/env python

```

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

```
3  import functools
4
5  class Person:                                          ❶
6      def __init__(self, name):
7          self.name = name
8      def __repr__(self):
9          return f'Person({self.name})'
10
11 class Student(Person):
12     def __init__(self, name, snumber):
13         super().__init__(name)
14         self.snumber = snumber
15     def __repr__(self):
16         return f'Student({self.name}, {self.snumber})'
17
18 @functools.singledispatch                             ❷
19 def aFunc(arg, *args, **kwargs):
20     raise NotImplementedError(
21          f'{arg} of type {type(arg).__name__} '
22           'not handled by aFunc')
23
24 @aFunc.register(int)                                   ❸
25 def _whatever(v):
26     print('aFunc for int called:', v)
27
28 @aFunc.register(float)                                 ❹
29 def _whatever(v1, v2):
30     print('aFunc for float called:', v1, v2)
31
32 @aFunc.register(complex)                               ❺
33 def _anyname(v, *args, **kwargs):
34     print('aFunc for complex called:', v, kwargs)
35
36 @aFunc.register(Person)                                ❻
37 def _(arg):
38     print('aFunc for Person called:', arg)
39
40 print(type(aFunc))                                     ❼
41 print(vars(aFunc))
42                                                        ❽
43 aFunc(5)                          # int
44 aFunc(5.5, 'Hello')               # float
45 aFunc(2+1j, a=1, b=2)             # complex
46 aFunc(True)                       # bool -> int
47 aFunc(Student('Sue', 's12345'))   # Student -> Person
48 aFunc('BOOM')                     # string -> default
```

Here,

- We define a function **aFunc** and decorate it with the **singledispatch** decorator

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

(❷). The implementation just raises an exception, but this is not required, we could have implemented it in any other way. This implementation will play the rôle of the default: it will be used if the type of the first argument will *not* be one of those registered (see below). The decorator returns our function, but decorated with additional attributes, in particular

  - method **register**,[1] which will register types of the first argument and assign different versions of the function to them; register
  - attribute registry, which is a dictionary with types as keys and functions as values.

- Now in line ❸, we register the version ("overload") of our function for invocations with an **int** as the first argument. Then we do the same with other types: **float** (❹), **complex** (❺), and our own type **Person** (❻). Note that names of these overloads are irrelevant. They can be the same (as **_whatever** for both **int**s and **float**s), or different, or even just **_**. What's important, it cannot be the name of our "original" function (**aFunc** in our case). Also note that the arity (number of parameters) of "overloads" doesn't have to be the same: the function is unary for types **int** and **Person**, binary for **float**, and *vararg* (any number of arguments) for **complex** and for the the default implementation.
- In line ❼, we print the type of **aFunc** and, in the next line, attributes of the object representing it: in the output, we will see the **register** function and registry dictionary mentioned above.
- Below the line ❽, we invoke our function **aFunc** with various types of the first argument.

The program prints (for clarity, the output is slightly shortened and reformatted)

```
<class 'function'>
{'register': <function singledispatch.<locals>.register at ...,
 'dispatch': <function singledispatch.<locals>.dispatch at ...,
 'registry': mappingproxy(
    {<class 'object'>: <function aFunc at 0x7f25fcd194e0>,
     <class 'int'>: <function _whatever at 0x7f25fcb96520>,
     <class 'float'>: <function _whatever at 0x7f25fcb96480>,
     <class 'complex'>: <function _anyname at 0x7f25fcb96340>,
     <class '__main__.Person'>: <function _ at 0x7f25fca3eb60>}),
     ...
 '__wrapped__': <function aFunc at 0x7f25fcd194e0>}

aFunc for int called: 5
aFunc for float called: 5.5 Hello
aFunc for complex called: (2+1j) {'a': 1, 'b': 2}
aFunc for int called: True
aFunc for Person called: Student(Sue, s12345)

Traceback (most recent call last):
  File "/home/werner/python/examples/singleDispatch.py", ...
    aFunc('BOOM')                    # string -> default
    ~~~~~~~~~~~~
  File "/usr/lib/python3.13/functools.py", line 929, ...
```

[1] Which itself is a decorator factory.

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

```
    return dispatch(args[0].__class__)(*args, **kw)
           ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
  File "/home/werner/python/examples/singleDispatch.py", ...
    raise NotImplementedError(
        f'{arg} of type {type(arg).__name__} '
         'not handled by aFunc')
NotImplementedError: BOOM of type str not handled by aFunc
```

Note that for the **bool** type, which hasn't been registered, the version for **int** has been selected — this is understandable, as **bool** inherits from **int**. Similarly, in the penultimate line of the program, we use object of class **Student** and the version for **Person** is invoked, because **Student** hasn't been registered but it inherits from **Person** which was.[1] In the final line, we invoke **aFunc** with a string: the **str** type is not registered and doesn't inherit from any of the registered types, so the default version (for **object**) is called, and raises an exception.

[1] More about classes and inheritance – see chap. **??**, p. **??**.

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

Chapter 6

Comprehensions and generators

Sequences are the basic data structures in Python, and so are iterables and iterators. As we remember, anything that we can pass to the built-in **iter** function and get an iterator *is* iterable. Iterator in turn is something that we can pass to the built-in **next** function and get the next item, or, if there are no more items to return, get the **StopIteration** exception. For the **next** function to work, iterators should provide the **__next__** *method;* if they don't also **__getitem__** (taking a zero based index) will be tried.

Iterators are themselves iterable, i.e., passed to **iter** function, they return iterator, most often just themselves.

Normally, we don't have to call **iter** and **next** "manually". Instead, we use **for**-loop which is really a **while** loop, but does what's needed under the hood:

```
iterable = [1, 2, 3] # lists are iterable
print('for loop:   ', end='')

for e in iterable:
    print(e, end=' ')

## equivalent while loop

print('\nwhile loop: ', end='')

itr = iter(iterable)
while True:
    try:
        e = next(itr)
    except StopIteration:
        break
    else:
        print(e, end=' ')
del itr
```

prints

```
for loop:   1 2 3
while loop: 1 2 3
```

Comprehensions and generators, as loops, are used for performing iteration. They both take items one by one from a source and do something with each of them in turn.

6.1 Comprehensions

Comprehensions[1] in Python provide a concise syntax for creating lists, sets and dictionaries from other iterables in a declarative/functional style.
They always act on existing iterables and iterating over their elements produces results that are then collected to a list, a set, or a dictionary. As loops, they can be nested.

[1]The term borrowed from *set comprehension* in mathematics.

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

It is also possible to define some condition that must be met for the result of a given iteration to contribute to the final collection.

6.1.1 List comprehension

List comprehensions have the form

```
[ expr for var in iterable]
```

where iterable is an iterable (e.g., a list), var is the name of a variable that will hold references to consecutive elements of the iterable, and expr is an expression (usually depending on var) the value of which will become an element of the resulting list. For example

```
>>> iterable = [1, 2, 3, 4, 5, 6]
>>> [val**2 for val in iterable]
[1, 4, 9, 16, 25, 36]
```

or

```
>>> iterable = [ [1, 2], [3], [4, 5, 6] ]
>>> [sum(v) for v in iterable]
[3, 3, 15]
```

After an iterable, one can add a condition which will decide if expr from this iteration will be evaluated and included into the resulting list, or not:

```
>>> [v**2 for v in range(1,10) if v % 2]
[1, 9, 25, 49, 81]
```

Here, we take squares of element of the range (numbers from the interval $[1, 9]$) but only of those which are odd (so v%2 is 1, which is interpreted as True).

Note that the variable used for iteration (in our case v) is local inside the comprehension; it doesn't conflict with possibly existing another variable of the same name in the surrounding scope:

```
>>> v = 'Hello, World'
>>> [v**2 for v in range(1,10) if v % 2]
[1, 9, 25, 49, 81]
>>> v
'Hello, World'
```

Iterations can be nested, and for each of them we can add a condition:

```
>>> ls = [ [1, 2], 'abc', [3, 4, 5, 6,], 'xy' ]
>>> [k for v in ls if isinstance(v, list) for k in v if k%2 == 0]
[2, 4, 6]
```

which is equivalent to

```
ls = [ [1, 2], 'abc', [3, 4, 5, 6,], 'xy' ]
res = []
for v in ls:
    if isinstance(v, list):
        for k in v:
            if k % 2 == 0:
                res.append(k)
print(res)
```

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

and prints

```
[2, 4, 6]
```

In order to get used to comprehensions, let us consider examples.

Example 1
Squares of numbers:

```
>>> [x*x for x in range(2,11)]
[4, 9, 16, 25, 36, 49, 64, 81, 100]
```

Example 2
Squares of even numbers:

```
>>> [x*x for x in range(2, 11) if x % 2 == 0]
[4, 16, 36, 64, 100]
```

Example 3
Different powers of even and odd numbers:

```
>>> [x*x if x%2 else x*x*x for x in range(1,7)]
[1, 8, 9, 64, 25, 216]
```

Note the expression x%2 as a condition — it will be interpreted as **True** for odd numbers.

Example 4
Converting strings to numbers:

```
>>> [float(x) for x in ['3.3' , '1.25e2', 'NaN', '-1', '-INF']]
[3.3, 125.0, nan, -1.0, -inf]
```

Example 5
Selecting only numbers or only strings:

```
>>> [x for x in [1, 'a', 'bc', 1.2e2] if isinstance(x, int | float)]
[1, 120.0]
>>> [x for x in [1, 'a', 'bc', 1.2e2] if isinstance(x, str)]
['a', 'bc']
```

Example 6
Number of digits in huge numbers:

```
>>> from math import factorial
>>> [len(str(factorial(x))) for x in range(100, 111)]
[158, 160, 162, 164, 167, 169, 171, 173, 175, 177, 179]
```

Example 7
Prime numbers <= 31

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

```
>>> from math import isqrt
>>> [2] + [k for k in range(3, 32, 2)
...        if all(k%m != 0 for m in range(2, isqrt(k)+1))]
[2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31]
```

Note: the **all** function takes a predicate and an iterable and yields **True** if, and only if, all elements meet the predicate. The **isqrt** function (*integer square root*) returns for argument `k` the greatest integer `s` such that $s^2 \leqslant k$.

Example 8
Removing spaces:

```
>>> ''.join([x for x in 'to be or not to be' if x != ' '])
'tobeornottobe'
```

Example 9
Common elements of two list:

```
>>> ls1 = [1, 'a', 3, 'b', 5, 'c']
>>> ls2 = [4, 'c', 5, 'f', 8]
>>> [a for a in ls1 if a in ls2]
[5, 'c']
```

Example 10
Max elements of sequences:

```
>>> lst = [[1, 4, 3, 2], [19 , -1, 2], [-1, -20, -6], [5, 9]]
>>> [max(x) for x in lst]
[4, 19, -1, 9]
```

Example 11
Converting a matrix of strings to numbers:

```
>>> ls = [['4', '8'],
...       ['4', '2', '28'],
...       ['1', '12'],
...       ['3', '6', '2']]
>>> [[int(e) for e in row] for row in ls]
[[4, 8], [4, 2, 28], [1, 12], [3, 6, 2]]
```

Note that here the expression is itself a comprehension list.

Example 12
Flattening lists:

```
>>> lst = [[1, 2, 3, 4], [5, 6], [7, 8, 9]]
>>> [ch for ls in lst for ch in ls]
[1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Example 13
Couples with names starting with the same letter:

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

```
>>> boys  =  ['Adam', 'Burt', 'Joe', 'Jim', 'Sam']
>>> girls =  ['Bea', 'Jill', 'Ava', 'Eve']
>>> [(she, he) for she in girls for he in boys if she[0] == he[0]]
[('Bea', 'Burt'), ('Jill', 'Joe'), ('Jill', 'Jim'), ('Ava', 'Adam')]
```

Example 14
Pythagorean triplets with a < b < c <= 50:

```
>>> from math import gcd
>>> def gcd3(a, b, c): return gcd(a, gcd(b, c))
...
>>> [(a, b, c) for a in range(1,49)
...            for b in range(a+1, 50)
...            for c in range(b+1, 51)
...                if a*a + b*b == c*c and gcd3(a, b, c) == 1]
[(3, 4, 5), (5, 12, 13), (7, 24, 25), (8, 15, 17),
(9, 40, 41), (12, 35, 37), (20, 21, 29)]
```

Note that no two triangles are similar to each other.

Example 15
The **getUniqueElems** function below takes any iterable object and returns a list of its elements not changing their order but removing repetitions (and has linear computational complexity):

```
>>> def getUniqueElems(iterable):
...     s = set()
...     return [e for e in iterable
...                 if e not in s and s.add(e) == None]
...
>>> elems = (1, 2, 3, 1, 2, 3, 4, 1)
>>> list(getUniqueElems(elems))
[1, 2, 3, 4]
```

6.1.2 Set comprehension

Set comprehensions are similar to list comprehensions but values of expr are collected to sets (so without duplicates) rather than to lists. Here, we use curly braces instead of square brackets.

Examples:

Example 1
Unique words:

```
>>> ss = ['Warsaw Paris Berlin', 'Praha Warsaw Berlin Caen']
>>> {word for s in ss for word in s.split()}
{'Paris', 'Warsaw', 'Praha', 'Berlin', 'Caen'}
```

Example 2
Common elements of lists:

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

```
>>> list1 = [1, 3, 7, 4]
>>> list2 = [3, 4, 5, 7]
>>> list3 = [4, 5, 6, 7]
>>> {x for x in list1 if x in list2 and x in list3}
{4, 7}
```

Example 3
Set of Polish vowels in a text:

```
>>> string = 'Pozbądź się zbędnych kilogramów'
>>> {x for c in string if (x := c.lower()) in 'aąeęiouy'}
{'ę', 'y', 'ą', 'a', 'i', 'o'}
```

Example 4
Unique letters in a text:

```
>>> string = 'carrot: 20; apples: 30'
>>> {c for c in string if c.isalpha()}
{'s', 'r', 'e', 'c', 'p', 'a', 'l', 't', 'o'}
```

Example 5
Cartesian product of sets:

```
>>> l1 = [1, 2, 3]
>>> l2 = ['a', 'b']
>>> {(x, y) for x in l1 for y in l2}
{(1, 'b'), (3, 'a'), (2, 'a'), (2, 'b'), (1, 'a'), (3, 'b')}
```

Example 6
Set of numbers from a list:

```
>>> vals = [1, 'a', 2, 'b', 3, 'c']
>>> {x for x in vals if isinstance(x, int | float)}
{1, 2, 3}
```

Example 7
Unicode codes of Polish diacritic letters as four-digit hexadecimal numbers:

```
>>> for k, v in {c : f'0x{hex(ord(c))[2:]:0>4}'
...               for c in 'ĄąĆćĘęŁłŃńÓóŚśŹźŻż'}.items():
...     k, v
...
('Ą', '0x0104')
('ą', '0x0105')
('Ć', '0x0106')
('ć', '0x0107')
('Ę', '0x0118')
('ę', '0x0119')
('Ł', '0x0141')
('ł', '0x0142')
```

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

```
('Ń', '0x0143')
('ń', '0x0144')
('Ű', '0x00d3')
('ó', '0x00f3')
('Ś', '0x015a')
('ś', '0x015b')
('Ź', '0x0179')
('ź', '0x017a')
('Ż', '0x017b')
('ż', '0x017c')
```

6.1.3 Dictionary comprehension

Finally, colon separated *pairs* of values of two expressions

```
{ expr1 : expr for var in iterable }
```

will be collected to a dictionary (as for sets, comprehension expression must be enclosed in curly braces).

A few examples:

Example 1
Dictionary number → parity:

```
>>> nums = [1, 2, 3, 4]
>>> {n : 'odd' if n%2 else 'even' for n in nums}
{1: 'odd', 2: 'even', 3: 'odd', 4: 'even'}
```

Example 2
Letter counts in a text:

```
>>> string = 'to be or not to be'
>>> {c : string.count(c) for c in set(string) if c.isalpha()}
{'r': 1, 'n': 1, 'b': 2, 'e': 2, 't': 3, 'o': 4}
```

Example 3
Dictionary name → its length:

```
>>> names = ['Argentina', 'Poland', 'Spain', 'Italy']
>>> {name: len(name) for name in names}
{'Argentina': 9, 'Poland': 6, 'Spain': 5, 'Italy': 5}
```

Example 4
Tuple to dictionary:

```
>>> lst = [('a', 'Alice'), ('b', 'Bea'), ('c', 'Cecilia')]
>>> {k: v for k, v in lst}
{'a': 'Alice', 'b': 'Bea', 'c': 'Cecilia'}
```

Example 5
Marks from scores:

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

```
>>> scores = {'Eve': 83, 'Bob': 47, 'Sue': 65, 'Dave': 77}
>>> {name:
...      '5'  if score >= 90 else
...      '4+' if score >= 80 else
...      '4'  if score >= 70 else
...      '3+' if score >= 60 else
...      '3'  if score >= 50 else '2'
...   for name, score in scores.items()}
{'Eve': '4+', 'Bob': '2', 'Sue': '3+', 'Dave': '4'}
```

Example 6
Filtering a dictionary:

```
>>> dct = {'a': 9, 'b': 12, 'c': 7, 'd': 14}
>>> {k: v for k, v in dct.items() if v <= 10}
{'a': 9, 'c': 7}
```

Example 7
Reversing dictionary:

```
>>> dct = {'Mary': 'John', 'Kate': 'Bill', 'Sophia': 'Kevin'}
>>> {v : k for k, v in dct.items()}
{'John': 'Mary', 'Bill': 'Kate', 'Kevin': 'Sophia'}
```

6.2 Generator functions

Generator is a special kind of objects that can be used to lazily produce a sequence of values. Lazily means that consecutive elements are created and returned only when they are needed, not all of them in advance. Generators are *iterable* — they provide the `__iter__` function which returns an iterator. Main benefits of generators are:

- They pause execution until the next value is demanded; when paused, they do nothing without consuming processor time and therefore are completely "lazy".
- They process only values that have been demanded; values that are not needed will never be even evaluated.
- They can represent streams of data of unlimited size, because they do not store values to be yielded anywhere; when asked for, they process just a single value.

On the other hand

- The user doesn't know the number of values in the remaining stream of data until the generator is exhausted.
- Values yielded by a generator cannot be obtained by indexing or slicing.
- To get a specific item, all preceding values must be fetched.
- You cannot restart or "rewind" a generator

Definition of a generator function looks like a definition of a function. However, if it is a generator, instead of `return`, we use `yield` — by the presence of `yield` statement(s), the interpreter recognizes that this is a generator, not a "plain" function (although `return` may still appear in generator functions). Calling such a "function" behaves

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

differently than calling a "normal" function: invocation returns an object of type **generator** without executing any code!

The returned generator is iterable: it has the `__iter__` method which returns an iterator which in turn has `__next__` method returning consecutive values "returned" by **yield**. Execution of the code starts when the `__next__` method is called for the first time. Every time **yield** is encountered, the value given after this keyword is returned, but the state of the function is "saved" and will be resumed after the next invocation of **next**:

```
>>> def squares():
...     i = 1
...     while True:
...         yield i*i
...         i += 1
...
>>> gen = squares()
>>> type(squares), type(gen)
(<class 'function'>, <class 'generator'>)
>>> hasattr(gen, '__iter__')
True
>>> it = iter(gen)
>>> type(it)
<class 'generator'>
>>> hasattr(it, '__next__')
True
>>> it.__next__()
1
>>> next(it)
4
```

Note how **yield** behaves as **return**, but the execution of the body of the generator is only suspended, and the next invocation of **next** resumes it where it was stopped (until next **yield**). When the flow of control leaves the body of a generator "naturally" or by executing **return**), the **StopIteration** exception is raised:

```
>>> def squares(n):
...     i = 1
...     while i < n:
...         yield i*i
...         i += 1
...
>>> gen = squares(3)
>>> it = iter(gen)
>>> next(it)
1
>>> next(it)
4
>>> next(it)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
StopIteration
```

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

To stop the iteration, one can also use a "normal" **return** statement — then we will also get **StopIteration** exception:

```
>>> def squares():
...     i = 1
...     while True:
...         yield i*i
...         i += 1
...         if i == 3: return
...
>>> gen = squares()
>>> next(gen)
1
>>> next(gen)
4
>>> next(gen)
Traceback (most recent call last):
File "<stdin>", line 1, in <module>
StopIteration
```

Another example — Fibonacci numbers generator:

```
def Fibo(n):
    a, b, count = 0, 1, 0
    while True:
        if (count > n): return
        yield a
        a, b = b, a + b
        count += 1

fib = Fibo(13)
for e in fib:
    print(e, end=' ')
print()
```

prints

```
0 1 1 2 3 5 8 13 21 34 55 89 144 233
```

(note that the **StopIteration** exception has been handled by the **for** loop so we will not see it).

However, there is a difference between using **return** and just exiting the generator. In the former case, we can return *a value,* and it will be remembered as an attribute of the exception object; for example

```
def Fibo():
    a, b, count = 0, 1, 0
    while True:
        if (count > 2): return f'{count=}'
        yield a
        a, b = b, a + b
        count += 1
```

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

```
fib = Fibo()
print(next(fib))
print(next(fib))
print(next(fib))
print(next(fib))
```

prints

```
0
1
1
Traceback (most recent call last):
File "/usr/lib/python3.10/idlelib/run.py", line 578, in runcode
  exec(code, self.locals)
File "/home/werner/p.py", line 13, in <module>
  print(next(fib))
StopIteration: count=3
```

It's very important to understand the difference between iterable collections (like lists) and generator functions. In both cases, we can iterate over elements, but for generators we can do it once only as illustrated by the following program

```
def genl(n):
    v = 0
    while True:
        v += 1
        yield v
        if v == 5: return

lst = [1, 2, 3, 4, 5]
for e in lst: print(e, end=' ') # 1 2 3 4 5
print()
for e in lst: print(e, end=' ') # 1 2 3 4 5
print()

gen = genl(5)
for e in gen: print(e, end=' ') # 1 2 3 4 5
print()
for e in gen: print(e, end=' ') # NOTHING!!!
print()
```

6.2.1 Generator comprehension

Generators can also be created by using the syntax similar to list comprehension; the difference is that now round parentheses are used instead of brackets (this is called **generator expression**):

```
>>> g = (x**2 for x in [2, 4, 7])
>>> type(g)
<class 'generator'>
>>> for i in g:
...     print(i)
```

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

```
4
16
49
```

Note that there is no explicit **yield** statements here and the resulting generator can be iterated over more or less like a list (because it does have methods **__iter__** and **__next__**). But the main advantage of generator is that when a list is created, it is created once with all its elements: generators produce consecutive elements on demand when they are explicitly asked for. Therefore, generators consume virtually no memory and are created very fast.

```
>>> from sys import getsizeof
>>> lst = [x**3 for x in range(10_000_000)]
>>> gen = (x**3 for x in range(10_000_000))
>>> lst.__sizeof__()
89095144
>>> gen.__sizeof__()
88
>>> getsizeof(lst)
89095160
>>> getsizeof(gen)
104
```

As we can see, the **getsizeof** reports slightly larger values than **__sizeof__** method. Actually, it uses the latter internally, but adds some overhead for the garbage collector stuff (experience shows that this is 16 bytes, independently of the object's size).

The example below shows how to create a generator which will generate consecutive Fibonacci numbers:

Listing 30 ABA-GenFib/GenFib.py

```
#!/usr/bin/env python3

def fibonacci(n):
    curr = 0
    prev = 1
    count = 0
    while count <= n:
        yield curr
        prev, curr = curr, prev + curr
        count += 1

fibogen = fibonacci(100)
print(type(fibonacci), type(fibogen))
for i, f in enumerate(fibogen):
    print('{:3d}:{:22d}'.format(i, f))
```

The program prints

```
<class 'function'> <class 'generator'>
  0:                     0
  1:                     1
```

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

```
  2:                    1
  3:                    2
  4:                    3
  5:                    5
  6:                    8
  7:                   13
           ...

 95:  31940434634990099905
 96:  51680708854858323072
 97:  83621143489848422977
 98: 135301852344706746049
 99: 218922995834555169026
100: 354224848179261915075
```

Another example: a generator which reads a text file and yields words read from it but all different, ignoring duplicates:

Listing 31 ABL-UniqueWords/UniqueWords.py

```
#!/usr/bin/env python

def uniqueLines(inpFile):
    prevWords = set()
    with open(inpFile) as file:
        for line in file:
              # get rid of everything but letters and spaces
            line = ''.join([x.lower() for x in line
                            if x.isalpha() or x == ' '])
            for word in line.split():
                if word not in prevWords:
                    prevWords.add(word)
                    yield word           # yielding new word

for w in uniqueLines('UniqueWords.txt'): print(w)
```

If the ***UniqueWords.txt*** file contains

```
Kraków Paris Zürich
Warszawa Berlin
Paris
Kraków Zürich Warszawa
```

the result will be

```
kraków
paris
zürich
warszawa
berlin
```

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

And another example: this time we create a generating function **limiter** which takes, as the first argument, another generating function. Then it calls this function, and iterates over the obtained generator only a prescribed number of times (10 in the example).[1] In this way it will be possible to pass to it potentially infinite generators, for example, as below, a generator **Fibo** generating Fibonacci numbers *ad infinitum,* or **Die** which constantly rolls a die:

Listing 32 ABR-GenOfGen/GenOfGen.py

```
#!/usr/bin/env python

from random import randint

def limiter(generator, n):
    g = generator()
    for i in range(n):
        yield next(g)

def Fibo():
    a, b = 0, 1
    while True:
        yield a
        a, b = b, a+b

def Die():
    while True:
        yield randint(1, 6)

for e in limiter(Fibo, 10):
    print(e, end=' ')
print()

for e in limiter(Die, 10):
    print(e, end=' ')
print()
```

The program prints

```
0 1 1 2 3 5 8 13 21 34
2 1 2 5 1 4 3 3 1 2
```

6.3 Communicating with generators

Generators can not only return values but also receive them. We send values (objects) to a generator by invoking the **send** method on it. This method sends a value *to* a generator and also returns a value yielded *from* this generator. In order to receive a value sent by the **send** function, execution of the generator function must be stopped on a line containing the **yield** *expression* — expression, so, for example, on the right-hand side of an assignment. Such generators are sometimes called **coroutines**.

[1] Somewhat like **Stream::limit** in Java.

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

To achieve this situation, we have to "prime" the generator by invoking **next** on it (or sending **None** by its **send** method).

Suppose, there is a **yield** expression of the form `yield expr` on the line where the generator is stopped. Then, after the invocation of **send(v)**:

- The value of v becomes the result of the current `yield expr` expression (but *not* of the expr itself!);[1]
- The execution continues, until the `yield expr` expression is encountered again (or the **StopIteration** exception is raised, if the generator exits without yielding another value);
- Current value of expr is returned to the caller;
- Execution of the generator is again suspended.

Let us consider an example which will show all this in action. The example is somewhat contrived, but will allow us to see the order of events here.

We define a coroutine **twoWayGener** with one **yield** *expression* (❷).

Listing 33 ABO-TwoWayGener/TwoWayGener.py

```
1  #!/usr/bin/env python
2
3  def twoWayGener():
4      print("Gene: started")
5      value = None
6      while True:
7          print(f'Gene: ready to receive, {value=}')   ❶
8          x = yield value                              ❷
9          print(f'Gene: resumed, {value=}, {x=}')      ❸
10         value = 2*x                                  ❹
11
12 gen = twoWayGener()                                  ❺
13 print('Main: priming')
14   # or gen.send(None) - priming
15 ret = next(gen)                                      ❻
16 print(f'Main: primed:  {ret=}\n')                    ❼
17 for n in [1, 3]:
18     print(f'Main: sending  {n=}')
19     ret = gen.send(n)                                ❽
20     print(f'Main: received {ret=}\n')
```

The program prints

```
Main: priming
Gene: started
Gene: ready to receive, value=None
Main: primed:  ret=None
```

[1] The value of the expression `expr` and the value of the expression `yield expr` are two different things!

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

```
Main: sending  n=1
Gene: resumed, value=None, x=1
Gene: ready to receive, value=2
Main: received ret=2

Main: sending  n=3
Gene: resumed, value=2, x=3
Gene: ready to receive, value=6
Main: received ret=6
```

In line ❺, we call this coroutine and what we get is a generator gen. Note that nothing is printed — no line of the twoWayGener function has been executed.

In line ❻, we "prime" the generator using next (what is equivalent to sending None). Its code is executed, prints "Gene: started", and stops at line ❷, yielding the current value of the variable value, which is None, to the caller. This value is printed by the caller in line ❼.

Now the caller sends 1 (❽). It becomes the value of the expression (yield value) (❷) and is assigned to the x variable. Note that the variable value is still None. We can see it from the result of the print statement in line ❸. Execution of the coroutine continues, value becomes 2, is printed in line ❶, and then, in line ❷, its value is returned to the caller and the coroutine pauses again.

Then the value 3 is sent (❽) and an analogous sequence of events happens: x is assigned the value 3, value is still 2, they are printed, then value becomes 6, is printed and returned in line ❷, and the coroutine pauses again.

The next example will be a little more complex. We build a generator which yields random elements from a list passed as the argument. Moreover, by sending non-None values to the generator, we can append new elements to the list:

Listing 34 ABP-Chooser/Chooser.py

```
1  #!/usr/bin/env python
2
3  from random import choice
4
5  def chooser(aList):
6      item = None
7      while True:
8          if item != None:
9              if item not in aList:
10                 aList.append(item)              ❶
11             item = yield item                   ❷
12         else:
13             item = yield choice(aList)          ❸
14
15 ch = chooser(['Joe', 'Ben', 'Alice'])
16 # priming
17 next(ch)                                        ❹
18
19 for i in range(3):
20     print(next(ch)) # equivalently: print(ch.send(None))
```

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

```
21
22 print('*** Adding Eve and Kenny to the list')
23 ch.send('Eve')                                    ❺
24 ch.send('Kenny')                                  ❻
25
26 for i in range(4):                                ❼
27     print(next(ch)) # equivalently: print(ch.send(None))
```

prints, for example (specific values are random):

```
Alice
Alice
Ben
*** Adding Eve and Kenny to the list
Kenny
Joe
Eve
Alice
```

Let us analyse what's happening here. At first item is None, so, after priming (line ❹), the generator goes to the line ❸ and stops.
Then, in a loop, we send None (next is equivalent to sending None) to the generator: the value of the expression `yield choice(aList)` is therefore None, generator resumes, assigns this None to item and goes again to the line ❸, yields the value of `choice(aList)` to the caller, and is suspended again. And this is repeated three times.

Now, we send `'Eve'` (line ❺).
This value becomes the value of the expression `yield choice(aList)` which is assigned to item making it not-None. Execution resumes, goes this time to line ❶, appends `'Eve'` to the list, proceeds to line ❷, returns this `'Eve'` to the caller (who ignores it) and is suspended. Now we send `'Kenny'` (❻). This becomes the value of the expression `yield item` and is assigned to item, so we again reach the lines ❶ and ❷. The next value sent to the generator will be None (from the loop in line ❼), so item will become None again and we will proceed to line ❸and a new random element of the list will be returned. And this will be repeated 4 times.

Having to "prime" generators before sending data to them may be cumbersome and it's easy to forget about it. The standard method to handle this problem is by using a decorator which takes a coroutine and returns it already primed, as illustrated below. To clarify the logic, the program prints some additional information.

The longlines coroutine accepts texts sent to it and prints only those which contain more than n words. In order to use it without priming it first, we decorate it with the coroutine decorator: it just calls the generating function to get the generator, calls next on it once, and returns the generator that doesn't need priming any more:

Listing 35 ABV-Priming/Priming.py

```
1 #!/usr/bin/env python
2
3 import functools
```

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

```
4
5  def coroutine(func):
6      print('** 2 ** Executing coroutine')
7      @functools.wraps(func)
8      def primed(*args, **kwargs):
9          p = func(*args, **kwargs)     # p is a generator
10         print('** 4 ** From primed')
11         next(p)     # priming
12         return p    # returning already primed generator
13     return primed
14
15 print('** 1 ** Defining longlines')
16
17 @coroutine
18 def longlines(n):
19     """Select lines with more than n words."""
20     print('** 5 ** Longlines advances to yield')
21     while True:
22         line = yield
23         if len(line.split()) > n: print('Got long line:', line)
24
25 print('** 3 ** Main starting, creating generator')
26 gen = longlines(3)
27 print(type(gen))
28 print(type(longlines), longlines.__name__, longlines.__doc__, '\n')
29
30 gen.send("one two")
31 gen.send("üks kaks kolm neli")
32 gen.send("eins zwei drei")
33 gen.send("jeden dwa")
34 gen.send("moja mbili tatu nne tano")
35 gen.send("un deux trois")
36 gen.send("uno dos")
37 gen.send("uno due tre quattro")
38
39 gen.close()
40 print('\n** 6 ** Generator closed\n')
41
42 gen.send("ichi ni san")
```

The program prints

```
** 1 ** Defining longlines
** 2 ** Executing coroutine
** 3 ** Main starting, creating generator
** 4 ** From primed
** 5 ** Longlines advances to yield
<class 'generator'>
<class 'function'> longlines Select lines with more than n words.
```

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

```
Got long line: üks kaks kolm neli
Got long line: moja mbili tatu nne tano
Got long line: uno due tre quattro

** 6 ** Generator closed

Traceback (most recent call last):
File "/home/werner/python/py_progs/sources/ABV-Priming/Priming.py",
        line 42, in <module>
    gen.send("ichi ni san")
    ~~~~~~~~~~~~~~~~~~~~~~
StopIteration
```

We also illustrated the method **close()** which closes the generator: attempting to send more information to it raises an exception.

6.4 Pipes of generators and coroutines

Generators' and coroutines' ability to receive and send information allows us to built out of them configurable "pipes" of actions where output of one of them is input of the next. It somehow resembles streams with their intermediate and terminal operations in Java.

Let us first consider such a pipe of coroutines. They will receive consecutive pieces of information (using **yield** expressions) from the previous coroutine and, after some kind of processing, "push" them, (using **send**) to the next. Of course, we have to start from generating initial stream of data which will be pushed into the pipe, and eventually end the whole procedure with a "sink" — something that consumes data but doesn't pass it any further.

Let us consider and example. In the program below

Listing 36 ABW-PushPipe/PushPipe.py

```
#!/usr/bin/env python

import functools

  # decorator priming coroutines
def coroutine(f):
    @functools.wraps(f)
    def primed(*args,**kwargs):
        corou = f(*args,**kwargs)
        next(corou)
        return corou
    return primed

  # passes only n items
@coroutine
def limi(n, target):
    while True:
        item = (yield)
```

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

```
19          if n > 0:
20              print('lim'+str(item), end=' ')
21              target.send(item)
22          n -= 1
23
24    # applies consumer, passes everything unchanged
25  @coroutine
26  def peek(consumer, target):
27      while True:
28          item = (yield)
29          consumer(item)
30          target.send(item)
31
32    # passes only what meets predicate
33  @coroutine
34  def filt(predicate, target):
35      while True:
36          item = (yield)
37          if predicate(item):
38              print('fil'+str(item), end=' ')
39              target.send(item)
40
41    # passes everything mapped by func
42  @coroutine
43  def mapp(func, target):
44      while True:
45          item = (yield)
46          print('map'+str(item), end=' ')
47          target.send(func(item))
48
49    # pushes data to pipe
50  def gen(source, target):
51      for e in source: target.send(e)
52
53    # sink: appends everything to list
54  @coroutine
55  def toList(lst):
56      while True:
57          item = (yield)
58          print("sink" + str(item), end=' ')
59          lst.append(item)
60
61
62  src = (x for x in range(1, 7))      # for initial generator
63  pred = lambda n: not n%2            # for filtering
64  func = lambda x: 'Res'+str(x)       # for mapping
65  cons = lambda x: print('peek'+str(x), end=' ')  # for peek
66  lst = []                            # for toList
67
68    # either this...
```

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

```
69 crMapp = mapp(func, toList(lst))
70 crFilt = filt(pred, crMapp)
71 crLimi = limi(4, crFilt)
72 crPeek = peek(cons, crLimi)
73 gen(src, crPeek)                    # starting the pipe
74
75   # or this
76 #gen(src, peek(cons, limi(4, filt(pred, mapp(func, toList(lst))))))
77
78 print('\nResulting list:', lst)
```

- The **gen** function iterates over an iterable source (defined on line 62) and sends elements, one by one, to target. In the example, this target will be the **peek** coroutine.[1]
- The **peek** coroutine applies the **consumer** function (defined on line 65) to all elements, but otherwise sends them intact to the next coroutine, which is **limi**.
- The **limi** coroutine passes only the first n elements (4 in the example), ignoring all the remaining, to **filt**.
- The **filt** coroutine takes a predicate (defined on line 63) and rejects all elements which do not meet it; those which do are passed to **mapp**.
- The **mapp** coroutine applies the **func** function (defined on line 64) to all elements and sends the results to **toList**.
- The **toList** coroutine is the sink: it just appends all arriving elements to a list, which was passed to it as the argument.

We can compose one chain of operations as on line 76, or to make it more clear, split the problem into individual operations, as on lines 69-73 (which should be read backwards).

In both cases, the result printed is (the first two lines are in fact one line)

```
peek1 lim1 peek2 lim2 fil2 map2 sinkRes2 peek3 lim3
    peek4 lim4 fil4 map4 sinkRes4 peek5 peek6
Resulting list: ['Res2', 'Res4']
```

(as we can see, elements 5 and 6 were pushed to **peek**, although they were not needed because of **limi**.)

Let us now consider the same chain of operations, but using generators "pulling" data from the previous ones.

Listing 37 ABX-PullPipe/PullPipe.py

```
1 #!/usr/bin/env python
2
3 import functools
4
5   # passes only n items
6 def limi(n, source):
```

[1] All coroutines are primed by the **coroutine** decorator.

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

```
    for _ in range(n):
        item = next(source)
        print('lim'+str(item), end=' ')
        yield item

  # applies consumer, passes everything unchanged
def peek(consumer, source):
    for item in source:
        consumer(item)
        yield item

  # passes only what meets predicate
def filt(predicate, source):
    for item in source:
        if predicate(item):
            print('fil'+str(item), end=' ')
            yield item

  # passes everything mapped by func
def mapp(func, source):
    for item in source:
        print('map'+str(item), end=' ')
        yield func(item)

  # pushes data to pipe
def gen(source, target):
    for e in source:
        target.send(e)

  # sink: appends everything to list
def toList(source):
    lst = []
    for item in source:
        print("sink" + str(item), end=' ')
        lst.append(item)
    return lst


src = (x for x in range(1, 7))      # for initial generator
pred = lambda n: not n%2            # for filtering
func = lambda x: 'Res'+str(x)       # for mapping
cons = lambda x: print('peek'+str(x), end=' ')  # for peek
lst = []                            # for toList

  # either this
grPeek = peek(cons, src)
grLimi = limi(4, grPeek)
grFilt = filt(pred, grLimi)
grMapp = mapp(func, grFilt)
res = toList(grMapp)                # starting the pipe
```

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

```
57
58    # or this
59  #res = toList(mapp(func, filt(pred, limi(4, peek(cons, src)))))
60
61  print('\nResulting list:', res)
```

This time we start from the other end:

- The **toList** function collects all elements in the list, pulling data from **mapp**.
- The **mapp** generator applies the **func** function (defined on line 47) to all elements pulled from the source, which is **filt**.
- The **filt** generator takes a predicate (defined on line 46) and rejects all elements which do not meet it; it pulls data from **limi**.
- The **limi** generator passes only the first `n` elements (4 in the example), ignoring all the remaining; it pulls data from **peek**.
- The **peek** generator applies the **consumer** function (defined on line 48) to all elements, but otherwise resends them intact; it pulls data from the source defined as a generator on line 45.

Again, we can compose one chain of operations as on line 59, or to make it more clear, split the problem into individual operations, as on lines 52-56 (which should again be read backwards).

In both cases, the result printed is (the first two lines are in fact one line)

```
peek1 lim1 peek2 lim2 fil2 map2 sinkRes2 peek3 lim3
    peek4 lim4 fil4 map4 sinkRes4
Resulting list: ['Res2', 'Res4']
```

(as we can see, this time elements 5 and 6 were *not* pulled by **peek**, because **limi** demanded only 4 elements.)

6.5 Some tools from itertools

The ***itertools*** module contains several tools that one can use when dealing with iterables (and returning iterables). One important example was already described, namely the **accumulate** function (see sec. 5.4.4, p. 87).

Other useful functions from this module are **dropwhile** and **takewhile**. They both take a predicate and an iterable and return an iterable. The first just *ignores* elements from the beginning of the input iterable as long as they meet the predicate, while the second includes *only* these elements:

```
>>> from itertools import dropwhile, takewhile
>>> ls = [1, 2, 3, 4, 5, 6]
>>>
>>> drop = dropwhile(lambda x: x < 4, ls)
>>> type(drop)
<class 'itertools.dropwhile'>
>>> drop
<itertools.dropwhile object at 0x7f8d1050ddc0>
>>> list(drop)
[4, 5, 6]
```

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

```
>>>
>>> take = takewhile(lambda x: x < 5, ls)
>>> type(take)
<class 'itertools.takewhile'>
>>> take
<itertools.takewhile object at 0x7f8d1050e080>
>>> list(take)
[1, 2, 3, 4]
```

Note that the **takewhile** function may be useful when dealing with potentially infinite iterables and we need to ensure that the iteration will stop at some point.

Somewhat related to the two previous function is the **islice** function. It allows us to select from an iterable only some of the yielded elements. Its syntax is similar to that of **range** (and also to slicing)

```
itertools.islice(iterable, stop)
itertools.islice(iterable, start, stop[, step])
```

The function returns an iterable which will return only items selected by specifying start, step and step.
Invoking it with one argument (besides the iterable), it is equivalent to

```
itertools.islice(iterable, 0, stop, 1)])
```

i.e., first stop elements will be selected (we could say *from index 0 inclusive to index* stop *exclusive,* but of course the input iterable is not necessarily indexable).
With two arguments (besides the iterable), it is equivalent to

```
itertools.islice(iterable, start, stop)])
```

i.e., first start elements will be skipped, and the next stop−start selected; stop may be **None**, which means *until the input iterable is exhausted.* (we could thus say *from index* start *inclusive to index* stop *exclusive*).
In both previous cases, the step is assumed to be 1, i.e., all elements from a given range are selected. As is the case for **range** (and slices), setting step to a value greater than 1, say n, selects every n-th item; we then have to use three arguments, although both may be **None** (for start it's equivalent to *from the beginning,* for stop – to *to the end*).
For example:

Listing 38 ABM-Islice/Islice.py

```
#!/usr/bin/env python

from sys import maxsize
from itertools import islice

iterable = '0123456789'

for e in islice(iterable, 5):
    print(e, end=' ')   # 0 1 2 3 4
print()
```

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

```
11
12 for e in islice(iterable, 2, 8):
13     print(e, end=' ')   # 2 3 4 5 6 7
14 print()
15
16 for e in islice(iterable, None, None, 3):
17     print(e, end=' ')   # 0 3 6 9
18 print()
19
20
21   # 'almost infinite' sequence of primes
22 iterable = (k for k in range(2, maxsize)
23             if all(k%m != 0 for m in range(2, int(k**0.5)+1)))
24   # first 10 primes
25 for e in islice(iterable, 10):
26     print(e, end=' ')   # 2 3 5 7 11 13 17 19 23 29
27 print()
28
29   # we have to recreate the iterable, because
30   # generators cannot be 'rewound'!
31 iterable = (k for k in range(2, maxsize)
32             if all(k%m != 0 for m in range(2, int(k**0.5)+1)))
33  # 100-th to 105-th primes
34 for e in islice(iterable, 100, 106):
35     print(e, end=' ')   # 547 557 563 569 571 577
36 print()
37
38 iterable = (k for k in range(2, maxsize)
39             if all(k%m != 0 for m in range(2, int(k**0.5)+1)))
40  # every third prime number from 1000-th to 1021-th
41 for e in islice(iterable, 1000, 1022, 3):
42     print(e, end=' ')   # 7927 7949 7993 8017 8059 8087 8101 8123
43 print()
```

The **count** is another useful function. It is similar to **range**, but produces always *infinite* sequence starting from start by step; both stop and step do not have to be integer, as they don't correspond to "indices"

```
itertools.count(start=0, step=1)
```

For example

```
>>> from itertools import count, takewhile, islice
>>>
>>> cnt = count()
>>> type(cnt)
<class 'itertools.count'>
>>> list( takewhile(lambda x: x < 10, cnt) )
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
>>>
```

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

```
>>> list( islice(count(1.5, 2.5), 100, 106) )
[251.5, 254.0, 256.5, 259.0, 261.5, 264.0]
```

Keep in mind that **count** produces infinite stream of items, so we always have to ensure that only finite number of elements will remain, for example, as in the example above, by using **takewhile** or **islice**.

As the last function from the *itertools* module, we will briefly describe the **groupby** function

```
itertools.groupby(iterable, key=None)
```

It takes an iterable and a key function, similar to what we use in the **sorted**/**sort** functions. This key function should yield, for elements of the input iterable, values which are then used as the keys by which the elements are to be grouped (by default this is just identity function). Elements corresponding to the same key value that should fall into the same group must be adjacent; therefore, very often, the iterable must be first sorted with same key function.
The function returns an iterable yielding pairs with a group key as the first element and an iterable of this group's elements as the second one.
For example

Listing 39 ABN-Groups/Groups.py

```
#!/usr/bin/env python

from itertools import groupby

lst = ['frog', 'goat', 'France', 'Greg', 'floor']

  # key function: by first letter case insensitively
keyfun = lambda s: s.lower()[0]

  # sorting
lst.sort(key=keyfun)

print(lst, end='\n\n') # ['frog', 'France', 'floor', 'goat',
 ↪  'Greg']

groups = groupby(lst, key=keyfun)
print(type(groups), end='\n\n')  # <class 'itertools.groupby'>

for k, v in groups:
    print(type(v))
    print(f"Group '{k}' -> {list(v)}\n")
```

The program prints

```
['frog', 'France', 'floor', 'goat', 'Greg']

<class 'itertools.groupby'>
```

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

```
<class 'itertools._grouper'>
Group 'f' -> ['frog', 'France', 'floor']

<class 'itertools._grouper'>
Group 'g' -> ['goat', 'Greg']
```

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

Listings

List of listings

1 AAB-First/First.py . . . 12
2 AAC-Refs/Refs.py . . . 13
3 AAY-Funcs/funcs.py . . . 14
4 AAM-Pass/Pass.py . . . 21
5 AAG-Iffs/Iffs.py . . . 22
6 AAJ-While/While.py . . . 23
7 AAK-For/For.py . . . 24
8 AAN-DictLoop/DictLoop.py . . . 25
9 AAL-ForEnum/Enum.py . . . 26
10 AAO-EnumStart/EnumStart.py . . . 26
11 AAQ-MatchBasic/MatchBasic.py . . . 28
12 AAT-MatchLen/MatchLen.py . . . 29
13 AAV-MatchVS/MatchValStr.py . . . 30
14 AAX-MatchSN/MatchSeqNam.py . . . 31
15 ABF-MatchT/MatchTypeAs.py . . . 32
16 ABB-Cond/Cond.py . . . 61
17 AAP-FunCompos/Compos.py . . . 68
18 AAR-FunCount/Counter.py . . . 69
19 AAS-Scopes/Scopes.py . . . 71
20 AEB-Closures/Closures.py . . . 72
21 AAU-DefArgs/DefArgs.py . . . 75
22 AAW-ArgsKwargs/ArgsKwargs.py . . . 77
23 AAZ-Filter/Filter.py . . . 84
24 ACP-UnDeco/undeco.py . . . 93
25 ABS-Deco/Deco.py . . . 95
26 ABT-DecoNest/DecoNest.py . . . 96
27 ABU-DecoParam/DecoParam.py . . . 97
28 ABY-DecoTime/DecoTime.py . . . 98
29 ACN-SingDis/singleDispatch.py . . . 99
30 ABA-GenFib/GenFib.py . . . 114
31 ABL-UniqueWords/UniqueWords.py . . . 115
32 ABR-GenOfGen/GenOfGen.py . . . 116
33 ABO-TwoWayGener/TwoWayGener.py . . . 117
34 ABP-Chooser/Chooser.py . . . 118
35 ABV-Priming/Priming.py . . . 119
36 ABW-PushPipe/PushPipe.py . . . 121
37 ABX-PullPipe/PullPipe.py . . . 123
38 ABM-Islice/Islice.py . . . 126
39 ABN-Groups/Groups.py . . . 128

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

Index

** (operator), 12
= (operator), 43

ABC (language), 1
accumulate (function), 83, 87
accumulator, 86
all (function), 106
alternative (logical), 39
and (operator), 39, 59
annotation, 91
argument
    default, 75
arithmetic comparison operator, 57
arithmetic operator, 54
arity, 101
assignment
    chained, 43
assignment (operator), 43, 44
await expression, 53

bin (function), 37, 63
bit_length (function), 11
bitwise and (operator), 65
bitwise not (operator), 65
bitwise operator, 63
bitwise or (operator), 65
bitwise xor (operator), 65
bool (class), 61
bool (type), 38
boolean operator, 59
break (instruction), 23

C/C++, 1
__call__ method, 52
call (operator), 52
callable (function), 52
chained assignments, 43
class
    bool, 61
    slice, 50
close (function), 121
closure, 71, 81
cmp_to_key (function), 90
Collatz sequence, 61
comparison operator, 57
complex (type), 7, 41
comprehension, 103
    dictionary, 109
    generator, 113
    list, 104
    set, 107
conditional expression, 22, 61
conjunction (logical), 39
continue (instruction), 23
copy (function), 52
coroutine, 116
    pipe, 121
count (function), 127
CWI, 1

data type, 34
Decimal, 34
decorator, 91
    nested, 96
deepcopy (function), 14, 52
default argument, 75
dict (type), 9
dictionary comprehension, 109
Dijkstra, E., 23
dir (function), 19
division (operator), 55
dropwhile (function), 125
duck typing, 16, 67
dunder method, 52
dynamic typing, 1

ellipsis type, 35
enumerate (function), 25
enumerate object, 25
exception
    StopIteration, 24, 103, 111
exponentiation (operator), 12, 53
expression
    await, 53
    conditional, 22, 61
    if-else, 22
    lambda, 79
    parenthesized, 46

False, 38
falsy, 39
filter (function), 83, 84
filterfalse (function), 85
find (function), 49
float (type), 7, 39
floor division, 55

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

flow control statement, 21
for loop, 24
Fraction, 34
frozenset (type), 9
function, 14, 67
    __getitem__, 24
    __iter__, 24
    __next__, 24
    accumulate, 83, 87
    all, 106
    bin, 37, 63
    bit_length, 11
    callable, 52
    close, 121
    cmp_to_key, 90
    copy, 52
    count, 127
    decorator, 91
    deepcopy, 14, 52
    dir, 19
    dropwhile, 125
    enumerate, 25
    filter, 83, 84
    filterfalse, 85
    find, 49
    generator, 103
    getsizeof, 114
    groupby, 128
    hex, 37
    higher order, 68
    id, 10
    index, 48
    indices, 51
    input, 3
    islice, 126
    isqrt, 106
    iter, 24, 103
    map, 83, 85
    next, 24
    oct, 37
    pow, 53
    print, 4
    range, 26
    reduce, 83, 86
    sort, 83, 88
    sorted, 79, 83, 88
    takewhile, 125
    to_bytes, 65
    type, 10
    variadic, 76
    wraps, 93
function overloading, 99
functools (module), 99
funkcja
    next, 103

generator
    pipe, 121
generator comprehension, 113
generator expression, 113
generator function, 103, 110
__getitem__ (function), 24
getsizeof (function), 114
groupby (function), 128

hailstone sequence, 61
Haskell, 2, 79
hex (function), 37
higher order function, 68

id (function), 10
identity comparison operator, 58
IDLE, 5
if (statement), 22
if-else (expression), 22
immutable, 12
import, 16
in (operator), 49, 59
indentation, 4
index (function), 48
indexing, 47
indices (function), 51
infinity, 40
input (function), 3
instruction
    break, 23
    continue, 23
int, 36
int (type), 7
is (operator), 13, 58
is not (operator), 58
islice (function), 126
isqrt (function), 106
__iter__ (function), 24
iter (function), 24, 103
iterable, 103
iterable object, 24
iterator, 24, 103
itertools (module), 125

Java, 1

keyword, 20

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

soft, 20
keyword arguments, 74

lambda, 79
lazy evaluation, 110
LEGB rule, 70
Lisp, 2, 79
list, 13
list (type), 7
list comprehension, 104
loop
for, 24
while, 23

magic method, 52
map (function), 83, 85
match statement, 28
membership test operator, 59
memoization, 94
method
__call__, 52
dunder, 52
magic, 52
ML, 2
Modula-3, 1
module
itertools, 125
modulo (operator), 56
mutable, 13

NaN, 40
negation (logical), 39
nested decorator, 96
__next__ (function), 24
next (function), 24
next (funkcja), 103
None, 34
NoneType type, 34
nonlocal, 69
not (operator), 39, 59
not a number, 40
not in (operator), 49, 59
NotImplementedType (type), 35

oct (function), 37
operator, 43
**, 12
and, 39, 59
arithmetic, 54
arithmetic comparison, 57
assignment, 43, 44
bitwise, 63
bitwise and, 65
bitwise not, 65
bitwise or, 65
bitwise xor, 65
boolean, 59
call, 52
comparison, 57
division, 55
exponentiation, 12, 53
identity comparison, 58
in, 49, 59
is, 13, 58
is not, 58
membership test, 59
modulo, 56
not, 39, 59
not in, 49, 59
or, 39, 59
precedence, 45
remainder, 56
shift, 63
slicing, 47
subscription, 47
unary, 54
walrus, 44
or (operator), 39, 59
overloading functions, 99

parenthesized expression, 46
pass statement, 21
PEP, 6
8, 6
20, 5
636, 33
Peters, Tim, 88
pipes, 121
positional parameter, 74
pow (function), 53
precedence, 45
print (function), 4
PyPI, 2
Python shell, 2

range (function), 26
range (type), 8
raw string, 42
recursive lambdas, 83
reduce (function), 83, 86
remainder (operator), 56
REPL, 2
van Rossum, Guido, 1, 79

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych

scope, 69
set (type), 9
set comprehension, 107
shebang, 11
shift (operator, 63
single dispatch, 99
slice, 49
slice (class), 50
slicing, 47
soft keyword, 20
sort (function), 83, 88
sorted (function), 79, 83, 88
statement
    for loop, 24
    if, 22
    match, 28
    pass, 21
    while loop, 23
StopIteration, 111
StopIteration (exception), 24, 103
str (type), 41
string
    raw, 42
    triple quote, 42
strong typing, 1
subscription (operator), 47

takewhile (function), 125
to_bytes (function), 65
triple quote string, 42
True, 38
truthy, 39
tuple (type), 8
type
    bool, 38
    complex, 7, 41
    dict, 9
    ellipsis, 35
    float, 7, 39
    frozenset, 9
    int, 7, 36
    list, 7
    NoneType, 34
    NotImplementedType, 35
    range, 8
    set, 9
    str, 41
    tuple, 8
type (function), 10
typing
    dynamic, 1
    strong, 1

unary operator, 54

variadic function, 76

walrus (operator), 44
while loop, 23
wraps (function), 93

© Tomasz R. Werner
© Polish-Japanese Academy of Information Technology
© Polsko-Japońska Akademia Technik Komputerowych